

Bölüm 1: Oyun Kuramı Temelleri

Algoritmik Oyun Kuramı

Temel kavramlar

Normal ve genişletilmiş form

Baskın strateji ve Nash dengesi

Mahkûmun ikilemi

2/3 ortalaması oyunu

Kombinatorial oyunlar ve minimaks

Mekanizma tasarımına bakış

Sercan KÜLCÜ · Giresun Üniversitesi · Bilgisayar Mühendisliği

Öğrenme Çıktıları

Kavramlar

Oyuncu, strateji, kazanç, bilgi, rasyonellik ve ortak bilgiyi bir örnek üzerinde tanımlamak.

Temsil

Normal form (matris) ve genişletilmiş form (ağaç); sıfır toplamı ve kombinatorial oyunları tanımak.

Çözüm

En iyi tepki, baskın strateji eleme ve saf Nash aramasını elle ve Python ile yapmak.

Klasik oyunlar

Mahkûmun İkilemi, Cinsiyetler Savaşı, Geyik Avı, Tavuk, Yazı-Tura'nın dengelerini okumak.

Algoritmalar

Minimaks, geriye doğru çıkarım ve Nim'in XOR stratejisiyle küçük oyunları çözmek.

Tasarım

Seviye-k akıl yürütmeyi, ikinci fiyat ve medyan kurallarının neden doğru söylediğini açıklamak.

Oyun Kuramı (Game Theory) Nedir?

- Karar vericilerin **stratejik etkileşim** içinde olduğu durumları inceleyen matematiksel bir araçtır: benim en iyi kararım, senin ne yapacağına bağlıdır.
- Karar vericilere **oyuncu** (ajan) denir; nasıl davrandıklarını ve sonucun nasıl şekillendiğini açıklamaya çalışır.
- Oyuncuların **rasyonel** ve kendi çıkarlarına odaklı hareket ettiği varsayılır.
- Ekonomi, siyaset, biyoloji, psikoloji ve bilgisayar biliminde yaygın kullanılır.

Ekonomi

Şirketlerin fiyat rekabeti, açık artırmalar, pazarlık.

Bilgisayar ağları

Her paketin/sürücünün kendi yolunu seçtiği bencil yönlendirme.

Yapay zekâ

Çoklu ajan sistemleri, oyun oynayan programlar, öğrenen ajanlar.

Algoritmik oyun kuramı iki soru ekler: Dengeyi *hesaplamak* ne kadar zor? Bencil ajanlar arasında iyi sonuç veren *kuralları* (algoritma, protokol, açık artırma) nasıl tasarlarız?

Kısa Tarihçe: Oyun Kuramından Algoritmik Oyun Kuramına

Yıl	Gelişme
1913	Zermelo: satranç gibi sonlu, mükemmel bilgili oyunlarda bir taraf kazanmayı ya da iki taraf beraberliği garanti eder.
1928	von Neumann: iki kişilik sıfır toplamlı oyunlar için minimaks teoremi.
1944	von Neumann ve Morgenstern, <i>Theory of Games and Economic Behavior</i> : oyun kuramı ayrı bir alan olur.
1950	Nash: her sonlu oyunda denge vardır. Aynı yıl RAND'daki Flood–Dresher deneyi; Tucker Mahkûmun ikilemi hikâyesini anlatır.
1961	Vickrey: ikinci fiyat açık artırmada dürüst teklif baskındır.
1994	Nash, Harsanyi ve Selten'e Nobel ekonomi ödülü.
1999	Koutsoupias–Papadimitriou (en kötü denge, Anarşinin Bedeli) ve Nisan–Ronen (algoritmik mekanizma tasarımı): algoritmik oyun kuramı doğar.
2006	Nash dengesi bulmak PPAD-tam (Daskalakis–Goldberg–Papadimitriou; iki oyuncu: Chen–Deng).
2012, 2020	Nobel: Roth ve Shapley (kararlı eşleştirme, piyasa tasarımı); Milgrom ve Wilson (açık artırma tasarımı).

Bir Oyunun Bileşenleri

Oyuncular

Karar alan kişi veya kurumlar.

$$N = \{1, \dots, n\}$$

Eylemler / Stratejiler

Her oyuncunun seçebileceği eylemler kümesi.

$$S_i = \{\text{Taş}, \text{Kağıt}, \text{Makas}\}$$

Kazanç (payoff)

Her sonucun oyuncu için değeri; *herkesin* seçimine bağlıdır.

$$u_i(s_1, \dots, s_n)$$

Bilgi

Kim neyi, ne zaman biliyor?
Rakibin hamlesini görüyor musun?

satranç $\neq$ poker

Kurallar ve sıra

Eşzamanlı mı, sırayla mı?
Kazançları eylemlere bağlayan kurallar.

normal / genişletilmiş form

Sonuç

Tüm oyuncuların seçimlerinin bir araya gelmesiyle oluşur.

$$s = (s_1, \dots, s_n)$$

Oyun: oyuncuların etkileşime girdiği, her birinin seçiminin herkesin kazancını etkilediği durum.

Oyuncular ve Rasyonellik

- **Rasyonel:** Her oyuncu, inançlarına göre kendi beklenen faydasını en büyükleyen seçimi yapar.
- **Kendi çıkarı:** Oyuncu kendi fayda fonksiyonunu izler; başkalarının kazancı ancak kendi faydasına girerse önemlidir.
- **Ortak bilgi** (common knowledge): Herkes rasyonel, herkes bunu biliyor, herkes herkesin bildiğini biliyor, ...

2/3 oyununda her bilgi düzeyi bir aralığı eler

Düzy 1: herkes rasyonel
kimse 60'tan büyük yazmaz

Düzy 2: herkes bunu bilir
kimse 40'tan büyük yazmaz

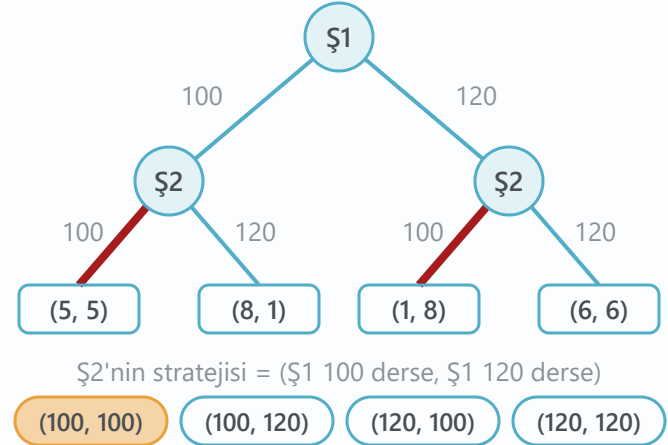
Düzy 3: bunu bildiğini de bilir
26,7'nin üstü de elenir

Ortak bilgi: ... sonsuza dek
yalnızca 0 kalır

Rasyonel $\neq$ bencil: fayda fonksiyonu adalet, özgeçilik ya da itibar kaygılarını da içerebilir. Sonuç, hiçbir oyuncunun tek başına kontrol edemediğı bir **etkileşimin** ürünüdür.

Eylem ve Strateji

- **Strateji:** Oyuncunun seçtiği eylem planı. Sıralı oyunlarda, *her* olası durumda ne yapacağını söyleyen *tam plandır*.
- Sağda Ş2, Ş1'in fiyatını görüp yanıt veriyor: 2 karar düğümü $\times$ 2 eylem = **4 strateji**, oysa yalnız 2 eylem var.
- Strateji, diğerlerinin davranışına ilişkin **beklentilere** dayanır.



Saf strateji (pure)

Tek bir planı kesin olarak seçmek.

“her zaman Taş”

Karışık strateji (mixed)

Saf stratejiler üzerinde olasılık dağılımı.

(Taş 1/3, Kağıt 1/3, Makas 1/3)

Strateji profili: her oyuncudan bir strateji, $s = (s_1, \dots, s_n)$. Sonucu profil belirler.

Örnek: Çevrimiçi Reklam Açık Artırmaları

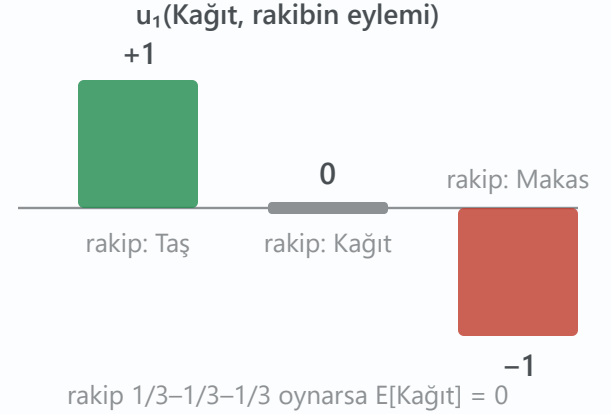
Oyuncular	Reklamverenler (ve kuralı koyan platform)
Stratejiler	Her reklam alanı için verilen teklif
Kazanç	Tıklamanın değeri – ödenen tutar
Bilgi	Herkes kendi değerini ve bütçesini bilir, başkalarınınkini bilmez
Kural	Teklifler gerçek zamanlı; kazanan ve ödeme açık artırma kuralıyla belirlenir

- Reklamverenlerin bütçe kısıtları ve farklı tercihleri var (üst sıra ya da ucuz görünürlük).
- Platform geliri artırmak, ama alanları **verimli ve adil** dağıtmak ister.
- Reklamverenler bütçe içinde teklif stratejilerini optimize eder.

Ekonomi + bilgisayar bilimi + oyun kuramı: milisaniyeler içinde milyonlarca açık artırma. Ayrıntı: Bölüm 5 ve 9.

Kazanç (Payoff) ve Fayda Fonksiyonu

- Oyuncunun, **tüm oyuncuların** seçtiği strateji profiline bağlı olarak elde ettiği değer; fayda fonksiyonu u_i ile ölçülür.
- Oyuncular stratejilerini kazançlarını en büyükmek için seçer.
- Para, puan, hapis yılı (eksi işaretle) ya da memnuniyet düzeyi olabilir.
- Karışık stratejilerde oyuncu **beklenen** kazancını hesaplar.



Aynı eylemin (Kağıt) kazancı rakibin eylemine göre +1, 0 ya da -1 olur: kazanç bir **profilin** fonksiyonudur, $u_1(Kağıt, Taş) = +1$.

Örnek: Monopoly'de Taşınmaz ve Kazanç

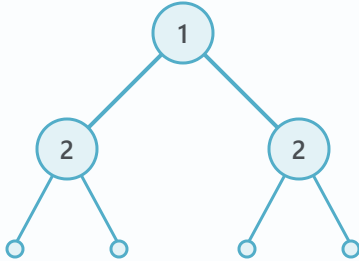
- Oyuncular taşınmaz (property) satın alır ve geliştirir; amaç kira toplamak ve rakipleri iflas ettirmek.
- Örnek: En değerli arsa **Boardwalk** (ABD sürümü; 400 \$).
- Kazanç yalnızca sizin yatırımınıza değil, rakiplerin o kareye **gelip gelmemesine** de bağlıdır.
- Soru: Ev mi dikmeli, nakit mi saklamalı? Yatırımın getirisi riske ve rakiplerin kararlarına karşı tartılır.

Boardwalk gelişimi	Kira (\$)
Boş arsa	50
Renk grubunun tamamı	100
1 ev	200
2 ev	600
4 ev	1.700
Otel	2.000

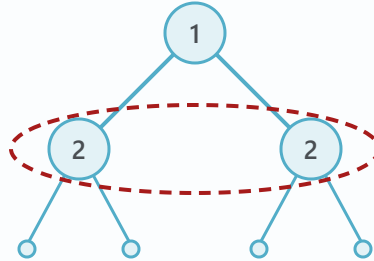
Ev başına maliyet 200 \$: getiri, rakiplerin kareye gelme olasılığına bağlı.

Bilgi: Kim Neyi Biliyor?

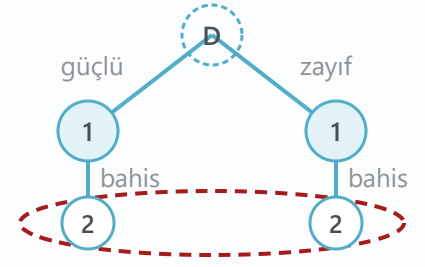
- Her oyuncunun oyun, kazançlar ve yapılan hamleler hakkında sahip olduğu bilgi; stratejiyi ve sonucu doğrudan etkiler.



2, 1'in seçimini görür
Mükemmel bilgi
satranç, XOX, Nim



kesikli: bilgi kümesi (2 ayırt edemez)
Eşzamanlı hamle
TKM, fiyat rekabeti



D: Doğa kartı dağıtır; 2 eli görmez
Mükemmel olmayan bilgi
poker, açık artırma

Kesikli çizgi **bilgi kümesidir**: oyuncu o düğümleri birbirinden ayıramaz. Bilgi türlerinin ayrıntılı sınıflandırması Bölüm 2'de, Bayesçi oyunlar Bölüm 4'te.

Örnek: Poker ve Blöf Stratejileri

Oyun kurulumu (Texas Hold'em)

- Her oyuncunun 2 kapalı kartı var; ortadaki açık kartlarla en iyi eli oluşturmaya çalışır.
- Rakibin kapalı kartlarını göremezsin: bilgi gizlidir.
- Güçlü bir el gördüğünde bile, rakiplerin ellerini bahislerinden tahmin etmeye çalışırsın.

Oyun kuramı yönü

- Rakibin davranışı (bahis, bekleme) ona dair **bilgi** taşır.
- **Blöf**, elin gücünü gizlemek için yanıltıcı sinyal göndermektir.
- Yalnızca güçlü elde bahis yapan oyuncu kolayca okunur.

Blöf aslında bir **karışık strateji**dir: zayıf elde de belirli bir olasılıkla bahis yapmak, rakibin sizi okumasını engeller. Bilgisayar poker programları (ör. Libratus, 2017) bu dengeleri hesaplayarak profesyonelleri yendi.

Normal Form (Stratejik Form)

- Oyuncuların **eşzamanlı** (ya da birbirinin seçimini bilmeden) karar verdiği durumları modeller.
- Oyuncular kümesi $\mathbf{N} = \{1, \dots, n\}$; her oyuncunun strateji kümesi $\mathbf{S}_i$.
- **Strateji profili** (strategy profile): $s = (s_1, \dots, s_n)$.
- **Fayda fonksiyonu**: $u_i : S_1 \times \dots \times S_n \rightarrow \mathbb{R}$.
- İki oyuncuda **kazanç matrisi**: satır = oyuncu 1, sütun = oyuncu 2, hücre = (u_1, u_2) .

		Oyuncu 2	
		Sol	Sağ
Oyuncu 1	Üst	(4 , 3)	(1 , 2)
	Alt	(3 , 1)	(2 , 4)

$S_1 = \{\text{Üst, Alt}\}, S_2 = \{\text{Sol, Sağ}\}$

vurgulu profil $s = (\text{Üst, Sağ})$: $u_1 = 1, u_2 = 2$

$G = (N, S_1, S_2, u_1, u_2), N = \{1, 2\}$

Örnek: Taş-Kağıt-Makas Normal Formda

- İki oyuncu ($n = 2$), **aynı anda** Taş, Kağıt ya da Makas seçer. Taş makası, makas kağıdı, kağıt taşı yener.

		Oyuncu 2			Kazanan +1 Kaybeden -1 Beraberlik 0 Toplam her hücrede 0 (sıfır toplamlı oyun)
Oyuncu 1		Taş	Kağıt	Makas	
Taş	(0 , 0)	(-1 , 1)	(1 , -1)		
Kağıt	(1 , -1)	(0 , 0)	(-1 , 1)		
Makas	(-1 , 1)	(1 , -1)	(0 , 0)		

Hiçbir hücrede iki oyuncunun da en iyi tepkisi buluşmaz: TKM'nin **saf** Nash dengesi yoktur.

Taş-Kağıt-Makas'ta Strateji Seçimi

1. Rastgele seçim

Her turda her sembolü $1/3$ olasılıkla seç.
Beklenen kazanç, rakip ne yaparsa yapsın 0. Hiç sömürülemez: **denge** stratejisi.

2. Geçmişe göre seçim

Rakip son turda Taş dediyse bu tur Kağıt de.
Örüntülü rakibe karşı kazandırır; ama bu kural da bir örüntüdür ve rakip onu sömürebilir.

3. Dengesiz karışık strateji

Ör. yarı zamanda Kağıt, yarı zamanda Makas.
Rakip hep Makas derse beklenen kazancı **+1/2** olur: bu karışım **sömürülebilir**.

Basit bir oyunda bile strateji seçimi önemlidir. Tahmin edilemez olmak yetmez; doğru olasılıklarla tahmin edilemez olmak gerekir. Hesap: "Denge Kavramları" kısmında.

Sıfır Toplamlı Oyunlar

- Her profilde kazançların toplamı sıfır: $u_1(s) + u_2(s) = 0$. Birinin kazancı, diğerinin kaybıdır.
- Toplam sabitse (ör. bir puan paylaşıyorsa) aynı analiz geçerli: *sabit toplamlı* oyun.
- Örnek: TKM, Yazı-Tura, satranç (+1 / 0 / -1).
Değil: Mahkûmun ikilemi, fiyat rekabeti.
- Tek matris yeter: satır büyütmek, sütun küçültmek ister → **maksimin** ve **minimaks**.

Ayrıştıran

Eşleştiren

Yazı

Yazı	(1 , -1)
Tura	(-1 , 1)

Tura

(-1 , 1)	(1 , -1)
------------	------------

$u_2 = -u_1$ olduğundan yalnız u_1 yazılır

↓

Yazı

Yazı	1
Tura	-1

Tura

-1	1
----	---

A

von Neumann (1928): Karışık stratejilere izin verilirse her sonlu iki kişilik sıfır toplamlı oyunda maksimin = minimaks (oyunun *değeri*). Saf stratejilerde bu ancak eyer noktası varsa olur. Ayrıntı: Bölüm 3.

Kod İzleme: Maksimin = Minimaks (Eyer Noktası)

```

▶ A = [[4, 2, 5],
▶      [3, 1, 0],
▶      [6, 3, 4]]      # satırın kazancı
▶ alt, ust = -99, 99

5 for i in range(3):    # satır min'i
6     m = min(A[i])
7     if m > alt: alt, si = m, i
8 for j in range(3):    # sütun max'ı
9     M = max(A[k][j] for k in range(3))
10    if M < ust: ust, sj = M, j
11 print(alt, ust, (si, sj))

```

		sütun j			
	satır i	0	1	2	min
0		4	2	5	
1		3	1	0	
2		6	3	4	
		max			

Sıfır toplamlı: hücrede yalnız satırın kazancı var; sütun oyuncusu bunun eksisini alır, yani küçültmek ister.

Adım 1 / 8

DEĞİŞKENLER

alt = -99

ust = 99

ÇIKTI

Kod İzleme: Maksimin = Minimaks (Eyer Noktası)

```

1 A = [[4, 2, 5],
2       [3, 1, 0],
3       [6, 3, 4]]      # satırın kazancı
4 alt, ust = -99, 99
5 for i in range(3):    # satır min'i
6     ▶ m = min(A[i])
7     ▶ if m > alt: alt, si = m, i
8 for j in range(3):    # sütun max'ı
9     M = max(A[k][j] for k in range(3))
10    if M < ust: ust, sj = M, j
11 print(alt, ust, (si, sj))

```

		sütun j			
satır i	0	1	2	min	
0	4	2	5	2	
1	3	1	0		
2	6	3	4		
max					

Satır 0: en kötü sonuç min = 2. $2 >$ önceki en iyi $\rightarrow$ **alt = 2** (maksimin adayı).

Adım 2 / 8

DEĞİŞKENLER

i = 0

m = 2

alt = 2

ust = 99

si = 0

ÇIKTI

Kod İzleme: Maksimin = Minimaks (Eyer Noktası)

```

1 A = [[4, 2, 5],
2       [3, 1, 0],
3       [6, 3, 4]]      # satırın kazancı
4 alt, ust = -99, 99
5 for i in range(3):    # satır min'i
6     ▶ m = min(A[i])
7     ▶ if m > alt: alt, si = m, i
8 for j in range(3):    # sütun max'ı
9     M = max(A[k][j] for k in range(3))
10    if M < ust: ust, sj = M, j
11 print(alt, ust, (si, sj))

```

		sütun j			
		0	1	2	min
satır i	0	4	2	5	2
	1	3	1	0	0
	2	6	3	4	
		max			

Satır 1: en kötü sonuç min = 0. 0, 2'den iyi değil → değişmez.

Adım 3 / 8

DEĞİŞKENLER

i = 1

m = 0

alt = 2

ust = 99

si = 0

ÇIKTI

Kod İzleme: Maksimin = Minimaks (Eyer Noktası)

```

1 A = [[4, 2, 5],
2       [3, 1, 0],
3       [6, 3, 4]]      # satırın kazancı
4 alt, ust = -99, 99
5 for i in range(3):    # satır min'i
6     ▶ m = min(A[i])
7     ▶ if m > alt: alt, si = m, i
8 for j in range(3):    # sütun max'ı
9     M = max(A[k][j] for k in range(3))
10    if M < ust: ust, sj = M, j
11 print(alt, ust, (si, sj))

```

		sütun j			
satır i		0	1	2	min
	0	4	2	5	2
	1	3	1	0	0
	2	6	3	4	3
		max			

Satır 2: en kötü sonuç min = 3. $3 >$ önceki en iyi $\rightarrow$ **alt = 3** (maksimin adayı).

Adım 4 / 8

DEĞİŞKENLER

i = 2

m = 3

alt = 3

ust = 99

si = 2

ÇIKTI

Kod İzleme: Maksimin = Minimaks (Eyer Noktası)

```

1 A = [[4, 2, 5],
2      [3, 1, 0],
3      [6, 3, 4]]      # satırın kazancı
4 alt, ust = -99, 99
5 for i in range(3):    # satır min'i
6     m = min(A[i])
7     if m > alt: alt, si = m, i
8 for j in range(3):    # sütun max'ı
9     M = max(A[k][j] for k in range(3))
10    if M < ust: ust, sj = M, j
11 print(alt, ust, (si, sj))

```

		sütun j			
		0	1	2	min
satır i	0	4	2	5	2
	1	3	1	0	0
	2	6	3	4	3
max		6			

Sütun 0: satır oyuncusu en fazla 6 alabilir. 6 < önceki → **ust = 6** (minimaks adayı).

Adım 5 / 8

DEĞİŞKENLER

j = 0

M = 6

alt = 3

ust = 6

si = 2

sj = 0

ÇIKTI

Kod İzleme: Maksimin = Minimaks (Eyer Noktası)

```

1 A = [[4, 2, 5],
2       [3, 1, 0],
3       [6, 3, 4]]      # satırın kazancı
4 alt, ust = -99, 99
5 for i in range(3):    # satır min'i
6     m = min(A[i])
7     if m > alt: alt, si = m, i
8 for j in range(3):    # sütun max'ı
9     M = max(A[k][j] for k in range(3))
10    if M < ust: ust, sj = M, j
11 print(alt, ust, (si, sj))

```

		sütun j			
satır i	0	1	2	min	
0	4	2	5	2	
1	3	1	0	0	
2	6	3	4	3	
max	6	3			

Sütun 1: satır oyuncusu en fazla 3 alabilir. 3 < önceki → **ust = 3** (minimaks adayı).

Adım 6 / 8

DEĞİŞKENLER

j = 1

M = 3

alt = 3

ust = 3

si = 2

sj = 1

ÇIKTI

Kod İzleme: Maksimin = Minimaks (Eyer Noktası)

```

1 A = [[4, 2, 5],
2       [3, 1, 0],
3       [6, 3, 4]]      # satırın kazancı
4 alt, ust = -99, 99
5 for i in range(3):    # satır min'i
6     m = min(A[i])
7     if m > alt: alt, si = m, i
8 for j in range(3):    # sütun max'ı
9     M = max(A[k][j] for k in range(3))
10    if M < ust: ust, sj = M, j
11 print(alt, ust, (si, sj))

```

		sütun j			
		0	1	2	min
satır i	0	4	2	5	2
	1	3	1	0	0
	2	6	3	4	3
max		6	3	5	

Sütun 2: satır oyuncusu en fazla 5 alabilir. 5 daha kötü → değişmez.

Adım 7 / 8

DEĞİŞKENLER

j = 2

M = 5

alt = 3

ust = 3

si = 2

sj = 1

ÇIKTI

Kod İzleme: Maksimin = Minimaks (Eyer Noktası)

```

1 A = [[4, 2, 5],
2       [3, 1, 0],
3       [6, 3, 4]]      # satırın kazancı
4 alt, ust = -99, 99
5 for i in range(3):    # satır min'i
6     m = min(A[i])
7     if m > alt: alt, si = m, i
8 for j in range(3):    # sütun max'ı
9     M = max(A[k][j] for k in range(3))
10    if M < ust: ust, sj = M, j
    ► print(alt, ust, (si, sj))

```

		sütun j			
satır i	0	1	2	min	
0	4	2	5	2	
1	3	1	0	0	
2	6	3	4	3	
max	6	3	5		

Maksimin = minimaks = 3: (2, 1) bir **eyer noktası**, oyunun değeri 3. Yazı-Tura'da $-1 < +1$: eyer yok, karışık strateji gerekir.

Adım 8 / 8

DEĞİŞKENLER

alt = 3

ust = 3

si = 2

sj = 1

ÇIKTI

3 3 (2, 1)

Örnek: Fiyat Rekabeti Oyunu

- İki şirket aynı pazarda benzer ürün satıyor; aynı anda fiyat belirliyor: 100 TL ya da 120 TL.
- Ucuz olan müşteriyi kapar; ikisi de pahalıysa kâr marjı yüksek kalır.
- Hücreler kâr birimi: (Şirket 1, Şirket 2).

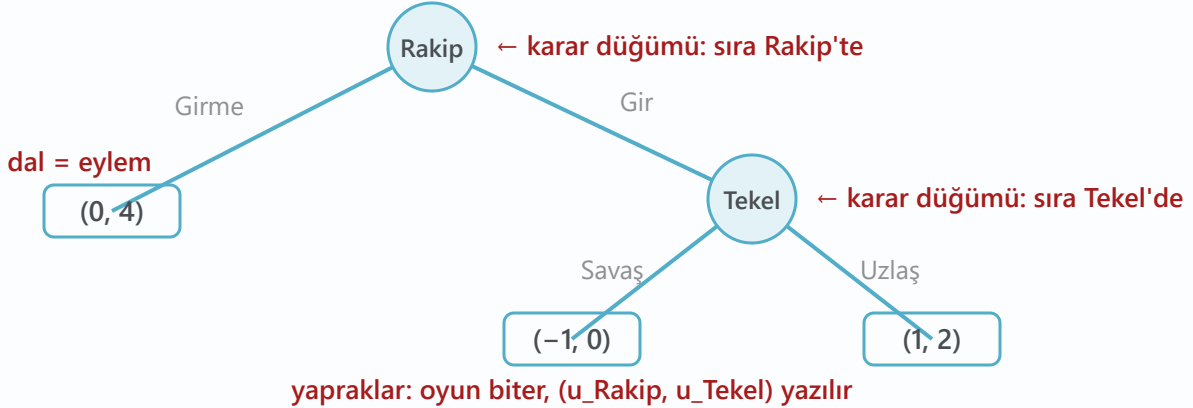
	Şirket 2: 100 TL	Şirket 2: 120 TL
Şirket 1: 100 TL	(5, 5)	(8, 1)
Şirket 1: 120 TL	(1, 8)	(6, 6)

İkisi de 100 TL derse 5'er, ikisi de 120 TL derse 6'şar birim kâr.

Karşı taraf 100 derse $5 > 1$, 120 derse $8 > 6$: her şirket için **100 TL baskın**. Denge (100, 100) → 5'er kâr; oysa (120, 120) ikisine de 6 verirdi. Bu yapıyı birazdan Mahkûmun İkilemi'nde yeniden göreceğiz.

Geniřletilmiş Form (Oyun Ağacı)

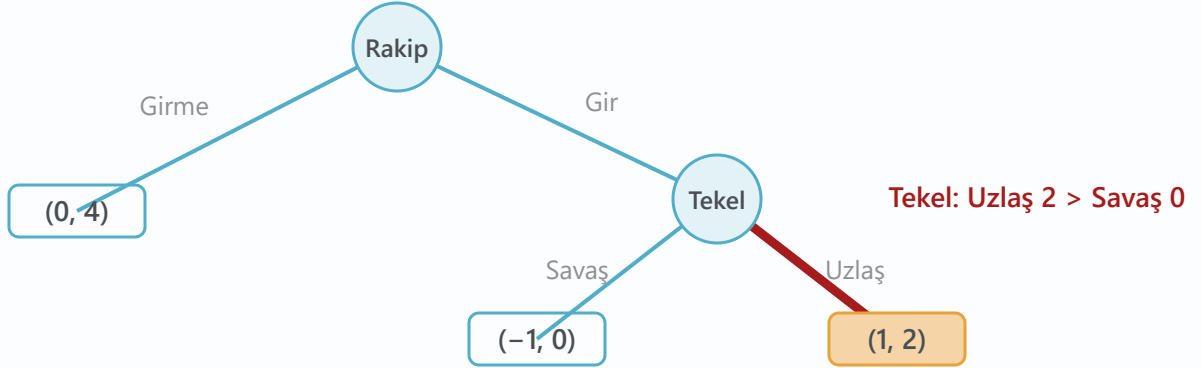
- Kararların **sırasını** ve her anda kimin neyi bildiğini gösteren **oyun ağacı**; sıralı etkileřimleri (satranç, pazarlık, piyasaya giriř) doğal biçimde modeller.



Piyasaya giriř oyunu: Rakip pazara girer ya da girmez; girerse Tekel savařır ya da uzlařır. Bilgi kümesi (ayırt edilemeyen düğümler) sonraki slaytta.

Geniřletilmiş Form (Oyun Ağacı)

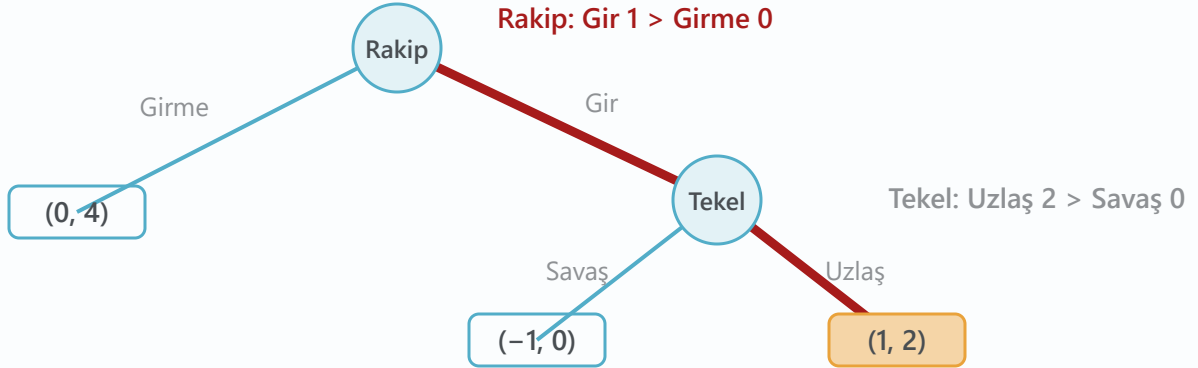
- Kararların **sırasını** ve her anda kimin neyi bildiğini gösteren **oyun ağacı**; sıralı etkileřimleri (satranç, pazarlık, piyasaya giriř) doğal biçimde modeller.



Geriye doğru çıkarım (backward induction): en alttaki karardan başla. Tekel için Uzlaş 2 > Savaş 0.

Geniřletilmiş Form (Oyun Ağacı)

- Kararların **sırasını** ve her anda kimin neyi bildiğini gösteren **oyun ağacı**; sıralı etkileřimleri (satranç, pazarlık, piyasaya giriř) doğal biçimde modeller.



Sonuç (Gir, Uzlaş) = $(1, 2)$: alt oyun mükemmel denge

Rakip, Tekel'in uzlaşacağını bilerek Gir 1 > Girme 0 seçer. "Girersen savaşırım" tehdidi **inandırıcı değildir** (Bölüm 2).

Normal Formdan Oyun Ağacına: Adım Adım

		Şirket 2	
Ş1		100 TL	120 TL
100	(5, 5)	(8, 1)	
120	(1, 8)	(6, 6)	

normal form

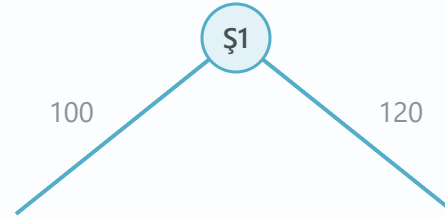
Başlangıç: fiyat rekabeti oyununun normal formu. Aynı oyunu bir **oyun ağacı** olarak çizeceğiz.

Adım 1 / 6

Normal Formdan Oyun Ağacına: Adım Adım

		Şirket 2	
Ş1		100 TL	120 TL
100		(5, 5)	(8, 1)
120		(1, 8)	(6, 6)

normal form



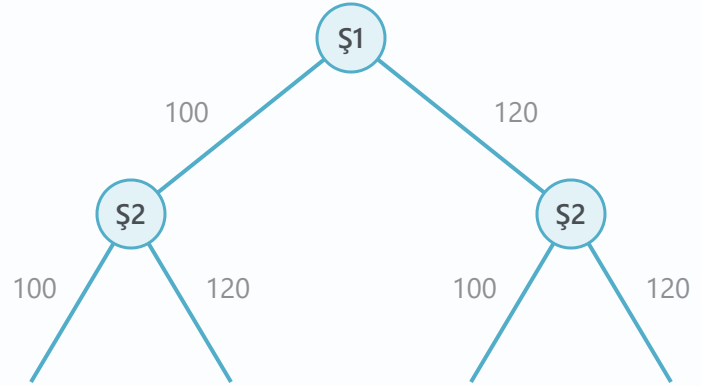
Kök düğüm: Şirket 1'in kararı. Her dal bir eylemdir: 100 TL ya da 120 TL.

Normal Formdan Oyun Ağacına: Adım Adım

Şirket 2

Ş1	100 TL	120 TL
100	(5, 5)	(8, 1)
120	(1, 8)	(6, 6)

normal form



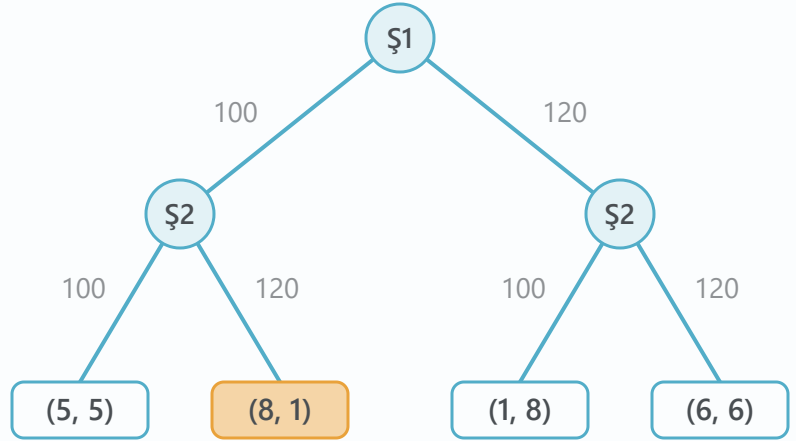
Her dalın sonunda Şirket 2'nin karar düğümü var; o da 100 ya da 120 seçer.

Normal Formdan Oyun Ağacına: Adım Adım

Şirket 2

Ş1	100 TL	120 TL
100	(5, 5)	(8, 1)
120	(1, 8)	(6, 6)

normal form



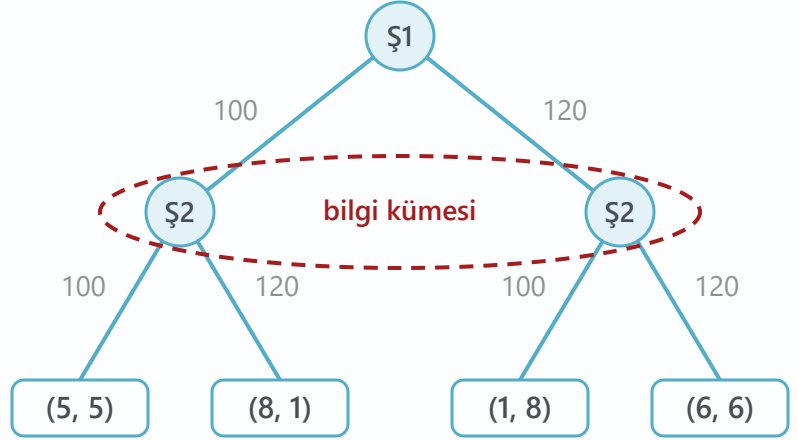
Yapraklar sonuçlardır: dört yol, matrisin dört hücreğine karşılık gelir. Örn. 100–120 yolu (8, 1) hücresi.

Normal Formdan Oyun Ağacına: Adım Adım

Şirket 2

Ş1	100 TL	120 TL
100	(5, 5)	(8, 1)
120	(1, 8)	(6, 6)

normal form



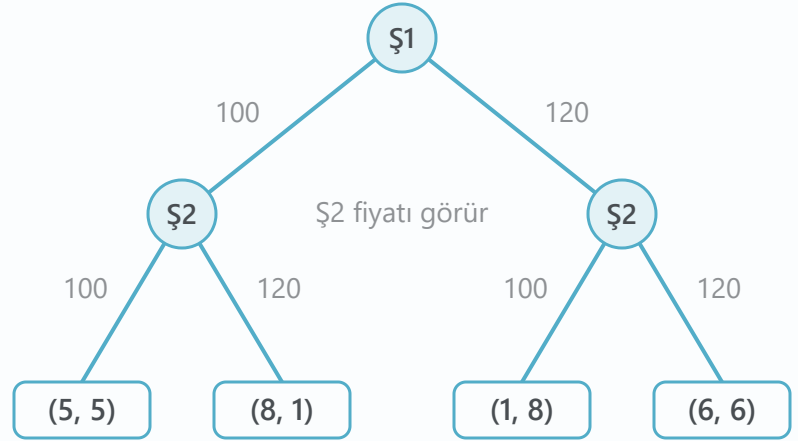
Ama iki şirket fiyatı **aynı anda** belirliyor: Ş2, Ş1'in seçimini görmez. Kesikli **bilgi kümesi** bunu gösterir; ağaç matrisle aynı oyundur.

Normal Formdan Oyun Ağacına: Adım Adım

Şirket 2

Ş1	100 TL	120 TL
100	(5, 5)	(8, 1)
120	(1, 8)	(6, 6)

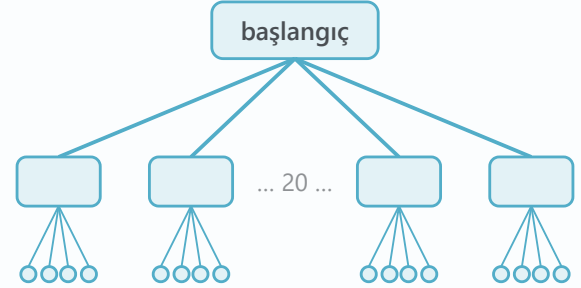
normal form



Bilgi kümesini kaldırırsak Ş2 önce Ş1'in fiyatını görür: artık **sıralı** bir oyun. Çözümü (geriye doğru çıkarım) Bölüm 2'de.

Örnek: Satranç Bir Genişletilmiş Form Oyunudur

- İki oyuncu sırayla hamle yapar; her hamle bir dal, her konum bir düğümdür.
- Mükemmel bilgi:** iki oyuncu da tahtayı ve tüm geçmiş hamleleri görür.
- Sonuç galibiyet, yenilgi **ya da beraberliktir** (pat, üç kez tekrar, 50 hamle kuralı).
- Zermelo (1913): Bu tür sonlu oyunlarda ya bir taraf kazanmayı ya da iki taraf beraberliği garanti edebilir; ama ağaç çok büyük olduğu için hangisi olduğu bilinmiyor.



Beyaz: 20 hamle · Siyah: 20 cevap

$20 \times 20 = 400$ konum; tüm ağaç $\approx 10^{120}$ oyun

Satranç programları ağacın yalnızca bir kısmını arar (minimaks + budama). XOX, Nim ve "21'i Söyleme" gibi küçük oyunların ağacı ise tamamen çözülebilir: bölüm sonundaki oyunlarda deneyin.

İşbirlikçi ve İşbirlikçi Olmayan Oyunlar

İşbirlikçi oyunlar

- + Oyuncular **bağlayıcı** (dışarıdan uygulatılabilir) anlaşmalar yapabilir.
- + Analiz birimi **koalisyon**: hangi gruplar oluşur, toplam değer nasıl paylaşılır?
- + Araçlar: Shapley değeri, çekirdek (Bölüm 7).

İşbirlikçi olmayan oyunlar

- Bağlayıcı anlaşma **yoktur**; konuşmalar bile sözlerini tutmaya zorlanamazlar.
- Her oyuncu stratejisini bağımsız, kendi çıkarına göre seçer.
- Analiz birimi **tek oyuncu**; araç: Nash dengesi.

Bu bölümdeki oyunların hepsi (TKM, fiyat rekabeti, Mahkûmun İkilemi, 2/3 oyunu) işbirlikçi olmayan oyunlardır.

Denge ve Nash Dengesi

- **En iyi tepki** (best response): Diğİrlerinin stratejileri s_{-i} sabitken oyuncu i'ye en yüksek kazancı veren strateji.
- **Nash dengesi**: Herkesin stratejisi diğİrlerinininkine en iyi tepki olan profil s^* ; hiç kimse *tek taraflı* sapmak istemez (Nash, 1950).

$$u_i(s_i^*, s_{-i}^*) \geq u_i(s_i, s_{-i}^*) \quad \text{her oyuncu } i \text{ ve her } s_i \in S_i \text{ iin}$$

		Şirket 2	
		100	120
Ş1	100	(5 , 5)	(8 , 1)
	120	(1 , 8)	(6 , 6)

mavi ↑: Şirket 1 saparak kazanır · turuncu ←: Şirket 2 sapar
okların hepsi (100, 100)'e akar: kimse ayrılmak istemez

Ne söyler?

Diğİrleri sabitken sapmak kazandırmaz; kimse pişman değildir. Sapma okları dengeye çıkar, dengeden çıkmaz.

Ne söylemez?

Dengenin herkes iin iyi olduğunu: (120, 120) ikisine de 6 verirdi. Mahkûmun İkilemi'nde yeniden.

Kod İzleme: En İyi Tepkileri Bulmak

```

▶ U = [(4, 2), (3, 1), (2, 4)],
▶     [(1, 0), (2, 3), (3, 4)],
▶     [(0, 2), (2, 0), (2, 1)]
▶ S = range(3)

```

```

5 def br1(j):                # sütun j'ye tepki
6     col = [U[i][j][0] for i in S]
7     return [i for i in S if col[i] == max(col)]
8 def br2(i):                # satır i'ye tepki
9     row = [U[i][j][1] for j in S]
10    return [j for j in S if row[j] == max(row)]
11 ne = [(i, j) for i in S for j in S
12        if i in br1(j) and j in br2(i)]
13 print(ne)

```

		Oyuncu 2 (sütun j)		
1 (i)		0	1	2
0	(4 , 2)	(3 , 1)	(2 , 4)	
1	(1 , 0)	(2 , 3)	(3 , 4)	
2	(0 , 2)	(2 , 0)	(2 , 1)	

Oyun: satır oyuncusu $i \in \{0,1,2\}$, sütun oyuncusu $j \in \{0,1,2\}$. Hücre = (u_1, u_2) .

Adım 1 / 9

DEĞİŞKENLER

S = range(3)

ÇIKTI



Kod İzleme: En İyi Tepkileri Bulmak

```

1 U = [[(4, 2), (3, 1), (2, 4)],
2      [(1, 0), (2, 3), (3, 4)],
3      [(0, 2), (2, 0), (2, 1)]]
4 S = range(3)
5 def br1(j):                # sütun j'ye tepki
6     ▶ col = [U[i][j][0] for i in S]
7     ▶ return [i for i in S if col[i] == max(col)]
8 def br2(i):                # satır i'ye tepki
9     row = [U[i][j][1] for j in S]
10    return [j for j in S if row[j] == max(row)]
11 ne = [(i, j) for i in S for j in S
12        if i in br1(j) and j in br2(i)]
13 print(ne)

```

		Oyuncu 2 (sütun j)		
1 (i)	0	1	2	
0	(4 , 2)	(3 , 1)	(2 , 4)	
1	(1 , 0)	(2 , 3)	(3 , 4)	
2	(0 , 2)	(2 , 0)	(2 , 1)	

Sütun 0 sabit: 1'in kazançları 4, 1, 0. En büyüğü 4 → en iyi tepki satır **0** (turuncu).

Adım 2 / 9

DEĞİŞKENLER

`j = 0`

`col = [4, 1, 0]`

`br1(j) = [0]`

ÇIKTI



Kod İzleme: En İyi Tepkileri Bulmak

```

1 U = [(4, 2), (3, 1), (2, 4)],
2     [(1, 0), (2, 3), (3, 4)],
3     [(0, 2), (2, 0), (2, 1)]
4 S = range(3)
5 def br1(j):                # sütun j'ye tepki
6     ▶ col = [U[i][j][0] for i in S]
7     ▶ return [i for i in S if col[i] == max(col)]
8 def br2(i):                # satır i'ye tepki
9     row = [U[i][j][1] for j in S]
10    return [j for j in S if row[j] == max(row)]
11 ne = [(i, j) for i in S for j in S
12        if i in br1(j) and j in br2(i)]
13 print(ne)

```

Oyuncu 2 (sütun j)

1 (i)	0	1	2
0	(4, 2)	(3, 1)	(2, 4)
1	(1, 0)	(2, 3)	(3, 4)
2	(0, 2)	(2, 0)	(2, 1)

Sütun 1 sabit: 1'in kazançları 3, 2, 2. En büyüğü 3 → en iyi tepki satır **0** (turuncu).

Adım 3 / 9

DEĞİŞKENLER

j = 1

col = [3, 2, 2]

br1(j) = [0]

ÇIKTI

Kod İzleme: En İyi Tepkileri Bulmak

```

1 U = [(4, 2), (3, 1), (2, 4)],
2     [(1, 0), (2, 3), (3, 4)],
3     [(0, 2), (2, 0), (2, 1)]
4 S = range(3)
5 def br1(j):                # sütun j'ye tepki
6     ▶ col = [U[i][j][0] for i in S]
7     ▶ return [i for i in S if col[i] == max(col)]
8 def br2(i):                # satır i'ye tepki
9     row = [U[i][j][1] for j in S]
10    return [j for j in S if row[j] == max(row)]
11 ne = [(i, j) for i in S for j in S
12        if i in br1(j) and j in br2(i)]
13 print(ne)

```

		Oyuncu 2 (sütun j)		
1 (i)	0	1	2	
0	(4, 2)	(3, 1)	(2, 4)	
1	(1, 0)	(2, 3)	(3, 4)	
2	(0, 2)	(2, 0)	(2, 1)	

Sütun 2 sabit: 1'in kazançları 2, 3, 2. En büyüğü 3 → en iyi tepki satır **1** (turuncu).

Adım 4 / 9

DEĞİŞKENLER

`j = 2`

`col = [2, 3, 2]`

`br1(j) = [1]`

ÇIKTI



Kod İzleme: En İyi Tepkileri Bulmak

```

1 U = [(4, 2), (3, 1), (2, 4)],
2     [(1, 0), (2, 3), (3, 4)],
3     [(0, 2), (2, 0), (2, 1)]
4 S = range(3)
5 def br1(j):                # sütun j'ye tepki
6     col = [U[i][j][0] for i in S]
7     return [i for i in S if col[i] == max(col)]
8 def br2(i):                # satır i'ye tepki
9     ▶ row = [U[i][j][1] for j in S]
10    ▶ return [j for j in S if row[j] == max(row)]
11 ne = [(i, j) for i in S for j in S
12        if i in br1(j) and j in br2(i)]
13 print(ne)

```

Oyuncu 2 (sütun j)

1 (i)	0	1	2
0	(4, 2)	(3, 1)	(2, 4)
1	(1, 0)	(2, 3)	(3, 4)
2	(0, 2)	(2, 0)	(2, 1)

Satır 0 sabit: 2'nin kazançları 2, 1, 4 → en iyi tepki sütun 2.

Adım 5 / 9

DEĞİŞKENLER

i = 0

row = [2, 1, 4]

br2(i) = [2]

ÇIKTI



Kod İzleme: En İyi Tepkileri Bulmak

```

1 U = [(4, 2), (3, 1), (2, 4)],
2     [(1, 0), (2, 3), (3, 4)],
3     [(0, 2), (2, 0), (2, 1)]
4 S = range(3)
5 def br1(j):                # sütun j'ye tepki
6     col = [U[i][j][0] for i in S]
7     return [i for i in S if col[i] == max(col)]
8 def br2(i):                # satır i'ye tepki
9     row = [U[i][j][1] for j in S]
10    return [j for j in S if row[j] == max(row)]
11 ne = [(i, j) for i in S for j in S
12        if i in br1(j) and j in br2(i)]
13 print(ne)

```

Oyuncu 2 (sütun j)

1 (i)	0	1	2
0	(4, 2)	(3, 1)	(2, 4)
1	(1, 0)	(2, 3)	(3, 4)
2	(0, 2)	(2, 0)	(2, 1)

Satır 1 sabit: 2'nin kazançları 0, 3, 4 → en iyi tepki sütun 2.

Adım 6 / 9

DEĞİŞKENLER

i = 1

row = [0, 3, 4]

br2(i) = [2]

ÇIKTI



Kod İzleme: En İyi Tepkileri Bulmak

```

1 U = [(4, 2), (3, 1), (2, 4)],
2     [(1, 0), (2, 3), (3, 4)],
3     [(0, 2), (2, 0), (2, 1)]
4 S = range(3)
5 def br1(j):          # sütun j'ye tepki
6     col = [U[i][j][0] for i in S]
7     return [i for i in S if col[i] == max(col)]
8 def br2(i):          # satır i'ye tepki
9     row = [U[i][j][1] for j in S]
10    return [j for j in S if row[j] == max(row)]
11 ne = [(i, j) for i in S for j in S
12        if i in br1(j) and j in br2(i)]
13 print(ne)

```

Oyuncu 2 (sütun j)

1 (i)	0	1	2
0	(4, 2)	(3, 1)	(2, 4)
1	(1, 0)	(2, 3)	(3, 4)
2	(0, 2)	(2, 0)	(2, 1)

Satır 2 sabit: 2'nin kazançları 2, 0, 1 → en iyi tepki sütun 0.

Adım 7 / 9

DEĞİŞKENLER

i = 2

row = [2, 0, 1]

br2(i) = [0]

ÇIKTI

Kod İzleme: En İyi Tepkileri Bulmak

```

1 U = [[(4, 2), (3, 1), (2, 4)],
2       [(1, 0), (2, 3), (3, 4)],
3       [(0, 2), (2, 0), (2, 1)]]
4 S = range(3)
5 def br1(j):           # sütun j'ye tepki
6     col = [U[i][j][0] for i in S]
7     return [i for i in S if col[i] == max(col)]
8 def br2(i):           # satır i'ye tepki
9     row = [U[i][j][1] for j in S]
10    return [j for j in S if row[j] == max(row)]
▶ ne = [(i, j) for i in S for j in S
▶       if i in br1(j) and j in br2(i)]
13 print(ne)

```

Oyuncu 2 (sütun j)

1 (i)	0	1	2
0	(4, 2)	(3, 1)	(2, 4)
1	(1, 0)	(2, 3)	(3, 4)
2	(0, 2)	(2, 0)	(2, 1)

İki sayısı da turuncu olan tek hücre (1, 2):
herkes diğerine en iyi tepki veriyor → **saf Nash dengesi**.

Adım 8 / 9

DEĞİŞKENLER

ne = [(1, 2)]

ÇIKTI

Kod İzleme: En İyi Tepkileri Bulmak

```

1 U = [(4, 2), (3, 1), (2, 4)],
2     [(1, 0), (2, 3), (3, 4)],
3     [(0, 2), (2, 0), (2, 1)]
4 S = range(3)
5 def br1(j):          # sütun j'ye tepki
6     col = [U[i][j][0] for i in S]
7     return [i for i in S if col[i] == max(col)]
8 def br2(i):          # satır i'ye tepki
9     row = [U[i][j][1] for j in S]
10    return [j for j in S if row[j] == max(row)]
11 ne = [(i, j) for i in S for j in S
12        if i in br1(j) and j in br2(i)]
13    print(ne)

```

Oyuncu 2 (sütun j)

1 (i)	0	1	2
0	(4, 2)	(3, 1)	(2, 4)
1	(1, 0)	(2, 3)	(3, 4)
2	(0, 2)	(2, 0)	(2, 1)

Sonuç: tek saf denge (1, 2) = (3, 4). Bu tarama $3 \times 3 = 9$ hücreye bakar; $m \times n$ oyunda $O(m \cdot n)$ karşılaştırma.

Adım 9 / 9

DEĞİŞKENLER

ne = [(1, 2)]

ÇIKTI

[(1, 2)]

Baskın Strateji (Dominant Strategy)

- Diğerleri **ne yaparsa yapsın** diğer tüm stratejilerden daha yüksek kazanç veren strateji.
- Baskın stratejisi olan oyuncu rakibini tahmin etmek zorunda değildir.
- Herkesin baskın stratejisi varsa bunların profili bir Nash dengesidir (**baskın strateji dengesi**).
- Her oyunda bulunmaz (TKM'de yok); o zaman en iyi tepkilerle denge aranır.

Şirket 2 seçerse

Ş1	100	120
100	5	8
120	1	6



$5 > 1$



$8 > 6$

Yalnız Şirket 1'in kârı gösteriliyor.

Her sütunda 100 daha iyi $\Rightarrow$ 100 TL baskın

Test: karşı tarafın *her* seçimi için ayrı karşılaştır; "ortalamada daha iyi" yetmez. Baskılanan stratejiler tekrar tekrar silinebilir: sonraki slaytta kodla.

Kod İzleme: Yinelemeli Baskın Strateji Eleme

```

▶ R, C = [0, 1, 2], [0, 1, 2]    # kalanlar
▶ def u1(i, j): return U[i][j][0]
▶ def u2(i, j): return U[i][j][1]

4 degisti = True
5 while degisti:
6     degisti = False
7     for a in R:                # satır a
8         if any(all(u1(b, j) > u1(a, j) for j in C)
9                 for b in R if b != a):
10            R.remove(a); degisti = True; break
11    for a in C:                # sütun a
12        if any(all(u2(i, b) > u2(i, a) for i in R)
13                for b in C if b != a):
14            C.remove(a); degisti = True; break
15 print(R, C)

```

Oyuncu 2 (sütun j)

1 (i)	0	1	2
0	(4 , 2)	(3 , 1)	(2 , 4)
1	(1 , 0)	(2 , 3)	(3 , 4)
2	(0 , 2)	(2 , 0)	(2 , 1)

U önceki slayttaki oyun. Kesin baskılanan (her durumda daha kötü) stratejileri tek tek sileceğiz.

Adım 1 / 15

DEĞİŞKENLER

R = [0, 1, 2]

C = [0, 1, 2]

degisti = True

ÇIKTI



Kod İzleme: Yinelemeli Baskın Strateji Eleme

```

1 R, C = [0, 1, 2], [0, 1, 2]    # kalanlar
2 def u1(i, j): return U[i][j][0]
3 def u2(i, j): return U[i][j][1]
4 degisti = True
5 while degisti:
6     degisti = False
7     ▶ for a in R:                # satır a
8         ▶ if any(all(u1(b, j) > u1(a, j) for j in C)
9             ▶ for b in R if b != a):
10             R.remove(a); degisti = True; break
11     for a in C:                # sütun a
12         if any(all(u2(i, b) > u2(i, a) for i in R)
13             for b in C if b != a):
14             C.remove(a); degisti = True; break
15 print(R, C)

```

Oyuncu 2 (sütun j)

1 (i)	0	1	2
0	(4 , 2)	(3 , 1)	(2 , 4)
1	(1 , 0)	(2 , 3)	(3 , 4)
2	(0 , 2)	(2 , 0)	(2 , 1)

Tur 1: satır 0 kontrol: kalanlar içinde onu her durumda yenen bir satır yok → kalır.

Adım 2 / 15

DEĞİŞKENLER

R = [0, 1, 2]

C = [0, 1, 2]

degisti = False

a = 0

ÇIKTI



Kod İzleme: Yinelemeli Baskın Strateji Eleme

```

1 R, C = [0, 1, 2], [0, 1, 2] # kalanlar
2 def u1(i, j): return U[i][j][0]
3 def u2(i, j): return U[i][j][1]
4 degisti = True
5 while degisti:
6     degisti = False
7     for a in R: # satır a
8         if any(all(u1(b, j) > u1(a, j) for j in C)
9               for b in R if b != a):
10             R.remove(a); degisti = True; break
11     for a in C: # sütun a
12         if any(all(u2(i, b) > u2(i, a) for i in R)
13               for b in C if b != a):
14             C.remove(a); degisti = True; break
15 print(R, C)

```

Oyuncu 2 (sütun j)

1 (i)	0	1	2
0	(4 , 2)	(3 , 1)	(2 , 4)
1	(1 , 0)	(2 , 3)	(3 , 4)
2	(0 , 2)	(2 , 0)	(2 , 1)

Tur 1: satır 1 kontrol: kalanlar içinde onu her durumda yenen bir satır yok → kalır.

Adım 3 / 15

DEĞİŞKENLER

R = [0, 1, 2]

C = [0, 1, 2]

degisti = False

a = 1

ÇIKTI



Kod İzleme: Yinelemeli Baskın Strateji Eleme

```

1 R, C = [0, 1, 2], [0, 1, 2] # kalanlar
2 def u1(i, j): return U[i][j][0]
3 def u2(i, j): return U[i][j][1]
4 degisti = True
5 while degisti:
6     degisti = False
7     for a in R: # satır a
8         if any(all(u1(b, j) > u1(a, j) for j in C)
9                 for b in R if b != a):
10             R.remove(a); degisti = True; break
11     for a in C: # sütun a
12         if any(all(u2(i, b) > u2(i, a) for i in R)
13                 for b in C if b != a):
14             C.remove(a); degisti = True; break
15 print(R, C)

```

Oyuncu 2 (sütun j)

1 (i)	0	1	2
0	(4 , 2)	(3 , 1)	(2 , 4)
1	(1 , 0)	(2 , 3)	(3 , 4)
2	(0 , 2)	(2 , 0)	(2 , 1)

Tur 1: satır 2 kontrol: kalanlar içinde onu her durumda yenen bir satır yok → kalır.

Adım 4 / 15

DEĞİŞKENLER

R = [0, 1, 2]

C = [0, 1, 2]

degisti = False

a = 2

ÇIKTI

Kod İzleme: Yinelemeli Baskın Strateji Eleme

```

1 R, C = [0, 1, 2], [0, 1, 2] # kalanlar
2 def u1(i, j): return U[i][j][0]
3 def u2(i, j): return U[i][j][1]
4 degisti = True
5 while degisti:
6     degisti = False
7     for a in R: # satır a
8         if any(all(u1(b, j) > u1(a, j) for j in C)
9                 for b in R if b != a):
10             R.remove(a); degisti = True; break
11     for a in C: # sütun a
12         if any(all(u2(i, b) > u2(i, a) for i in R)
13                 for b in C if b != a):
14             C.remove(a); degisti = True; break
15 print(R, C)

```

Oyuncu 2 (sütun j)

1 (i)	0	1	2
0	(4 , 2)	(3 , 1)	(2 , 4)
1	(1 , 0)	(2 , 3)	(3 , 4)
2	(0 , 2)	(2 , 0)	(2 , 1)

Tur 1: sütun 0 kontrol: kalanlar içinde onu her durumda yenen bir sütun yok → kalır.

Adım 5 / 15

DEĞİŞKENLER

R = [0, 1, 2]

C = [0, 1, 2]

degisti = False

a = 0

ÇIKTI

Kod İzleme: Yinelemeli Baskın Strateji Eleme

```

1 R, C = [0, 1, 2], [0, 1, 2] # kalanlar
2 def u1(i, j): return U[i][j][0]
3 def u2(i, j): return U[i][j][1]
4 degisti = True
5 while degisti:
6     degisti = False
7     for a in R: # satır a
8         if any(all(u1(b, j) > u1(a, j) for j in C)
9                 for b in R if b != a):
10            R.remove(a); degisti = True; break
11     for a in C: # sütun a
12         if any(all(u2(i, b) > u2(i, a) for i in R)
13                 for b in C if b != a):
14            C.remove(a); degisti = True; break
15 print(R, C)

```

Oyuncu 2 (sütun j)

1 (i)	0	1	2
0	(4 , 2)	(3 , 1)	(2 , 4)
1	(1 , 0)	(2 , 3)	(3 , 4)
2	(0 , 2)	(2 , 0)	(2 , 1)

Tur 1: sütun 1, sütun 2 tarafından baskılanıyor ($4 > 1$, $4 > 3$, $1 > 0$) → **sil**. Artık 1 de bunu bilir.

Adım 6 / 15

DEĞİŞKENLER

R = [0, 1, 2]

C = [0, 2]

degisti = True

a = 1

ÇIKTI



Kod İzleme: Yinelemeli Baskın Strateji Eleme

```

1 R, C = [0, 1, 2], [0, 1, 2] # kalanlar
2 def u1(i, j): return U[i][j][0]
3 def u2(i, j): return U[i][j][1]
4 degisti = True
5 while degisti:
6     degisti = False
7     for a in R: # satır a
8         if any(all(u1(b, j) > u1(a, j) for j in C)
9                 for b in R if b != a):
10             R.remove(a); degisti = True; break
11     for a in C: # sütun a
12         if any(all(u2(i, b) > u2(i, a) for i in R)
13                 for b in C if b != a):
14             C.remove(a); degisti = True; break
15 print(R, C)

```

Oyuncu 2 (sütun j)

1 (i)	0	1	2
0	(4 , 2)	(3 , 1)	(2 , 4)
1	(1 , 0)	(2 , 3)	(3 , 4)
2	(0 , 2)	(2 , 0)	(2 , 1)

Tur 2: satır 0 kontrol: kalanlar içinde onu her durumda yenen bir satır yok → kalır.

Adım 7 / 15

DEĞİŞKENLER

R = [0, 1, 2]

C = [0, 2]

degisti = False

a = 0

ÇIKTI

Kod İzleme: Yinelemeli Baskın Strateji Eleme

```

1 R, C = [0, 1, 2], [0, 1, 2] # kalanlar
2 def u1(i, j): return U[i][j][0]
3 def u2(i, j): return U[i][j][1]
4 degisti = True
5 while degisti:
6     degisti = False
7     for a in R: # satır a
8         if any(all(u1(b, j) > u1(a, j) for j in C)
9                 for b in R if b != a):
10             R.remove(a); degisti = True; break
11     for a in C: # sütun a
12         if any(all(u2(i, b) > u2(i, a) for i in R)
13                 for b in C if b != a):
14             C.remove(a); degisti = True; break
15 print(R, C)

```

Oyuncu 2 (sütun j)

1 (i)	0	1	2
0	(4 , 2)	(3 , 1)	(2 , 4)
1	(1 , 0)	(2 , 3)	(3 , 4)
2	(0 , 2)	(2 , 0)	(2 , 1)

Tur 2: satır 1 kontrol: kalanlar içinde onu her durumda yenen bir satır yok → kalır.

Adım 8 / 15

DEĞİŞKENLER

R = [0, 1, 2]

C = [0, 2]

degisti = False

a = 1

ÇIKTI

Kod İzleme: Yinelemeli Baskın Strateji Eleme

```

1 R, C = [0, 1, 2], [0, 1, 2] # kalanlar
2 def u1(i, j): return U[i][j][0]
3 def u2(i, j): return U[i][j][1]
4 degisti = True
5 while degisti:
6     degisti = False
7     for a in R: # satır a
8         if any(all(u1(b, j) > u1(a, j) for j in C)
9                 for b in R if b != a):
10            R.remove(a); degisti = True; break
11    for a in C: # sütun a
12        if any(all(u2(i, b) > u2(i, a) for i in R)
13                for b in C if b != a):
14            C.remove(a); degisti = True; break
15 print(R, C)

```

Oyuncu 2 (sütun j)

1 (i)	0	1	2
0	(4 , 2)	(3 , 1)	(2 , 4)
1	(1 , 0)	(2 , 3)	(3 , 4)
2	(0 , 2)	(2 , 0)	(2 , 1)

Tur 2: satır 2, satır 1 tarafından baskılanıyor ($1 > 0$, $3 > 2$). Rasyonel oyuncu 1 onu asla oynamaz → **sil**.

Adım 9 / 15

DEĞİŞKENLER

R = [0, 1]

C = [0, 2]

degisti = True

a = 2

ÇIKTI



Kod İzleme: Yinelemeli Baskın Strateji Eleme

```

1 R, C = [0, 1, 2], [0, 1, 2] # kalanlar
2 def u1(i, j): return U[i][j][0]
3 def u2(i, j): return U[i][j][1]
4 degisti = True
5 while degisti:
6     degisti = False
7     for a in R: # satır a
8         if any(all(u1(b, j) > u1(a, j) for j in C)
9                 for b in R if b != a):
10            R.remove(a); degisti = True; break
11     for a in C: # sütun a
12         if any(all(u2(i, b) > u2(i, a) for i in R)
13                 for b in C if b != a):
14            C.remove(a); degisti = True; break
15 print(R, C)

```

Oyuncu 2 (sütun j)

1 (i)	0	1	2
0	(4 , 2)	(3 , 1)	(2 , 4)
1	(1 , 0)	(2 , 3)	(3 , 4)
2	(0 , 2)	(2 , 0)	(2 , 1)

Tur 2: sütun 0, sütun 2 tarafından baskılanıyor ($4 > 2$, $4 > 0$) → **sil**. Artık 1 de bunu bilir.

Adım 10 / 15

DEĞİŞKENLER

R = [0, 1]

C = [2]

degisti = True

a = 0

ÇIKTI



Kod İzleme: Yinelemeli Baskın Strateji Eleme

```

1 R, C = [0, 1, 2], [0, 1, 2] # kalanlar
2 def u1(i, j): return U[i][j][0]
3 def u2(i, j): return U[i][j][1]
4 degisti = True
5 while degisti:
6     degisti = False
7     for a in R: # satır a
8         if any(all(u1(b, j) > u1(a, j) for j in C)
9                 for b in R if b != a):
10            R.remove(a); degisti = True; break
11    for a in C: # sütun a
12        if any(all(u2(i, b) > u2(i, a) for i in R)
13                for b in C if b != a):
14            C.remove(a); degisti = True; break
15 print(R, C)

```

Oyuncu 2 (sütun j)

1 (i)	0	1	2
0	(4 , 2)	(3 , 1)	(2 , 4)
1	(1 , 0)	(2 , 3)	(3 , 4)
2	(0 , 2)	(2 , 0)	(2 , 1)

Tur 3: satır 0, satır 1 tarafından baskılanıyor ($3 > 2$). Rasyonel oyuncu 1 onu asla oynamaz → **sil**.

Adım 11 / 15

DEĞİŞKENLER

R = [1]

C = [2]

degisti = True

a = 0

ÇIKTI



Kod İzleme: Yinelemeli Baskın Strateji Eleme

```

1 R, C = [0, 1, 2], [0, 1, 2] # kalanlar
2 def u1(i, j): return U[i][j][0]
3 def u2(i, j): return U[i][j][1]
4 degisti = True
5 while degisti:
6     degisti = False
7     for a in R: # satır a
8         if any(all(u1(b, j) > u1(a, j) for j in C)
9                 for b in R if b != a):
10            R.remove(a); degisti = True; break
11     for a in C: # sütun a
12         if any(all(u2(i, b) > u2(i, a) for i in R)
13                 for b in C if b != a):
14            C.remove(a); degisti = True; break
15 print(R, C)

```

Oyuncu 2 (sütun j)

1 (i)	0	1	2
0	(4 , 2)	(3 , 1)	(2 , 4)
1	(1 , 0)	(2 , 3)	(3 , 4)
2	(0 , 2)	(2 , 0)	(2 , 1)

Tur 3: sütun 2 tek başına kaldı; karşılaştıracak başka sütun yok.

Adım 12 / 15

DEĞİŞKENLER

R = [1]

C = [2]

degisti = True

a = 2

ÇIKTI

Kod İzleme: Yinelemeli Baskın Strateji Eleme

```

1 R, C = [0, 1, 2], [0, 1, 2] # kalanlar
2 def u1(i, j): return U[i][j][0]
3 def u2(i, j): return U[i][j][1]
4 degisti = True
5 while degisti:
6     degisti = False
7     for a in R: # satır a
8         if any(all(u1(b, j) > u1(a, j) for j in C)
9                 for b in R if b != a):
10             R.remove(a); degisti = True; break
11     for a in C: # sütun a
12         if any(all(u2(i, b) > u2(i, a) for i in R)
13                 for b in C if b != a):
14             C.remove(a); degisti = True; break
15 print(R, C)

```

Oyuncu 2 (sütun j)

1 (i)	0	1	2
0	(4 , 2)	(3 , 1)	(2 , 4)
1	(1 , 0)	(2 , 3)	(3 , 4)
2	(0 , 2)	(2 , 0)	(2 , 1)

Tur 4: satır 1 tek başına kaldı; karşılaştıracak başka satır yok.

Adım 13 / 15

DEĞİŞKENLER

R = [1]

C = [2]

degisti = False

a = 1

ÇIKTI

Kod İzleme: Yinelemeli Baskın Strateji Eleme

```

1 R, C = [0, 1, 2], [0, 1, 2] # kalanlar
2 def u1(i, j): return U[i][j][0]
3 def u2(i, j): return U[i][j][1]
4 degisti = True
5 while degisti:
6     degisti = False
7     for a in R: # satır a
8         if any(all(u1(b, j) > u1(a, j) for j in C)
9                 for b in R if b != a):
10            R.remove(a); degisti = True; break
11     for a in C: # sütun a
12         if any(all(u2(i, b) > u2(i, a) for i in R)
13                 for b in C if b != a):
14            C.remove(a); degisti = True; break
15 print(R, C)

```

Oyuncu 2 (sütun j)

1 (i)	0	1	2
0	(4 , 2)	(3 , 1)	(2 , 4)
1	(1 , 0)	(2 , 3)	(3 , 4)
2	(0 , 2)	(2 , 0)	(2 , 1)

Tur 4: sütun 2 tek başına kaldı; karşılaştıracak başka sütun yok.

Adım 14 / 15

DEĞİŞKENLER

R = [1]

C = [2]

degisti = False

a = 2

ÇIKTI

Kod İzleme: Yinelemeli Baskın Strateji Eleme

```

1 R, C = [0, 1, 2], [0, 1, 2] # kalanlar
2 def u1(i, j): return U[i][j][0]
3 def u2(i, j): return U[i][j][1]
4 degisti = True
5 while degisti:
6     degisti = False
7     for a in R: # satır a
8         if any(all(u1(b, j) > u1(a, j) for j in C)
9                 for b in R if b != a):
10            R.remove(a); degisti = True; break
11    for a in C: # sütun a
12        if any(all(u2(i, b) > u2(i, a) for i in R)
13                for b in C if b != a):
14            C.remove(a); degisti = True; break
15    print(R, C)

```

Oyuncu 2 (sütun j)

1 (i)	0	1	2
0	(4 , 2)	(3 , 1)	(2 , 4)
1	(1 , 0)	(2 , 3)	(3 , 4)
2	(0 , 2)	(2 , 0)	(2 , 1)

Değişiklik olmayan tur → dur. Geriye (1, 2) = (3, 4) kaldı: en iyi tepkiyle bulunan denginin aynısı.

Adım 15 / 15

DEĞİŞKENLER

R = [1]

C = [2]

degisti = False

ÇIKTI

[1] [2]

Klasik 2×2 Oyunlar: Saf Dengeleri Okumak

		sütun	
		Opera	Futbol
satır	Opera	(3 , 2)	(0 , 0)
	Futbol	(0 , 0)	(2 , 3)

turuncu: en iyi tepki · çerçeve: saf Nash dengesi

İki saf denge: hangisinde buluşulacağı belirsiz. Karışık dengede %52 buluşamama (Bölüm 2).

Adım 1 / 4

Hepsi aynı kural: sütun başına satırın, satır başına sütunun en iyi tepkisi; iki işaretli hücre = saf Nash dengesi. Karışık denge hesapları Bölüm 2'de.

Cinsiyetler Savaşı

Buluşmak ortak çıkar, nerede buluşulacağı çatışma.

Saf Nash: (Opera, Opera), (Futbol, Futbol)

Karışık: satır Opera 3/5, sütun Opera 2/5

Yapı: koordinasyon + çatışma

Klasik 2×2 Oyunlar: Saf Dengeleri Okumak

		sütun	
		Geyik	Tavşan
satır	Geyik	(2 , 2)	(0 , 1)
	Tavşan	(1 , 0)	(1 , 1)

turuncu: en iyi tepki · çerçeve: saf Nash dengesi

(Geyik, Geyik) herkese daha iyi, (Tavşan, Tavşan) risksiz: iki denge, güvene bağlı seçim.

Geyik Avı

Geyik ancak birlikte avlanır; tavşan tek başına garanti.

Saf Nash: (Geyik, Geyik), (Tavşan, Tavşan)

Karışık: satır Geyik 1/2, sütun Geyik 1/2

Yapı: güven ikilemi

Adım 2 / 4

Hepsi aynı kural: sütun başına satırın, satır başına sütunun en iyi tepkisi; iki işaretli hücre = saf Nash dengesi. Karışık denge hesapları Bölüm 2'de.

Klasik 2×2 Oyunlar: Saf Dengeleri Okumak

		sütun	
		Kır	Düz git
satır	Kır	(0 , 0)	(-1 , 1)
	Düz git	(1 , -1)	(-10 , -10)

turuncu: en iyi tepki · çerçeve: saf Nash dengesi

Dengeler farklı eylemlerde: biri kırar, biri geçer. Kavşak oyunu aynı yapıdadır.

Tavuk

İlk kıran "tavuk" olur; ikisi de düz giderse çarpışır.

Saf Nash: (Kır, Düz git), (Düz git, Kır)

Karışık: satır Kır 9/10, sütun Kır 9/10

Yapı: anti-koordinasyon

Adım 3 / 4

Hepsi aynı kural: sütun başına satırın, satır başına sütunun en iyi tepkisi; iki işaretli hücre = saf Nash dengesi. Karışık denge hesapları Bölüm 2'de.

Klasik 2×2 Oyunlar: Saf Dengeleri Okumak

		sütun	
		Yazı	Tura
satır	Yazı	(1 , -1)	(-1 , 1)
	Tura	(-1 , 1)	(1 , -1)

turuncu: en iyi tepki · çerçeve: saf Nash dengesi

Her hücrede birisi sapmak ister: saf denge yok, tek denge yarı yarıya karıştırmak.

Yazı-Tura Eşleştirme

Eşleşirse satır, eşleşmezse sütun kazanır.

Saf Nash dengesi yok

Karışık: satır Yazı 1/2, sütun Yazı 1/2

Yapı: sıfır toplamı

Adım 4 / 4

Hepsi aynı kural: sütun başına satırın, satır başına sütunun en iyi tepkisi; iki işaretli hücre = saf Nash dengesi. Karışık denge hesapları Bölüm 2'de.

Karışık Strateji (Mixed Strategy)

- Oyuncu eylemlerini bir **olasılık dağılımına** göre rastgele seçer.
- Kazanç artık rastgeledir; oyuncu **beklenen kazancını** en büyükler.
- Rakibi şaşırtmak ve sömürülmemek için kullanılır (TKM, penaltı, blöf, güvenlik devriyesi).
- **Nash teoremi** (1950): Her sonlu oyunun karışık stratejilerde en az bir Nash dengesi vardır (ispat: Bölüm 3).

Beklenen kazanç

Rakip $q = (q_T, q_K, q_M)$ oynarken Kağıt'ın beklenen kazancı:

$$E[\text{Kağıt}] = q_T \cdot (+1) + q_K \cdot 0 + q_M \cdot (-1)$$

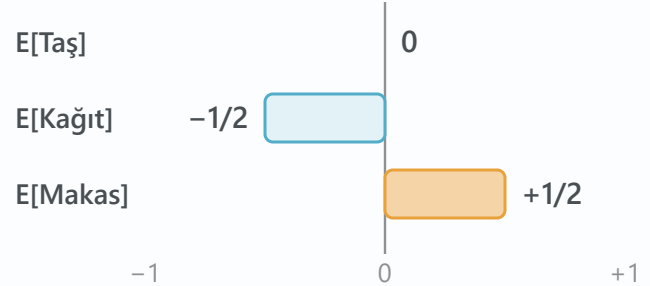
Dengede, kullanılan her eylemin beklenen kazancı eşittir (kayıtsızlık ilkesi).

Taş-Kağıt-Makas'ın Karışık Dengesi

Rakibin karışık stratejisi q



Satır oyuncusunun beklenen kazancı



$$E[\text{Taş}] = -q_K + q_M$$

$$E[\text{Kağıt}] = q_T - q_M$$

$$E[\text{Makas}] = -q_T + q_K$$

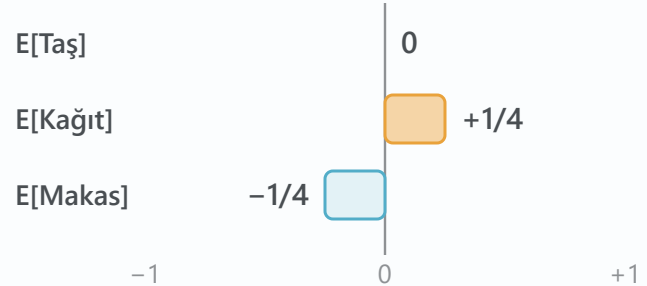
Rakip "yarı kağıt, yarı makas" oynarsa: $E[\text{Makas}] = +1/2$ en büyük. Hep Makas oynayarak onu **sömürürüz**.

Taş-Kağıt-Makas'ın Karışık Dengesi

Rakibin karışık stratejisi q



Satır oyuncusunun beklenen kazancı



$$E[\text{Taş}] = -q_K + q_M$$

$$E[\text{Kağıt}] = q_T - q_M$$

$$E[\text{Makas}] = -q_T + q_K$$

Rakip Taş'a fazla ağırlık verirse (1/2): Kağıt kazançlı (+1/4). Dengesiz her karışım sömürülebilir.

Taş-Kağıt-Makas'ın Karışık Dengesi

Rakibin karışık stratejisi q

Satır oyuncusunun beklenen kazancı

Taş

$q_T = ?$

$E[\text{Taş}]$

Kağıt

$q_K = ?$

$E[\text{Kağıt}]$

Makas

$q_M = ?$

$E[\text{Makas}]$

-1

0

+1

$$-q_K + q_M$$

$$= q_T - q_M$$

$$= -q_T + q_K$$

Rakibin sömürülmemesi için satır oyuncusu tüm eylemler arasında **kayıtsız** kalmalı: üç beklenen kazanç eşit.

Taş-Kağıt-Makas'ın Karışık Dengesi

Rakibin karışık stratejisi q

Satır oyuncusunun beklenen kazancı

Taş

$q_T = ?$

$E[\text{Taş}]$

Kağıt

$q_K = ?$

$E[\text{Kağıt}]$

Makas

$q_M = ?$

$E[\text{Makas}]$

-1

0

+1

Üçünün toplamı = 0

$\Rightarrow$ her biri = 0

$\Rightarrow q_T = q_K = q_M$

Üç ifadenin toplamı her q için 0'dır; eşitlerse her biri 0 olur. Buradan $q_M = q_K$ ve $q_T = q_M$.

Taş-Kağıt-Makas'ın Karışık Dengesi

Rakibin karışık stratejisi q

Taş



1/3

Kağıt



1/3

Makas



1/3

Satır oyuncusunun beklenen kazancı

$E[\text{Taş}]$

0

$E[\text{Kağıt}]$

0

$E[\text{Makas}]$

0

-1

0

+1

$q = (1/3, 1/3, 1/3)$

$E[\text{her eylem}] = 0$

sapma kazandırmaz

$q = (1/3, 1/3, 1/3)$: her eylem 0 getirir, hiçbir sapma kazandırmaz. Simetriyle iki oyuncu da $1/3-1/3-1/3$ oynar: **karışık Nash dengesi**, oyunun değeri 0.

Örnek: Kavşakta Kim Geçer?

		Sürücü 2	
		Geç	Dur
Sürücü 1	Geç	(-10 , -10)	(1 , 0)
	Dur	(0 , 1)	(-1 , -1)

Kaza: (-10, -10)

İkisi de bekler: (-1, -1)

Turuncu: en iyi tepki

Çerçeve: Nash dengesi

- Işıksız bir kavşağa iki sürücü aynı anda geliyor. Karşıdaki durursa geçmek, karşıdaki geçerse durmak en iyi tepkidir.
- İki saf Nash dengesi var: (Geç, Dur) ve (Dur, Geç). Ama hangisi? Koordinasyon yoksa karışık dengede her sürücü 1/6 olasılıkla geçer ve 1/36 olasılıkla **kaza** olur.

Bu oyun, klasik **Tavuk (Chicken)** oyununun bir biçimidir; karışık denge hesabı Bölüm 2'de.

Trafik Işıđı: Dengeyi Seçen Koordinasyon Aracı

Işık ne yapar?

- Rastgele (yarı yarıya) birine **yeşil**, diğİrine **kırmızı** verir.
- Yeşilde: karşıdaki duracak → **Geç** ($1 > -1$).
- Kırmızıda: karşıdaki geçecek → **Dur** ($0 > -10$).
- Işıđa uymak herkes için en iyi tepkidir.

Sonuç

	Beklenen kazanç	Kaza
Karışık denge	$-5/6$	$1/36$
Trafik ışığı	$+1/2$	0

Ortak sinyale dayalı bu dengeye **korelasyonlu denge** (Aumann) denir.

Dikkat: Sürücüler yeşilde **geçer**, kırmızıda durur. Işık sürelerini yoğunluđa göre ayarlamak bir Nash dengesi değıl, bir **tasarım** karardır. Sürücülerin rota seçimi: Bölüm 8.

Güvenlik Oyunları (Security Games)

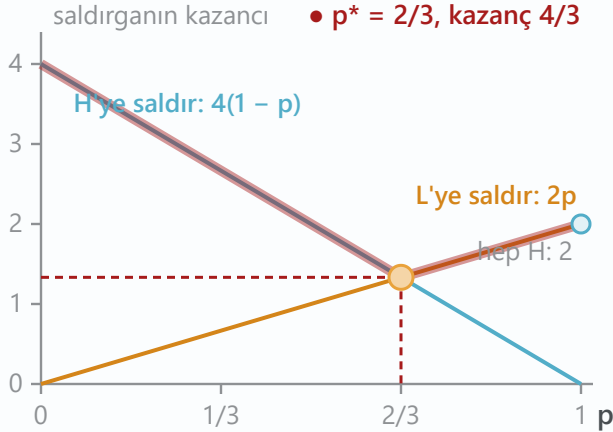
- Çok sayıda hedefin sınırlı kaynakla korunduğu durumları modeller: havalimanı, liman, orman devriyesi.
- n hedef var. **Oyuncu 1 (saldırgan)**: bir hedefe saldırır.
- **Oyuncu 2 (savunucu)**: aynı anda $k < n$ hedefi korur.
- Savunucunun saf stratejisi sayısı $C(n, k)$: $n = 20$, $k = 5$ için **15.504**. Kombinatorial patlama → verimli algoritmalar gerekir.
- Saldırgan devriye düzenini **gözlemleyebilir**: sabit düzen kolayca atlatılır, bu yüzden savunucu rastgeleleştirmelidir.

Uygulamada

Los Angeles Havalimanı'nda ARMOR sistemi (2007) kontrol noktalarını ve köpekli devriyeleri bu modelle rastgele planladı.

Kaynak: Tambe, *Security and Game Theory*, 2011.

Çalışılmış Örnek: Tek Bekçi, İki Hedef



Kurulum ve kayıtsızlık

Havalimanı H değeri 4, liman L değeri 2.

Bekçi H'yi p , L'yi $1 - p$ olasılıkla korur.

Korunmayan hedefe saldırın değerini kazanır, korunana saldırın 0.

H'ye saldırı: $(1 - p) \cdot 4$ · L'ye saldırı: $p \cdot 2$

Eşitle: $4 - 4p = 2p \Rightarrow p = 2/3$; saldırının kazancı $(1/3) \cdot 4 = 4/3$.

Saldırgan üst zarfı (kalın) seçer; bekçi onun en alçak noktasını arar.

Bekçi hep H'yi korursa saldırın L'den 2 alır. Rastgeleleştirme bunu $4/3$ 'e indirir; değerli hedef daha sık korunur ama hiçbir hedef tamamen açık bırakılmaz.

Mahkûmun İkilemi (Prisoner's Dilemma)

- İki şüpheli (Sam ve John) ayrı odalarda sorgulanıyor; birbirleriyle konuşamıyorlar.
- Her birinin iki seçeneği var: **sessiz kalmak** ya da suç ortağını ele verip **itiraf etmek**.

Durum	Sam	John
İkisi de sessiz	1 yıl	1 yıl
Sam itiraf eder, John sessiz	serbest	3 yıl
Sam sessiz, John itiraf eder	3 yıl	serbest
İkisi de itiraf eder	2 yıl	2 yıl

Terim birliği: İşbirliği = suç ortağıyla işbirliği = **sessiz kalmak**. **İhanet** = itiraf etmek (polisle işbirliği). Bundan sonra yalnızca "Sessiz" ve "İhanet" diyeceğiz.

Sam ve John'un Kazanç Matrisi

- Kazanç = $-(\text{hapis yılı})$: az ceza, büyük kazanç.
- Her hücrede ilk sayı **Sam'ın** (satır), ikinci sayı **John'un** (sütun) kazancı.
- İkisi de sessiz: ikisi de -1 .
- Sam sessiz, John ihanet: Sam -3 , John 0 .

	John: Sessiz	John: İhanet
Sam: Sessiz	a $(-1, -1)$	b $(-3, 0)$
Sam: İhanet	c $(0, -3)$	d $(-2, -2)$

Hücreler a, b, c, d olarak adlandırıldı.

Soru: Her mahkûm kendi cezasını en aza indirmek isterse ne olur? Sonraki slaytta en iyi tepkileri tek tek işaretliyoruz.

En İyi Tepkileri İşaretleyerek Dengeyi Bulmak

		John	
Sam		Sessiz	İhanet
Sessiz	a	(-1 , -1)	(-3 , 0)
	b		
İhanet	c	(0 , -3)	(-2 , -2)
	d		

Kazanç = -(hapis yılı)

Sam satır, John sütun oyuncusu

Hücre: (Sam'ın, John'un kazancı)

Kazançlar hapis yılının eksi işaretlisidir ($-1 = 1$ yıl). Her oyuncu kendi sayısını büyütmek ister.

En İyi Tepkileri İşaretleyerek Dengeyi Bulmak

		John	
Sam		Sessiz	İhanet
Sessiz	a	(-1 , -1)	b (-3 , 0)
	c	(0 , -3)	d (-2 , -2)
İhanet			

John Sessiz → Sam: İhanet ($0 > -1$)

John'un **Sessiz** kaldığını varsayalım: Sam bu sütunda yalnızca ilk sayılara bakar. $0 > -1$, en iyi tepki İhanet.

En İyi Tepkileri İşaretleyerek Dengeyi Bulmak

		John		
Sam		Sessiz	İhanet	
Sessiz	a	(-1 , -1)	b (-3 , 0)	John Sessiz → Sam: İhanet ($0 > -1$)
	c	(0 , -3)	d (-2 , -2)	John İhanet → Sam: İhanet ($-2 > -3$)
İhanet				

John **İhanet** ederse: $-2 > -3$, Sam için en iyi tepki yine İhanet.

En İyi Tepkileri İşaretleyerek Dengeyi Bulmak

John

	Sam	Sessiz	İhanet
Sessiz	a	(-1 , -1)	b (-3 , 0)
	c	(0 , -3)	d (-2 , -2)
İhanet			

John Sessiz → Sam: İhanet ($0 > -1$)
 John İhanet → Sam: İhanet ($-2 > -3$)
 ⇒ **Sam için İhanet baskın**

İki sütunda da işaret aynı satırda: John ne yaparsa yapsın İhanet daha iyi. Bu bir **baskın strateji**dir.

En İyi Tepkileri İşaretleyerek Dengeyi Bulmak

John

		Sessiz	İhanet
Sam			
Sessiz	a	(-1 , -1)	b (-3 , 0)
İhanet	c	(0 , -3)	d (-2 , -2)

John Sessiz → Sam: İhanet ($0 > -1$)

John İhanet → Sam: İhanet ($-2 > -3$)

⇒ **Sam için İhanet baskın**

Sam Sessiz → John: İhanet ($0 > -1$)

Şimdi John: Sam Sessiz kalırsa John ikinci sayılara bakar: $0 > -1$, en iyi tepki İhanet.

En İyi Tepkileri İşaretleyerek Dengeyi Bulmak

		John	
		Sessiz	İhanet
Sam	Sessiz	a (-1 , -1)	b (-3 , 0)
	İhanet	c (0 , -3)	d (-2 , -2)

John Sessiz → Sam: İhanet ($0 > -1$)

John İhanet → Sam: İhanet ($-2 > -3$)

⇒ **Sam için İhanet baskın**

Sam Sessiz → John: İhanet ($0 > -1$)

Sam İhanet → John: İhanet ($-2 > -3$)

Sam İhanet ederse: $-2 > -3$, John için de İhanet. Oyun simetrik olduğu için sonuç şaşırtıcı değil.

En İyi Tepkileri İşaretleyerek Dengeyi Bulmak

		John	
		Sessiz	İhanet
Sam	Sessiz	a (-1 , -1)	b (-3 , 0)
	İhanet	c (0 , -3)	d (-2 , -2)

John Sessiz → Sam: İhanet ($0 > -1$)

John İhanet → Sam: İhanet ($-2 > -3$)

⇒ Sam için İhanet baskın

Sam Sessiz → John: İhanet ($0 > -1$)

Sam İhanet → John: İhanet ($-2 > -3$)

⇒ John için de İhanet baskın

İki işaretin bulunduğu tek hücre d = (İhanet, İhanet): kimse tek başına sapmak istemez. **Nash dengesi** $(-2, -2)$.

En İyi Tepkileri İşaretleyerek Dengeyi Bulmak

		John	
		Sessiz	İhanet
Sam	Sessiz	a (-1 , -1)	b (-3 , 0)
	İhanet	c (0 , -3)	d (-2 , -2)

John Sessiz → Sam: İhanet ($0 > -1$)

John İhanet → Sam: İhanet ($-2 > -3$)

⇒ **Sam için İhanet baskın**

Sam Sessiz → John: İhanet ($0 > -1$)

Sam İhanet → John: İhanet ($-2 > -3$)

⇒ **John için de İhanet baskın**

a = (-1, -1) ikisi için de daha iyi!

Oysa a = (Sessiz, Sessiz) ikisine de yalnızca 1'er yıl verirdi. Denge **Pareto optimal değil**: ikilemin özü budur.

İhanet Neden Baskın? Durum Analizi

Durum 1: John sessiz kalırsa

Sam sessiz: 1 yıl (hücre a) · Sam ihanet: serbest (hücre c) ⇒ **ihanet daha iyi** ($0 < 1$ yıl).

Adım 1 / 4

İhanet Neden Baskın? Durum Analizi

Durum 1: John sessiz kalırsa

Sam sessiz: 1 yıl (hücre a) · Sam ihanet: serbest (hücre c) ⇒ **ihanet daha iyi** ($0 < 1$ yıl).

Durum 2: John ihanet ederse

Sam sessiz: 3 yıl (hücre b) · Sam ihanet: 2 yıl (hücre d) ⇒ **ihanet daha iyi** ($2 < 3$ yıl).

İhanet Neden Baskın? Durum Analizi

Durum 1: John sessiz kalırsa

Sam sessiz: 1 yıl (hücre a) · Sam ihanet: serbest (hücre c) ⇒ **ihanet daha iyi** ($0 < 1$ yıl).

Durum 2: John ihanet ederse

Sam sessiz: 3 yıl (hücre b) · Sam ihanet: 2 yıl (hücre d) ⇒ **ihanet daha iyi** ($2 < 3$ yıl).

John **ne yaparsa yapsın** ihanet daha az ceza getiriyor: ihanet **baskın strateji**. John'un olasılıklarını bilmek gerekmez; “ortalama cezayı azaltır” gerekçesi yanlıştır.

İhanet Neden Baskın? Durum Analizi

Durum 1: John sessiz kalırsa

Sam sessiz: 1 yıl (hücre a) · Sam ihanet: serbest (hücre c) ⇒ **ihanet daha iyi** ($0 < 1$ yıl).

Durum 2: John ihanet ederse

Sam sessiz: 3 yıl (hücre b) · Sam ihanet: 2 yıl (hücre d) ⇒ **ihanet daha iyi** ($2 < 3$ yıl).

John **ne yaparsa yapsın** ihanet daha az ceza getiriyor: ihanet **baskın strateji**. John'un olasılıklarını bilmek gerekmez; "ortalama cezayı azaltır" gerekçesi yanlıştır.

John da aynı ikilemi yaşar; onun için de ihanet baskın. Sonuç hücre **d**: ikisi de **2'şer yıl**. Oysa ikisi de sessiz kalsaydı (hücre a) **1'er yıl** yatacaklardı: itiraf "ikisi için en iyi sonuç" **değildir**; bireysel en iyi seçim ikisini birden kötüleştirir.

Kod İzleme: Saf Nash Dengelerini Bulmak

```

▶ U = {"S","S": (-1,-1), ("S","İ": (-3, 0),
▶     ("İ","S"): ( 0,-3), ("İ","İ": (-2,-2)}

```

```

3 A = ["S", "İ"]          # Sessiz, İhanet
4 def nash(U):
5     denge = []
6     for a in A:          # Sam'ın eylemi
7         for b in A:      # John'un eylemi
8             u1, u2 = U[a, b]
9             iyi1 = all(u1 >= U[x, b][0] for x in A)
10            iyi2 = all(u2 >= U[a, y][1] for y in A)
11            if iyi1 and iyi2:
12                denge.append((a, b))
13    return denge
14    print(nash(U))

```

		John	
		S	İ
Sam	S	(-1, -1)	(-3, 0)
	İ	(0, -3)	(-2, -2)

Kazanç tablosu bir sözlükte: anahtar (Sam, John) eylem çifti, değer (u_{Sam} , u_{John}).

Adım 1 / 11

DEĞİŞKENLER

—

ÇIKTI

Kod İzleme: Saf Nash Dengelerini Bulmak

```

1 U = {"S","S": (-1,-1), ("S","İ": (-3, 0),
2     ("İ","S"): ( 0,-3), ("İ","İ": (-2,-2)}
3 A = ["S", "İ"]           # Sessiz, İhanet
4 def nash(U):
5     ► denge = []
6     for a in A:           # Sam'ın eylemi
7         for b in A:       # John'un eylemi
8             u1, u2 = U[a, b]
9             iyi1 = all(u1 >= U[x, b][0] for x in A)
10            iyi2 = all(u2 >= U[a, y][1] for y in A)
11            if iyi1 and iyi2:
12                denge.append((a, b))
13    return denge
14    print(nash(U))

```

		John	
		S	İ
Sam	S	(-1, -1)	(-3, 0)
	İ	(0, -3)	(-2, -2)

Dengeleri toplayacağımız liste boş başlar; şimdi dört hücreyi tek tek tarayacağız.

Adım 2 / 11

DEĞİŞKENLER

denge = []

ÇIKTI



Kod İzleme: Saf Nash Dengelerini Bulmak

```

1 U = {("S","S"): (-1,-1), ("S","İ"): (-3, 0),
2     ("İ","S"): ( 0,-3), ("İ","İ"): (-2,-2)}
3 A = ["S", "İ"]           # Sessiz, İhanet
4 def nash(U):
5     denge = []
6     for a in A:           # Sam'ın eylemi
7         for b in A:       # John'un eylemi
8             ▶ u1, u2 = U[a, b]
9             ▶ iyi1 = all(u1 >= U[x, b][0] for x in A)
10            iyi2 = all(u2 >= U[a, y][1] for y in A)
11            if iyi1 and iyi2:
12                denge.append((a, b))
13 return denge
14 print(nash(U))

```

		John	
		S	İ
Sam	S	(-1, -1)	(-3, 0)
	İ	(0, -3)	(-2, -2)

Hücre (S, S): Sam İ oynasa 0 alırdı; $-1 < 0 \Rightarrow$ iyi1 = False.

Adım 3 / 11

DEĞİŞKENLER

a = S

b = S

u1 = -1

u2 = -1

iyi1 = False

denge = []

ÇIKTI

Kod İzleme: Saf Nash Dengelerini Bulmak

```

1 U = {"S","S": (-1,-1), ("S","İ"): (-3, 0),
2     ("İ","S"): ( 0,-3), ("İ","İ"): (-2,-2)}
3 A = ["S", "İ"]           # Sessiz, İhanet
4 def nash(U):
5     denge = []
6     for a in A:           # Sam'ın eylemi
7         for b in A:       # John'un eylemi
8             u1, u2 = U[a, b]
9             iyi1 = all(u1 >= U[x, b][0] for x in A)
10            iyi2 = all(u2 >= U[a, y][1] for y in A)
11            if iyi1 and iyi2:
12                denge.append((a, b))
13     return denge
14 print(nash(U))

```

		John	
		S	İ
Sam	S	(-1, -1)	(-3, 0)
	İ	(0, -3)	(-2, -2)

John İ oynasa 0 alırdı; $-1 < 0 \Rightarrow iyi2 = \text{False}$;
koşul sağlanmadı, denge değil.

Adım 4 / 11

DEĞİŞKENLER

a = S

b = S

u1 = -1

u2 = -1

iyi1 = False

iyi2 = False

denge = []

ÇIKTI

Kod İzleme: Saf Nash Dengelerini Bulmak

```

1 U = {("S","S"): (-1,-1), ("S","İ"): (-3, 0),
2     ("İ","S"): ( 0,-3), ("İ","İ"): (-2,-2)}
3 A = ["S", "İ"]           # Sessiz, İhanet
4 def nash(U):
5     denge = []
6     for a in A:           # Sam'ın eylemi
7         for b in A:       # John'un eylemi
8             ▶ u1, u2 = U[a, b]
9             ▶ iyi1 = all(u1 >= U[x, b][0] for x in A)
10            iyi2 = all(u2 >= U[a, y][1] for y in A)
11            if iyi1 and iyi2:
12                denge.append((a, b))
13    return denge
14    print(nash(U))

```

		John	
		S	İ
Sam	S	(-1, -1)	(-3, 0)
	İ	(0, -3)	(-2, -2)

Hücre (S, İ): Sam İ oynasa -2 alırdı; $-3 < -2$
 $\Rightarrow$ iyi1 = False.

Adım 5 / 11

DEĞİŞKENLER

a = S

b = İ

u1 = -3

u2 = 0

iyi1 = False

denge = []

ÇIKTI



Kod İzleme: Saf Nash Dengelerini Bulmak

```

1 U = {("S","S"): (-1,-1), ("S","İ"): (-3, 0),
2     ("İ","S"): ( 0,-3), ("İ","İ"): (-2,-2)}
3 A = ["S", "İ"]           # Sessiz, İhanet
4 def nash(U):
5     denge = []
6     for a in A:           # Sam'ın eylemi
7         for b in A:       # John'un eylemi
8             u1, u2 = U[a, b]
9             iyi1 = all(u1 >= U[x, b][0] for x in A)
10            iyi2 = all(u2 >= U[a, y][1] for y in A)
11            if iyi1 and iyi2:
12                denge.append((a, b))
13     return denge
14 print(nash(U))

```

		John	
		S	İ
Sam	S	(-1, -1)	(-3, 0)
	İ	(0, -3)	(-2, -2)

John S oynasa -1 alırdı; $0 \geq -1 \Rightarrow iyi2 = \text{True}$; koşul sağlanmadı, denge değil.

Adım 6 / 11

DEĞİŞKENLER

a = S b = İ u1 = -3 u2 = 0

iyi1 = False iyi2 = True

denge = []

ÇIKTI



Kod İzleme: Saf Nash Dengelerini Bulmak

```

1 U = {("S","S"): (-1,-1), ("S","İ"): (-3, 0),
2     ("İ","S"): ( 0,-3), ("İ","İ"): (-2,-2)}
3 A = ["S", "İ"]           # Sessiz, İhanet
4 def nash(U):
5     denge = []
6     for a in A:           # Sam'ın eylemi
7         for b in A:       # John'un eylemi
8             ▶ u1, u2 = U[a, b]
9             ▶ iyi1 = all(u1 >= U[x, b][0] for x in A)
10            iyi2 = all(u2 >= U[a, y][1] for y in A)
11            if iyi1 and iyi2:
12                denge.append((a, b))
13    return denge
14    print(nash(U))

```

		John	
		S	İ
Sam	S	(-1, -1)	(-3, 0)
	İ	(0, -3)	(-2, -2)

Hücre (İ, S): Sam S oynasa -1 alırdı; $0 \geq -1$
 $\Rightarrow$ iyi1 = True.

Adım 7 / 11

DEĞİŞKENLER

a = İ

b = S

u1 = 0

u2 = -3

iyi1 = True

denge = []

ÇIKTI



Kod İzleme: Saf Nash Dengelerini Bulmak

```

1 U = {"S","S": (-1,-1), ("S","İ": (-3, 0),
2     ("İ","S"): ( 0,-3), ("İ","İ": (-2,-2)}
3 A = ["S", "İ"]           # Sessiz, İhanet
4 def nash(U):
5     denge = []
6     for a in A:           # Sam'ın eylemi
7         for b in A:       # John'un eylemi
8             u1, u2 = U[a, b]
9             iyi1 = all(u1 >= U[x, b][0] for x in A)
10            iyi2 = all(u2 >= U[a, y][1] for y in A)
11            if iyi1 and iyi2:
12                denge.append((a, b))
13     return denge
14 print(nash(U))

```

		John	
		S	İ
Sam	S	(-1, -1)	(-3, 0)
	İ	(0, -3)	(-2, -2)

John İ oynasa -2 alırdı; $-3 < -2 \Rightarrow iyi2 = \text{False}$; koşul sağlanmadı, denge değil.

Adım 8 / 11

DEĞİŞKENLER

a = İ

b = S

u1 = 0

u2 = -3

iyi1 = True

iyi2 = False

denge = []

ÇIKTI



Kod İzleme: Saf Nash Dengelerini Bulmak

```

1 U = {("S","S"): (-1,-1), ("S","İ"): (-3, 0),
2     ("İ","S"): ( 0,-3), ("İ","İ"): (-2,-2)}
3 A = ["S", "İ"]           # Sessiz, İhanet
4 def nash(U):
5     denge = []
6     for a in A:           # Sam'ın eylemi
7         for b in A:       # John'un eylemi
8             ▶ u1, u2 = U[a, b]
9             ▶ iyi1 = all(u1 >= U[x, b][0] for x in A)
10            iyi2 = all(u2 >= U[a, y][1] for y in A)
11            if iyi1 and iyi2:
12                denge.append((a, b))
13    return denge
14    print(nash(U))

```

		John	
		S	İ
Sam	S	(-1, -1)	(-3, 0)
	İ	(0, -3)	(-2, -2)

Hücre (İ, İ): Sam S oynasa -3 alırdı; $-2 \geq -3$
 $\Rightarrow$ iyi1 = True.

Adım 9 / 11

DEĞİŞKENLER

a = İ

b = İ

u1 = -2

u2 = -2

iyi1 = True

denge = []

ÇIKTI



Kod İzleme: Saf Nash Dengelerini Bulmak

```

1 U = {("S","S"): (-1,-1), ("S","İ"): (-3, 0),
2     ("İ","S"): ( 0,-3), ("İ","İ"): (-2,-2)}
3 A = ["S", "İ"]          # Sessiz, İhanet
4 def nash(U):
5     denge = []
6     for a in A:          # Sam'ın eylemi
7         for b in A:      # John'un eylemi
8             u1, u2 = U[a, b]
9             iyi1 = all(u1 >= U[x, b][0] for x in A)
10            iyi2 = all(u2 >= U[a, y][1] for y in A)
11            if iyi1 and iyi2:
12                denge.append((a, b))
13 return denge
14 print(nash(U))

```

		John	
		S	İ
Sam	S	(-1, -1)	(-3, 0)
	İ	(0, -3)	(-2, -2)

John S oynasa -3 alırdı; $-2 \geq -3 \Rightarrow iyi2 = \text{True}$; iki koşul da doğru $\rightarrow$ dengeye eklenir.

Adım 10 / 11

DEĞİŞKENLER

a = İ

b = İ

u1 = -2

u2 = -2

iyi1 = True

iyi2 = True

denge = [(İ, İ)]

ÇIKTI



Kod İzleme: Saf Nash Dengelerini Bulmak

```

1 U = {("S","S"): (-1,-1), ("S","İ"): (-3, 0),
2     ("İ","S"): ( 0,-3), ("İ","İ"): (-2,-2)}
3 A = ["S", "İ"]           # Sessiz, İhanet
4 def nash(U):
5     denge = []
6     for a in A:           # Sam'ın eylemi
7         for b in A:       # John'un eylemi
8             u1, u2 = U[a, b]
9             iyi1 = all(u1 >= U[x, b][0] for x in A)
10            iyi2 = all(u2 >= U[a, y][1] for y in A)
11            if iyi1 and iyi2:
12                denge.append((a, b))
13
14     return denge
15     print(nash(U))

```

		John	
		S	İ
Sam	S	(-1, -1)	(-3, 0)
	İ	(0, -3)	(-2, -2)

Tek saf Nash dengesi (İhanet, İhanet). Kod, elle yaptığımız en iyi tepki işaretlemesinin aynısıdır.

Adım 11 / 11

DEĞİŞKENLER

denge = [(İ, İ)]

ÇIKTI

[('İ', 'İ')]

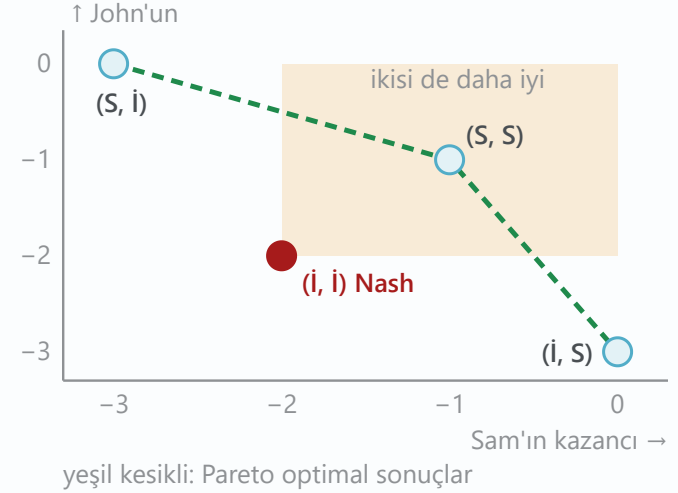
İkilemin Mesajı: Bireysel Rasyonellik, Ortak Kötü Sonuç

- Oyunun tek Nash dengesi (İhanet, İhanet) = $(-2, -2)$.
- (Sessiz, Sessiz) = $(-1, -1)$ ikisi için de daha iyi: denge **Pareto optimal değil**.
- Anlaşma yapsalar bile bağlayıcı değil (işbirlikçi olmayan oyun): her biri sözünü bozmaya teşviklidir.

Mesaj: Herkesin kendi çıkarına rasyonel davranması, toplum için iyi sonucu garanti etmez.

Nerede görülür?

Fiyat savaşları, silahlanma yarışı, iklim anlaşmaları, sporda doping, ortak kaynakların aşırı kullanımı.



Çıkış yolları

Tekrarlanan oyun ve itibar (Tit-for-Tat, Bölüm 12), sözleşme ve teşvik tasarımı (Bölüm 4).

2/3 Ortalamasını Tahmin Et

Oyun kuralları

1. Her katılımcı 0 ile 90 arasında (0 ve 90 dahil) bir sayı yazar.
2. Tüm tahminlerin ortalaması hesaplanır.
3. **Ortalamanın 2/3'üne** en yakın tahmini yapan kazanır (eşitlikte ödül paylaşılır).

Önce siz deneyin!

Bir kâğıda tahmininizi yazın, sonra slaytlara devam edin.

[Etkileşimli sürüm: 2/3 Ortalaması Oyunu →](#)

Kazanmak için yalnızca oyunu değil, **diğerlerinin ne düşündüğünü** (ve sizin ne düşündüğünüzü ne düşündüklerini) tahmin etmeniz gerekir.

Seviye-k Düşünme: 45 → 30 → 20 ...

Düşünme seviyesi	Varsayım	Tahmin
Seviye 0	Herkes rastgele yazar, ortalama 45	—
Seviye 1	Diğerleri seviye 0	$2/3 \cdot 45 = 30$
Seviye 2	Diğerleri seviye 1	$2/3 \cdot 30 = 20$
Seviye 3	Diğerleri seviye 2	$\approx 13,3$
Seviye ∞	Herkes tam rasyonel	0

- “Ortalama 45 olur, 2/3'ü 30” akıl yürütmesi yalnızca **seviye 1**'dir.
- Başkalarının da 30 diyeceğini düşünen daha düşük (ör. 20) yazar; herkesin fazla düşük yazacağını düşünen biraz yukarı kayabilir.
- En iyi tahmin, diğerlerinin kaçınıcı seviyede düşündüğüne dair inancınıza bağlıdır.

Kod İzleme: Seviye-k Akıl Yürütme ve En İyi Tahmin



Seviye 0 rastgele yazar (ortalama 45). Seviye k, herkesi seviye k-1 sanıp onun 2/3'ünü yazar.

Adım 1 / 10

```
▶ def tahmin(k, L0=45):      # seviye-k
▶     g = L0
▶     for _ in range(k):
▶         g = g * 2 / 3
▶     return g

6 pay = {0: 0.3, 1: 0.4, 2: 0.3} # inanç
7 ort = sum(p * tahmin(k) for k, p in pay.items())
8 hedef = ort * 2 / 3
9 for k in range(5):
10     print(k, round(tahmin(k), 1),
11           round(abs(tahmin(k) - hedef), 1))
```

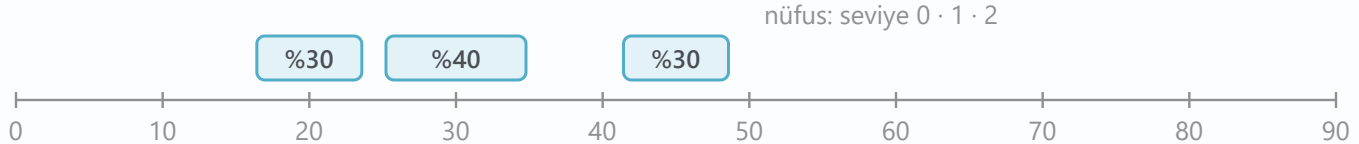
DEĞİŞKENLER

L0 = 45

ÇIKTI



Kod İzleme: Seviye-k Akıl Yürütme ve En İyi Tahmin



İnanç: sınıfın %30'u seviye 0, %40'ı seviye 1, %30'u seviye 2 düşünüyor.

Adım 2 / 10

```
1 def tahmin(k, L0=45):      # seviye-k
2     g = L0
3     for _ in range(k):
4         g = g * 2 / 3
5     return g
▶ pay = {0: 0.3, 1: 0.4, 2: 0.3} # inanç
7 ort = sum(p * tahmin(k) for k, p in pay.items())
8 hedef = ort * 2 / 3
9 for k in range(5):
10     print(k, round(tahmin(k), 1),
11           round(abs(tahmin(k) - hedef), 1))
```

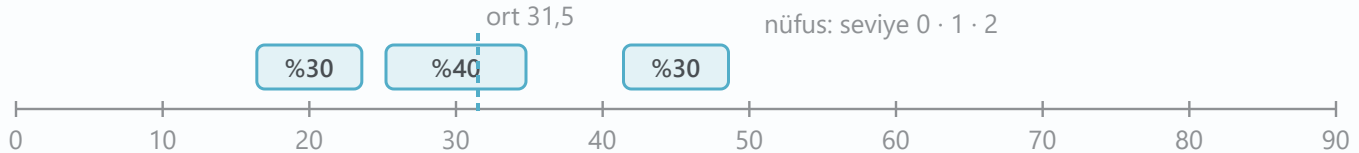
DEĞİŞKENLER

pay = {0: 0.3, 1: 0.4, 2: 0.3}

ÇIKTI



Kod İzleme: Seviye-k Akıl Yürütme ve En İyi Tahmin



Beklenen ortalama: $0,3 \cdot 45 + 0,4 \cdot 30 + 0,3 \cdot 20 = 31,5$.

Adım 3 / 10

```

1 def tahmin(k, L0=45):      # seviye-k
2     g = L0
3     for _ in range(k):
4         g = g * 2 / 3
5     return g
6 pay = {0: 0.3, 1: 0.4, 2: 0.3} # inanç
7 ort = sum(p * tahmin(k) for k, p in pay.items())
8 hedef = ort * 2 / 3
9 for k in range(5):
10    print(k, round(tahmin(k), 1),
11          round(abs(tahmin(k) - hedef), 1))

```

DEĞİŞKENLER

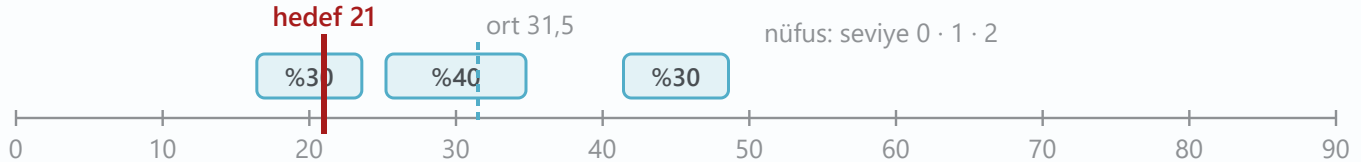
pay = {0: 0.3, 1: 0.4, 2: 0.3}

ort = 31.5

ÇIKTI

[Empty output box]

Kod İzleme: Seviye-k Akıl Yürütme ve En İyi Tahmin



Kazanan sayı ortalamanın 2/3'ü: **hedef = 21**.

Adım 4 / 10

```

1 def tahmin(k, L0=45):      # seviye-k
2     g = L0
3     for _ in range(k):
4         g = g * 2 / 3
5     return g
6 pay = {0: 0.3, 1: 0.4, 2: 0.3} # inanç
7 ort = sum(p * tahmin(k) for k, p in pay.items())
8 hedef = ort * 2 / 3
9 for k in range(5):
10    print(k, round(tahmin(k), 1),
11          round(abs(tahmin(k) - hedef), 1))

```

DEĞİŞKENLER

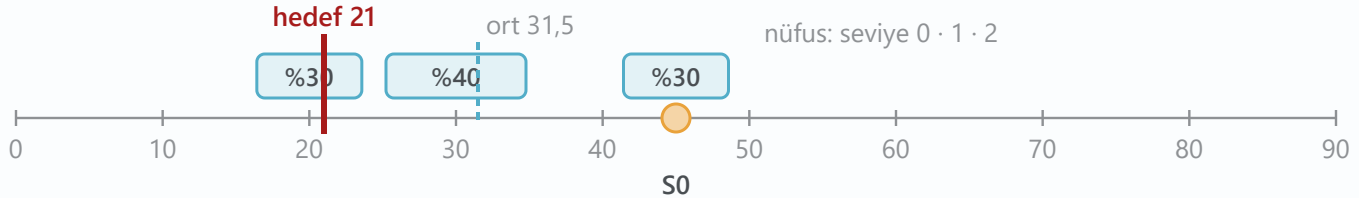
ort = 31.5

hedef = 21.0

ÇIKTI



Kod İzleme: Seviye-k Akıl Yürütme ve En İyi Tahmin



Seviye 0: tahmin 45, hedefe uzaklık 24. Ortalamayı yazmak çok yüksek.

Adım 5 / 10

```
1 def tahmin(k, L0=45):      # seviye-k
2     g = L0
3     for _ in range(k):
4         g = g * 2 / 3
5     return g
6 pay = {0: 0.3, 1: 0.4, 2: 0.3} # inanç
7 ort = sum(p * tahmin(k) for k, p in pay.items())
8 hedef = ort * 2 / 3

▶ for k in range(5):
▶     print(k, round(tahmin(k), 1),
▶         round(abs(tahmin(k) - hedef), 1))
```

DEĞİŞKENLER

k = 0

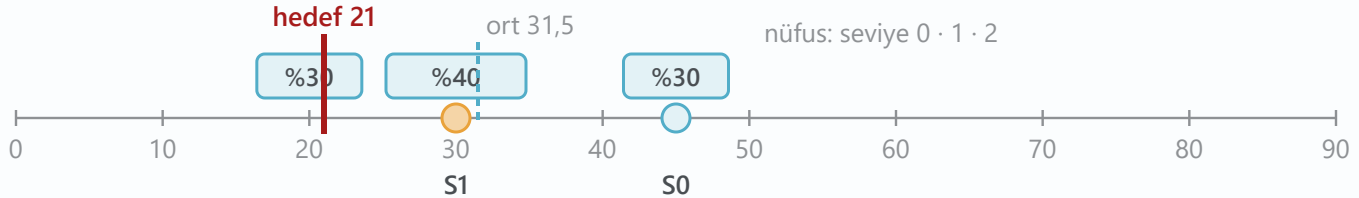
tahmin = 45.0

uzaklik = 24.0

ÇIKTI

0 45 24.0

Kod İzleme: Seviye-k Akıl Yürütme ve En İyi Tahmin



Seviye 1: tahmin 30, hedefe uzaklık 9. Klasik "ortalama 45 → 30" cevabı: hâlâ uzak.

Adım 6 / 10

```
1 def tahmin(k, L0=45):      # seviye-k
2     g = L0
3     for _ in range(k):
4         g = g * 2 / 3
5     return g
6 pay = {0: 0.3, 1: 0.4, 2: 0.3} # inanç
7 ort = sum(p * tahmin(k) for k, p in pay.items())
8 hedef = ort * 2 / 3

▶ for k in range(5):
▶     print(k, round(tahmin(k), 1),
▶           round(abs(tahmin(k) - hedef), 1))
```

DEĞİŞKENLER

k = 1

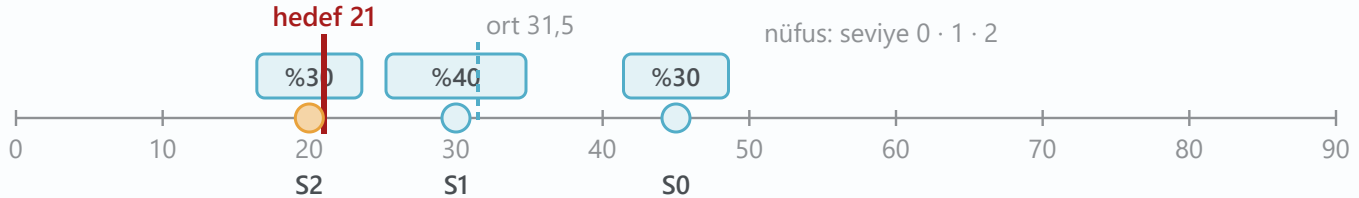
tahmin = 30.0

uzaklik = 9.0

ÇIKTI

```
0 45 24.0
1 30.0 9.0
```

Kod İzleme: Seviye-k Akıl Yürütme ve En İyi Tahmin



Seviye 2: tahmin 20, hedefe uzaklık 1. Hedefe en yakın: bu inançla **en iyi tepki**.

Adım 7 / 10

```
1 def tahmin(k, L0=45):      # seviye-k
2     g = L0
3     for _ in range(k):
4         g = g * 2 / 3
5     return g
6 pay = {0: 0.3, 1: 0.4, 2: 0.3} # inanç
7 ort = sum(p * tahmin(k) for k, p in pay.items())
8 hedef = ort * 2 / 3

▶ for k in range(5):
▶     print(k, round(tahmin(k), 1),
▶           round(abs(tahmin(k) - hedef), 1))
```

DEĞİŞKENLER

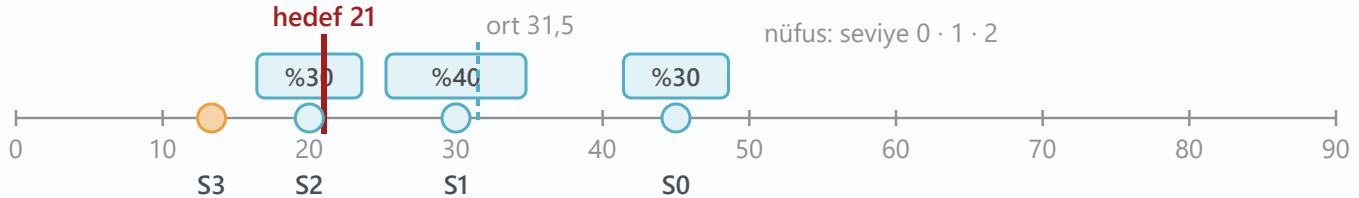
k = 2 tahmin = 20.0

uzaklik = 1.0

ÇIKTI

```
0 45 24.0
1 30.0 9.0
2 20.0 1.0
```

Kod İzleme: Seviye-k Akıl Yürütme ve En İyi Tahmin



Seviye 3: tahmin 13,3, hedefe uzaklık 7,7. Bir seviye fazla düşünmek de kaybettirir.

Adım 8 / 10

```
1 def tahmin(k, L0=45):      # seviye-k
2     g = L0
3     for _ in range(k):
4         g = g * 2 / 3
5     return g
6 pay = {0: 0.3, 1: 0.4, 2: 0.3} # inanç
7 ort = sum(p * tahmin(k) for k, p in pay.items())
8 hedef = ort * 2 / 3

▶ for k in range(5):
▶     print(k, round(tahmin(k), 1),
▶           round(abs(tahmin(k) - hedef), 1))
```

DEĞİŞKENLER

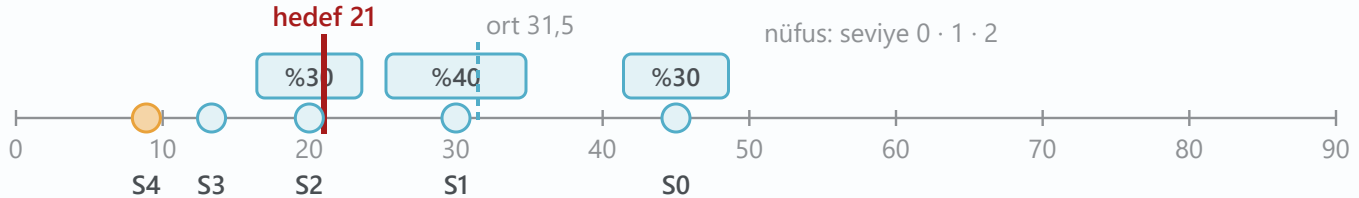
k = 3 tahmin = 13.33

uzaklik = 7.67

ÇIKTI

```
0 45 24.0
1 30.0 9.0
2 20.0 1.0
3 13.3 7.7
```

Kod İzleme: Seviye-k Akıl Yürütme ve En İyi Tahmin



Seviye 4: tahmin 8,9, hedefe uzaklık 12,1. Tam rasyonel cevaba (0) yaklaştıkça uzaklık artıyor.

Adım 9 / 10

```
1 def tahmin(k, L0=45):      # seviye-k
2     g = L0
3     for _ in range(k):
4         g = g * 2 / 3
5     return g
6 pay = {0: 0.3, 1: 0.4, 2: 0.3} # inanç
7 ort = sum(p * tahmin(k) for k, p in pay.items())
8 hedef = ort * 2 / 3

▶ for k in range(5):
▶     print(k, round(tahmin(k), 1),
▶           round(abs(tahmin(k) - hedef), 1))
```

DEĞİŞKENLER

k = 4

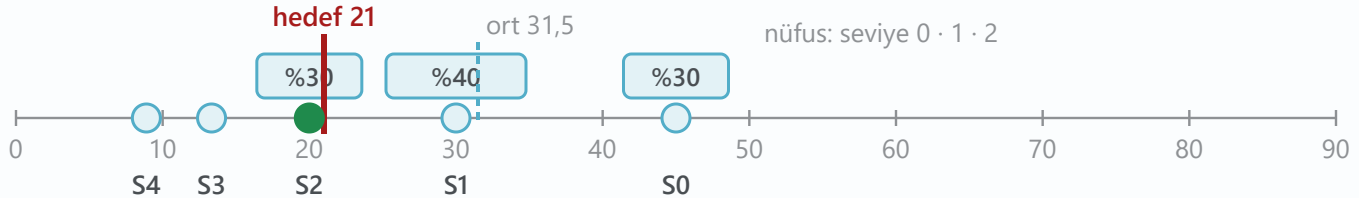
tahmin = 8.89

uzaklik = 12.11

ÇIKTI

```
0 45 24.0
1 30.0 9.0
2 20.0 1.0
3 13.3 7.7
4 8.9 12.1
```

Kod İzleme: Seviye-k Akıl Yürütme ve En İyi Tahmin



Sonuç: bu nüfusa karşı 20 yazan kazanır; Nash dengesi 0 ise kaybeder. En iyi tepki **inanca** bağlıdır.

Adım 10 / 10

```
1 def tahmin(k, L0=45):      # seviye-k
2     g = L0
3     for _ in range(k):
4         g = g * 2 / 3
5     return g
6 pay = {0: 0.3, 1: 0.4, 2: 0.3} # inanç
7 ort = sum(p * tahmin(k) for k, p in pay.items())
8 hedef = ort * 2 / 3
9
10 for k in range(5):
11     print(k, round(tahmin(k), 1),
12           round(abs(tahmin(k) - hedef), 1))
```

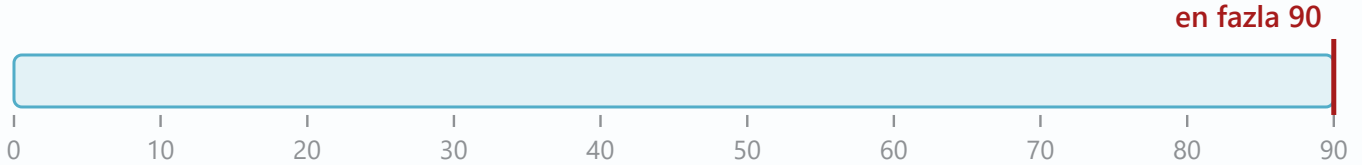
DEĞİŞKENLER

en_iiyi = seviye 2 tahmin = 20.0

ÇIKTI

```
0 45 24.0
1 30.0 9.0
2 20.0 1.0
3 13.3 7.7
4 8.9 12.1
```

Yinelemeli Eleme: $90 \rightarrow 60 \rightarrow 40 \rightarrow \dots \rightarrow 0$



Başlangıç: her tahmin 0 ile 90 arasında. Hiçbir şey elenmedi.

Adım 1 / 8

```
► ust = 90 # olası en büyük tahmin
2 for tur in range(1, 7):
3     # ortalama ≤ ust ⇒ hedef ≤ (2/3)·ust
4     ust = ust * 2 / 3
5     print(tur, round(ust, 1))
6 # ust = 90·(2/3)^k → 0 : tek denge herkes 0
```

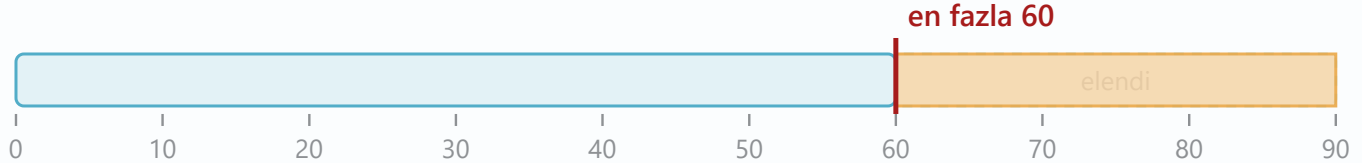
DEĞİŞKENLER

ust = 90

ÇIKTI



Yinelemeli Eleme: $90 \rightarrow 60 \rightarrow 40 \rightarrow \dots \rightarrow 0$



Ortalama en fazla 90 olabilir, hedef ($\frac{2}{3} \times \text{ortalama}$) en fazla 60. 60'ın üstündeki bir tahmin 60'tan asla daha iyi değildir → elenir.

Adım 2 / 8

```
1 ust = 90 # olası en büyük tahmin
2 for tur in range(1, 7):
3     # ortalama ≤ ust ⇒ hedef ≤ (2/3)·ust
4     ▶ ust = ust * 2 / 3
5     ▶ print(tur, round(ust, 1))
6 # ust = 90·(2/3)^k → 0 : tek denge herkes 0
```

DEĞİŞKENLER

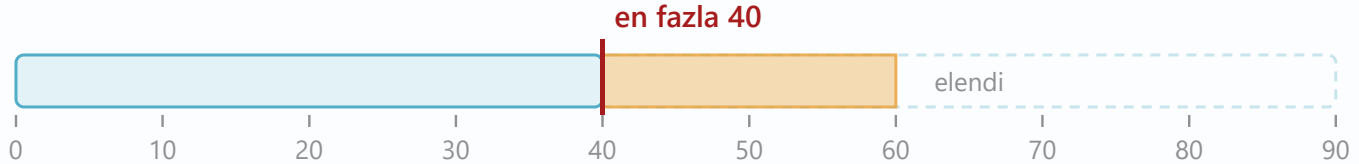
tur = 1

ust = 60.0

ÇIKTI

1 60.0

Yinelemeli Eleme: $90 \rightarrow 60 \rightarrow 40 \rightarrow \dots \rightarrow 0$



Herkes rasyonel ve bunu herkes biliyor: kimse 60'ı aşmaz. $\text{Ortalama} \leq 60 \Rightarrow \text{hedef} \leq 40$. 40'ın üstü elenir.

Adım 3 / 8

```
1 ust = 90 # olası en büyük tahmin
2 for tur in range(1, 7):
3     # ortalama ≤ ust ⇒ hedef ≤ (2/3)·ust
4     ▶ ust = ust * 2 / 3
5     ▶ print(tur, round(ust, 1))
6 # ust = 90·(2/3)^k → 0 : tek denge herkes 0
```

DEĞİŞKENLER

tur = 2

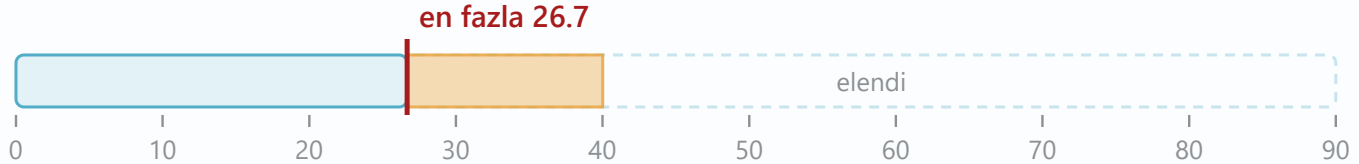
ust = 40.0

ÇIKTI

1 60.0

2 40.0

Yinelemeli Eleme: $90 \rightarrow 60 \rightarrow 40 \rightarrow \dots \rightarrow 0$



Bir tur daha: $\text{ortalama} \leq 40 \Rightarrow \text{hedef} \leq 26,7$. “Herkesin, herkesin rasyonel olduğunu bildiğini” bilmek gerekir.

Adım 4 / 8

```
1 ust = 90 # olası en büyük tahmin
2 for tur in range(1, 7):
3     # ortalama ≤ ust ⇒ hedef ≤ (2/3)·ust
4     ▶ ust = ust * 2 / 3
5     ▶ print(tur, round(ust, 1))
6 # ust = 90·(2/3)^k → 0 : tek denge herkes 0
```

DEĞİŞKENLER

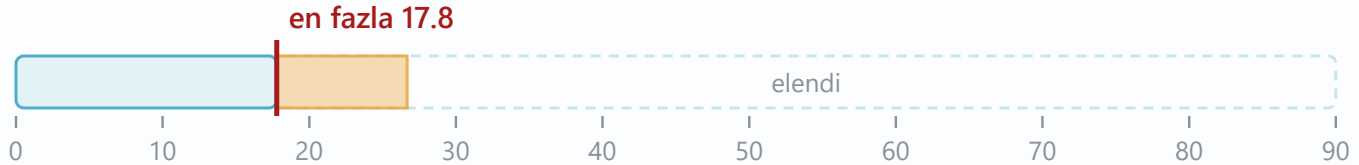
tur = 3

ust = 26.67

ÇIKTI

```
1 60.0
2 40.0
3 26.7
```

Yinelemeli Eleme: $90 \rightarrow 60 \rightarrow 40 \rightarrow \dots \rightarrow 0$



Ortalama $\leq 26,7 \Rightarrow$ hedef $\leq 17,8$. Her turda üst sınır $2/3$ ile çarpılıyor.

Adım 5 / 8

```
1 ust = 90 # olası en büyük tahmin
2 for tur in range(1, 7):
3     # ortalama  $\leq$  ust  $\Rightarrow$  hedef  $\leq (2/3) \cdot$  ust
4     ▶ ust = ust * 2 / 3
5     ▶ print(tur, round(ust, 1))
6 # ust =  $90 \cdot (2/3)^k \rightarrow 0$  : tek denge herkes 0
```

DEĞİŞKENLER

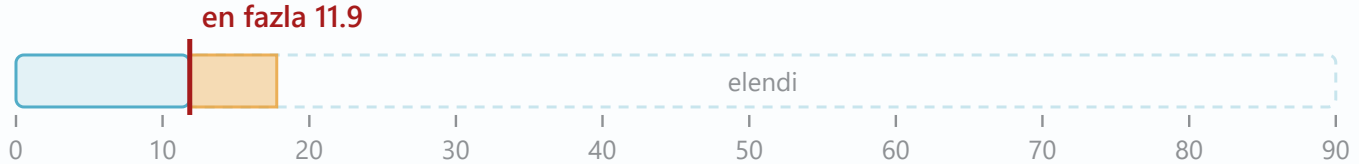
tur = 4

ust = 17.78

ÇIKTI

```
1 60.0
2 40.0
3 26.7
4 17.8
```

Yinelemeli Eleme: $90 \rightarrow 60 \rightarrow 40 \rightarrow \dots \rightarrow 0$



Ortalama $\leq 17,8 \Rightarrow$ hedef $\leq 11,9$. Aralık geometrik olarak küçülüyor.

Adım 6 / 8

```
1 ust = 90 # olası en büyük tahmin
2 for tur in range(1, 7):
3     # ortalama  $\leq$  ust  $\Rightarrow$  hedef  $\leq (2/3) \cdot$  ust
4     ust = ust * 2 / 3
5     print(tur, round(ust, 1))
6 # ust =  $90 \cdot (2/3)^k \rightarrow 0$  : tek denge herkes 0
```

DEĞİŞKENLER

tur = 5

ust = 11.85

ÇIKTI

```
1 60.0
2 40.0
3 26.7
4 17.8
5 11.9
```

Yinelemeli Eleme: $90 \rightarrow 60 \rightarrow 40 \rightarrow \dots \rightarrow 0$



Ortalama $\leq 11,9 \Rightarrow$ hedef $\leq 7,9$. k tur sonra üst sınır $90 \cdot (2/3)^k$.

Adım 7 / 8

```
1 ust = 90 # olası en büyük tahmin
2 for tur in range(1, 7):
3     # ortalama ≤ ust ⇒ hedef ≤ (2/3)·ust
4     ▶ ust = ust * 2 / 3
5     ▶ print(tur, round(ust, 1))
6 # ust = 90·(2/3)^k → 0 : tek denge herkes 0
```

DEĞİŞKENLER

tur = 6

ust = 7.9

ÇIKTI

```
1 60.0
2 40.0
3 26.7
4 17.8
5 11.9
6 7.9
```

Yinelemeli Eleme: $90 \rightarrow 60 \rightarrow 40 \rightarrow \dots \rightarrow 0$



Süreç sonsuza dek sürer ve üst sınır 0'a yakınsar: yinelemeli elemekten sağ çıkan tek profil **herkesin 0 söylemesidir**.

Adım 8 / 8

```
1 ust = 90 # olası en büyük tahmin
2 for tur in range(1, 7):
3     # ortalama ≤ ust ⇒ hedef ≤ (2/3)·ust
4     ust = ust * 2 / 3
5     print(tur, round(ust, 1))
▶ # ust = 90·(2/3)^k → 0 : tek denge herkes 0
```

DEĞİŞKENLER

tur = ...

ust = 0 (limit)

ÇIKTI

```
1 60.0
2 40.0
3 26.7
4 17.8
5 11.9
6 7.9
```

Tek Nash Dengesi: Herkes 0 Söyler

Denge olduğunu doğrula

- Herkes 0 derse ortalama 0, hedef 0: herkes kazanır (ödül paylaşılır).
- Biri $x > 0$ 'a sapsa ortalama x/n , hedef $2x/(3n)$ olur; diğerlerinin 0'ı hedefe daha yakın kalır $\rightarrow$ sapan kaybeder.
- Başka hiçbir profil yinelemeli elemenden sağ çıkmaz: denge **tektir**.

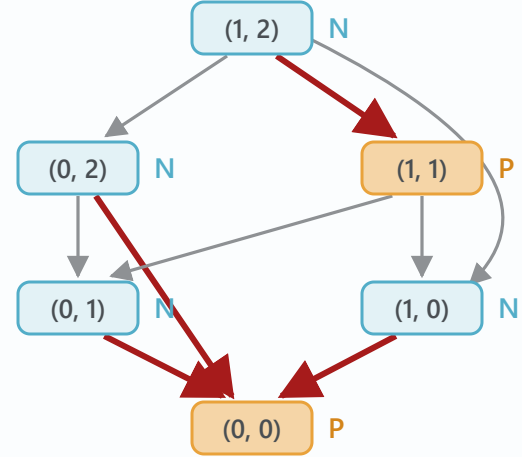
Peki gerçekte?

- Deneylerde kazanan sayı 0 çıkmaz: 1997'de Financial Times okurlarıyla yapılan 0–100 sürümünde kazanan tahmin **13** idi.
- Çünkü herkesin tam rasyonel olduğu ve bunun **ortak bilgi** olduğu varsayımı gerçekte tam tutmaz.

Dikkat: Bu oyunda Nash dengesini belirlemek zor değildir; denge tek ve saf stratejilidir (herkes 0). Zor olan, *gerçek* oyuncular karşısında en iyi tepkiyi tahmin etmektir. Oyunu tekrarlayınca tahminler dengeye doğru iner.

Kombinatoryal Oyunlar ve Zermelo Teoremi

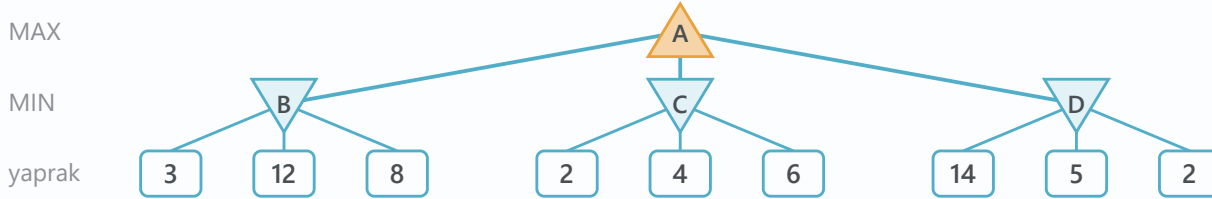
- İki oyuncu **sırayla** oynar; mükemmel bilgi, şans yok, oyun sonlu adımda biter.
- Normal oyun kuralı:** son hamleyi yapan kazanır (Nim; 21'i Söyleme'de ise 21'i söyleyen kaybeder). XOX ve satrançta beraberlik de var.
- Zermelo (1913):** Böyle bir oyunda ya bir taraf kazanmayı ya da iki taraf beraberliği garanti eder.
- Her konum **N** (sıradaki kazanır) ya da **P** (önceki kazanır): P'den her hamle N'ye, N'den en az bir hamle P'ye gider.



P (turuncu): sıradaki kaybeder · kalın ok: kazandıran

Etiketler yapraktan köke hesaplanır: (0, 0)'da hamle yok → P. Bu, bir sonraki slaytta kodlayacağımız minimaksın özel hâlidir.

Kod İzleme: Oyun Ağacında Minimaks



Kök A'da sıra MAX'ta (büyük değer ister); B, C, D'de MIN (küçük ister). Yapraklar oyun sonu kazançları.

Adım 1 / 17

```

1 T = {"A": ["B", "C", "D"], "B": [3, 12, 8],
2     "C": [2, 4, 6], "D": [14, 5, 2]}
3 def minimax(n, maxi):
4     if isinstance(n, int):          # yaprak
5         return n
6     best = -99 if maxi else 99
7     for c in T[n]:
8         v = minimax(c, not maxi)
9         best = max(best, v) if maxi else min(best, v)
10    return best
▶ print(minimax("A", True))
  
```

DEĞİŞKENLER

n = "A"

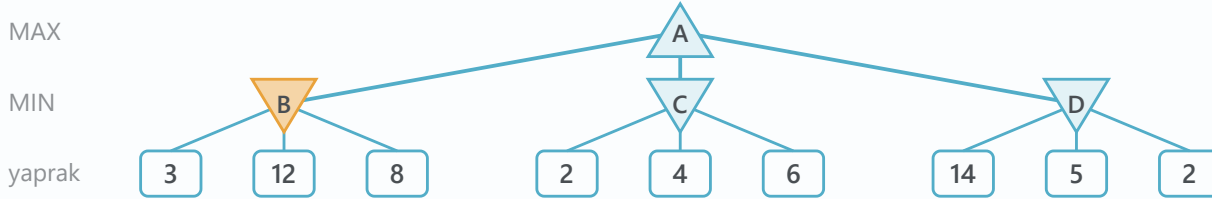
maxi = True

ÇAĞRI YIĞINI

minimax("A", True)

ÇIKTI

Kod İzleme: Oyun Ağacında Minimaks



B bir MIN düğümü: best = 99 ile başla, çocukları sırayla değerlendir.

Adım 2 / 17

```

1 T = {"A": ["B", "C", "D"], "B": [3, 12, 8],
2     "C": [2, 4, 6], "D": [14, 5, 2]}
3 def minimax(n, maxi):
4     if isinstance(n, int):          # yaprak
5         return n
6     best = -99 if maxi else 99
7     for c in T[n]:
8         v = minimax(c, not maxi)
9         best = max(best, v) if maxi else min(best, v)
10    return best
11 print(minimax("A", True))

```

DEĞİŞKENLER

n = "B"

maxi = False

best = 99

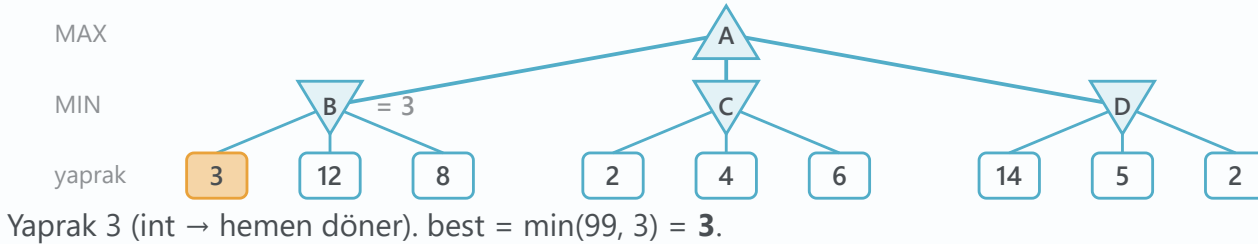
ÇAĞRI YIĞINI

minimax("B", False)

minimax("A", True)

ÇIKTI

Kod İzleme: Oyun Ağacında Minimaks



Adım 3 / 17

```

1 T = {"A": ["B", "C", "D"], "B": [3, 12, 8],
2     "C": [2, 4, 6], "D": [14, 5, 2]}
3 def minimax(n, maxi):
4     if isinstance(n, int):          # yaprak
5         return n
6     best = -99 if maxi else 99
7     for c in T[n]:
8         v = minimax(c, not maxi)
9         best = max(best, v) if maxi else min(best, v)
10    return best
11 print(minimax("A", True))

```

DEĞİŞKENLER

n = "B"

c = 3

v = 3

best = 3

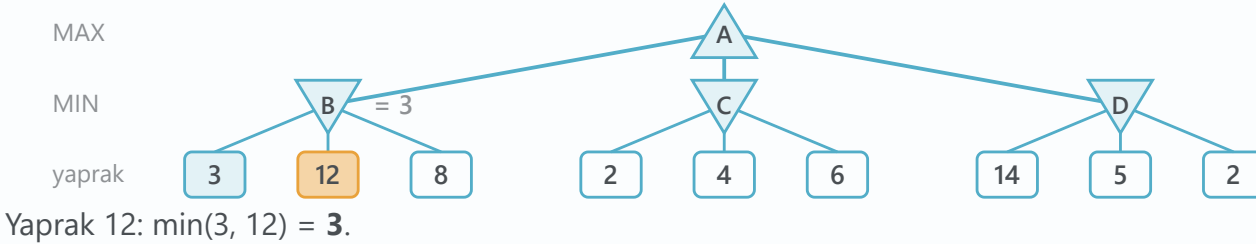
ÇAĞRI YIĞINI

minimax("B", False)

minimax("A", True)

ÇIKTI

Kod İzleme: Oyun Ağacında Minimaks



Adım 4 / 17

```

1 T = {"A": ["B", "C", "D"], "B": [3, 12, 8],
2     "C": [2, 4, 6], "D": [14, 5, 2]}
3 def minimax(n, maxi):
4     if isinstance(n, int):          # yaprak
5         return n
6     best = -99 if maxi else 99
7     for c in T[n]:
8         v = minimax(c, not maxi)
9         best = max(best, v) if maxi else min(best, v)
10    return best
11 print(minimax("A", True))

```

DEĞİŞKENLER

n = "B"

c = 12

v = 12

best = 3

ÇAĞRI YIĞINI

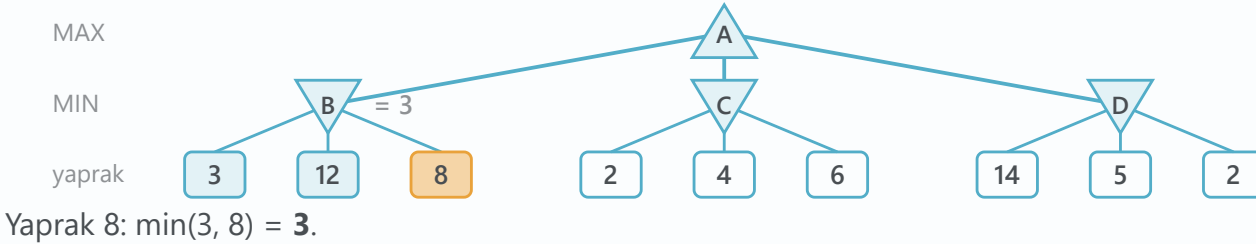
minimax("B", False)

minimax("A", True)

ÇIKTI



Kod İzleme: Oyun Ağacında Minimaks



Adım 5 / 17

```

1 T = {"A": ["B", "C", "D"], "B": [3, 12, 8],
2     "C": [2, 4, 6], "D": [14, 5, 2]}
3 def minimax(n, maxi):
4     if isinstance(n, int):          # yaprak
5         return n
6     best = -99 if maxi else 99
7     for c in T[n]:
8         v = minimax(c, not maxi)
9         best = max(best, v) if maxi else min(best, v)
10    return best
11 print(minimax("A", True))

```

DEĞİŞKENLER

n = "B"

c = 8

v = 8

best = 3

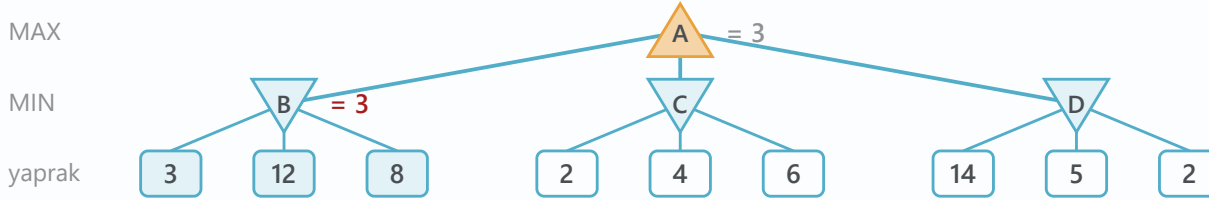
ÇAĞRI YIĞINI

minimax("B", False)

minimax("A", True)

ÇIKTI

Kod İzleme: Oyun Ağacında Minimaks



B değeri 3 olarak döner. A'da: $\max(-99, 3) = 3$.

Adım 6 / 17

```

1 T = {"A": ["B", "C", "D"], "B": [3, 12, 8],
2     "C": [2, 4, 6], "D": [14, 5, 2]}
3 def minimax(n, maxi):
4     if isinstance(n, int):          # yaprak
5         return n
6     best = -99 if maxi else 99
7     for c in T[n]:
8         v = minimax(c, not maxi)
9         best = max(best, v) if maxi else min(best, v)
10    return best
11 print(minimax("A", True))
  
```

DEĞİŞKENLER

`n = "A"`

`maxi = True`

`c = "B"`

`v = 3`

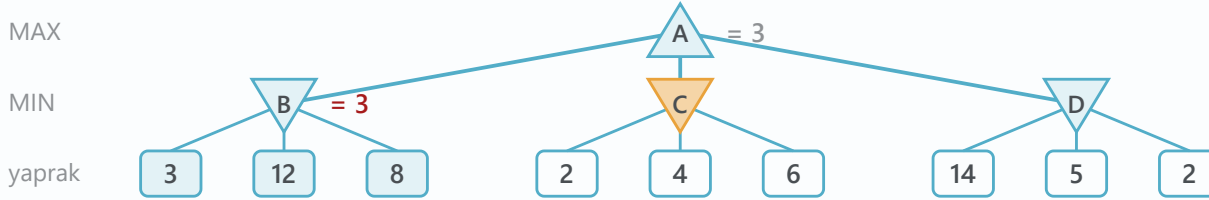
`best = 3`

ÇAĞRI YİĞİNİ

`minimax("A", True)`

ÇIKTI

Kod İzleme: Oyun Ağacında Minimaks



A'nın sıradaki çocuğu C (MIN). Yeniden best = 99.

Adım 7 / 17

```

1 T = {"A": ["B", "C", "D"], "B": [3, 12, 8],
2     "C": [2, 4, 6], "D": [14, 5, 2]}
3 def minimax(n, maxi):
4     if isinstance(n, int):          # yaprak
5         return n
6     best = -99 if maxi else 99
7     for c in T[n]:
8         v = minimax(c, not maxi)
9         best = max(best, v) if maxi else min(best, v)
10    return best
11 print(minimax("A", True))

```

DEĞİŞKENLER

n = "C"

maxi = False

best = 99

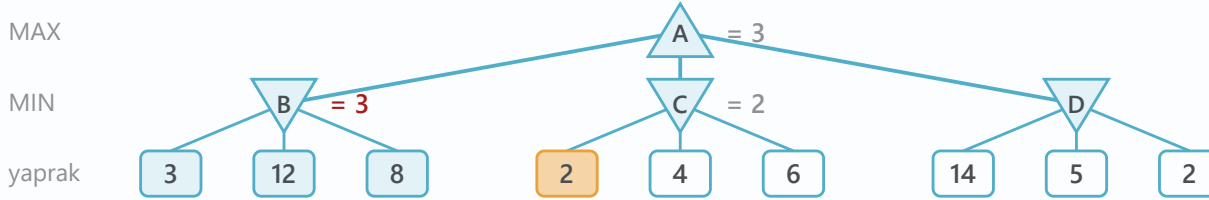
ÇAĞRI YIĞINI

minimax("C", False)

minimax("A", True)

ÇIKTI

Kod İzleme: Oyun Ağacında Minimaks



Yaprak 2 (int → hemen döner). best = min(99, 2) = 2.

Adım 8 / 17

```

1 T = {"A": ["B", "C", "D"], "B": [3, 12, 8],
2     "C": [2, 4, 6], "D": [14, 5, 2]}
3 def minimax(n, maxi):
4     if isinstance(n, int):          # yaprak
5         return n
6     best = -99 if maxi else 99
7     for c in T[n]:
8         v = minimax(c, not maxi)
9         best = max(best, v) if maxi else min(best, v)
10    return best
11 print(minimax("A", True))
  
```

DEĞİŞKENLER

n = "C"

c = 2

v = 2

best = 2

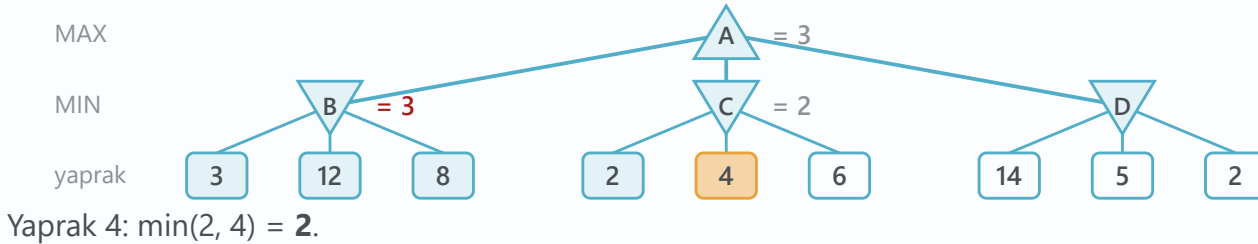
ÇAĞRI YIĞINI

minimax("C", False)

minimax("A", True)

ÇIKTI

Kod İzleme: Oyun Ağacında Minimaks



Adım 9 / 17

```

1 T = {"A": ["B", "C", "D"], "B": [3, 12, 8],
2     "C": [2, 4, 6], "D": [14, 5, 2]}
3 def minimax(n, maxi):
4     if isinstance(n, int):          # yaprak
5         return n
6     best = -99 if maxi else 99
7     for c in T[n]:
8         v = minimax(c, not maxi)
9         best = max(best, v) if maxi else min(best, v)
10    return best
11 print(minimax("A", True))

```

DEĞİŞKENLER

n = "C"

c = 4

v = 4

best = 2

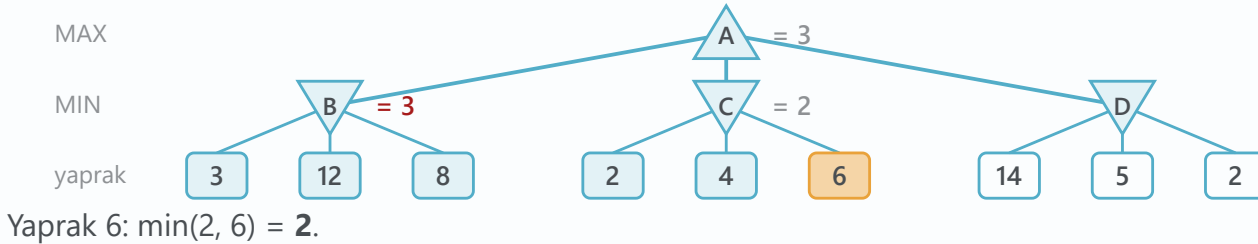
ÇAĞRI YIĞINI

minimax("C", False)

minimax("A", True)

ÇIKTI

Kod İzleme: Oyun Ağacında Minimaks



Adım 10 / 17

```

1 T = {"A": ["B", "C", "D"], "B": [3, 12, 8],
2     "C": [2, 4, 6], "D": [14, 5, 2]}
3 def minimax(n, maxi):
4     if isinstance(n, int):          # yaprak
5         return n
6     best = -99 if maxi else 99
7     for c in T[n]:
8         v = minimax(c, not maxi)
9         best = max(best, v) if maxi else min(best, v)
10    return best
11 print(minimax("A", True))

```

DEĞİŞKENLER

n = "C"

c = 6

v = 6

best = 2

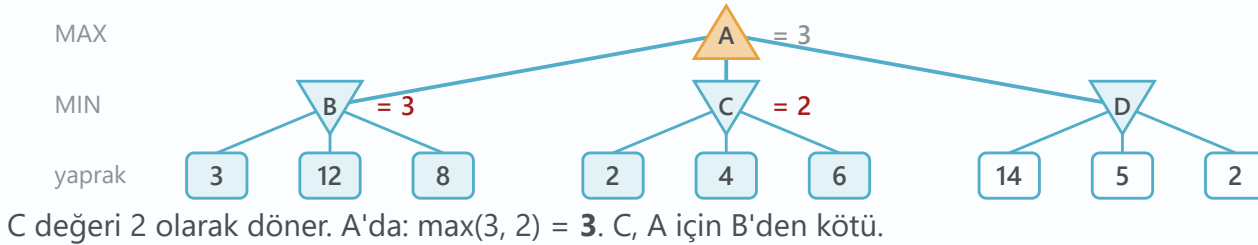
ÇAĞRI YIĞINI

minimax("C", False)

minimax("A", True)

ÇIKTI

Kod İzleme: Oyun Ağacında Minimaks



Adım 11 / 17

```

1 T = {"A": ["B", "C", "D"], "B": [3, 12, 8],
2     "C": [2, 4, 6], "D": [14, 5, 2]}
3 def minimax(n, maxi):
4     if isinstance(n, int):          # yaprak
5         return n
6     best = -99 if maxi else 99
7     for c in T[n]:
8         v = minimax(c, not maxi)
9         best = max(best, v) if maxi else min(best, v)
10    return best
11 print(minimax("A", True))

```

DEĞİŞKENLER

n = "A"

maxi = True

c = "C"

v = 2

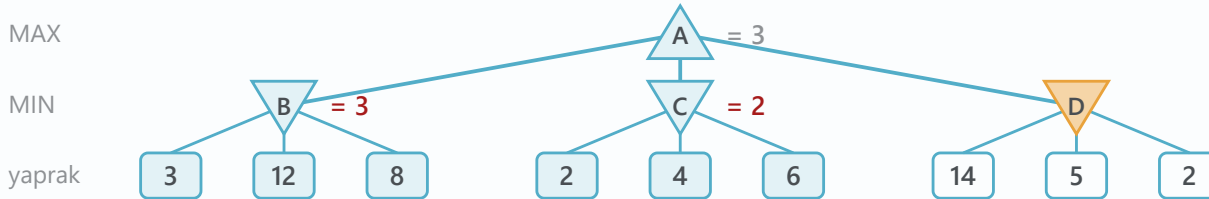
best = 3

ÇAĞRI YIĞINI

minimax("A", True)

ÇIKTI

Kod İzleme: Oyun Ağacında Minimaks



A'nın sıradaki çocuğu D (MIN). Yeniden best = 99.

Adım 12 / 17

```

1 T = {"A": ["B", "C", "D"], "B": [3, 12, 8],
2     "C": [2, 4, 6], "D": [14, 5, 2]}
3 def minimax(n, maxi):
4     if isinstance(n, int):          # yaprak
5         return n
6     best = -99 if maxi else 99
7     for c in T[n]:
8         v = minimax(c, not maxi)
9         best = max(best, v) if maxi else min(best, v)
10    return best
11 print(minimax("A", True))

```

DEĞİŞKENLER

n = "D"

maxi = False

best = 99

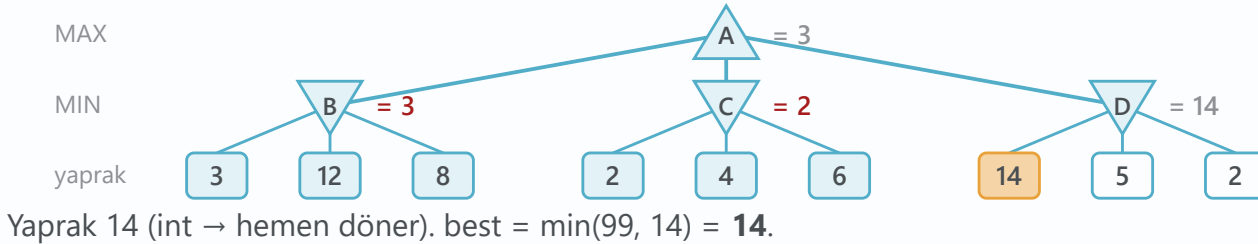
ÇAĞRI YIĞINI

minimax("D", False)

minimax("A", True)

ÇIKTI

Kod İzleme: Oyun Ağacında Minimaks



Adım 13 / 17

```

1 T = {"A": ["B", "C", "D"], "B": [3, 12, 8],
2     "C": [2, 4, 6], "D": [14, 5, 2]}
3 def minimax(n, maxi):
4     if isinstance(n, int):          # yaprak
5         return n
6     best = -99 if maxi else 99
7     for c in T[n]:
8         v = minimax(c, not maxi)
9         best = max(best, v) if maxi else min(best, v)
10    return best
11 print(minimax("A", True))

```

DEĞİŞKENLER

n = "D"

c = 14

v = 14

best = 14

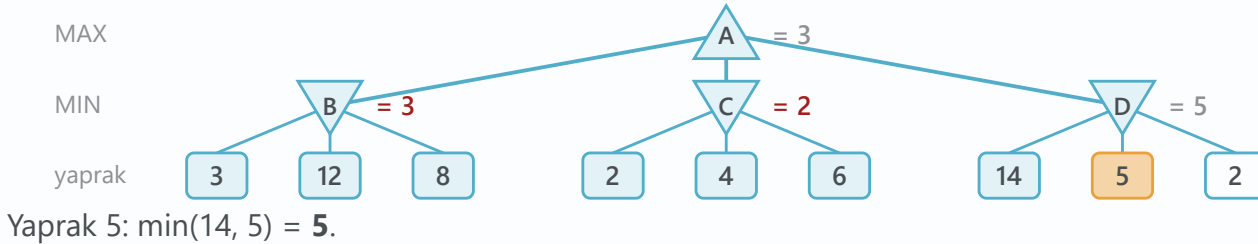
ÇAĞRI YIĞINI

minimax("D", False)

minimax("A", True)

ÇIKTI

Kod İzleme: Oyun Ağacında Minimaks



Adım 14 / 17

```

1 T = {"A": ["B", "C", "D"], "B": [3, 12, 8],
2     "C": [2, 4, 6], "D": [14, 5, 2]}
3 def minimax(n, maxi):
4     if isinstance(n, int):          # yaprak
5         return n
6     best = -99 if maxi else 99
7     for c in T[n]:
8         v = minimax(c, not maxi)
9         best = max(best, v) if maxi else min(best, v)
10    return best
11 print(minimax("A", True))

```

DEĞİŞKENLER

n = "D"

c = 5

v = 5

best = 5

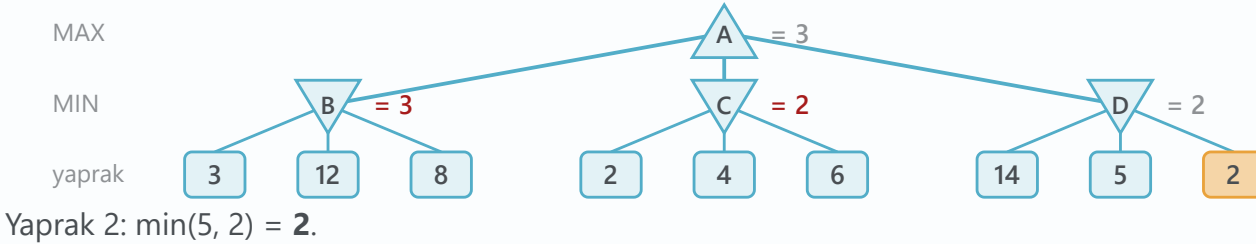
ÇAĞRI YIĞINI

minimax("D", False)

minimax("A", True)

ÇIKTI

Kod İzleme: Oyun Ağacında Minimaks



Adım 15 / 17

```

1 T = {"A": ["B", "C", "D"], "B": [3, 12, 8],
2     "C": [2, 4, 6], "D": [14, 5, 2]}
3 def minimax(n, maxi):
4     if isinstance(n, int):          # yaprak
5         return n
6     best = -99 if maxi else 99
7     for c in T[n]:
8         v = minimax(c, not maxi)
9         best = max(best, v) if maxi else min(best, v)
10    return best
11 print(minimax("A", True))

```

DEĞİŞKENLER

n = "D"

c = 2

v = 2

best = 2

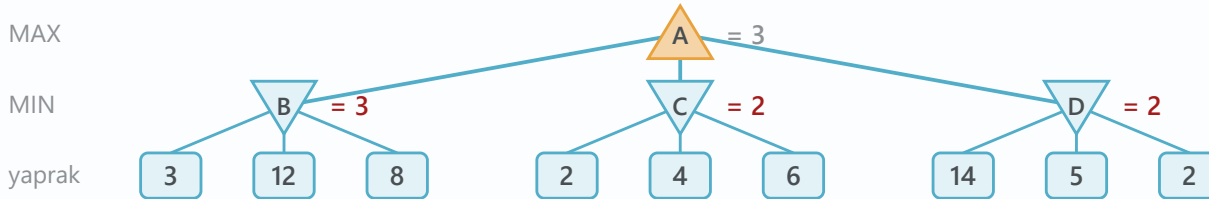
ÇAĞRI YIĞINI

minimax("D", False)

minimax("A", True)

ÇIKTI

Kod İzleme: Oyun Ağacında Minimaks



D değeri 2 olarak döner. A'da: $\max(3, 2) = 3$. D'deki 14 cazip görünse de MIN 2'yi seçer.

Adım 16 / 17

```

1 T = {"A": ["B", "C", "D"], "B": [3, 12, 8],
2     "C": [2, 4, 6], "D": [14, 5, 2]}
3 def minimax(n, maxi):
4     if isinstance(n, int):          # yaprak
5         return n
6     best = -99 if maxi else 99
7     for c in T[n]:
8         v = minimax(c, not maxi)
9         best = max(best, v) if maxi else min(best, v)
10    return best
11 print(minimax("A", True))

```

DEĞİŞKENLER

n = "A"

maxi = True

c = "D"

v = 2

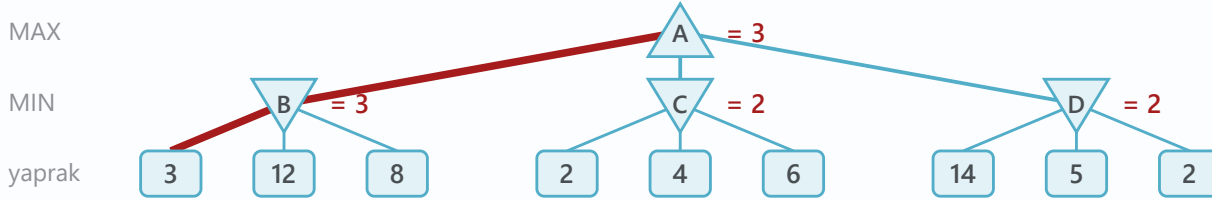
best = 3

ÇAĞRI YIĞINI

minimax("A", True)

ÇIKTI

Kod İzleme: Oyun Ağacında Minimaks



Oyunun değeri 3: MAX B'yi seçer, MIN ne yaparsa yapsın en az 3 garanti. 9 yaprağın hepsine bakıldı.

Adım 17 / 17

```

1 T = {"A": ["B", "C", "D"], "B": [3, 12, 8],
2     "C": [2, 4, 6], "D": [14, 5, 2]}
3 def minimax(n, maxi):
4     if isinstance(n, int):          # yaprak
5         return n
6     best = -99 if maxi else 99
7     for c in T[n]:
8         v = minimax(c, not maxi)
9         best = max(best, v) if maxi else min(best, v)
10    return best
11    print(minimax("A", True))

```

DEĞİŞKENLER

n = "A"

best = 3

ÇAĞRI YIĞINI

boş

ÇIKTI

3

XOX: Tüm Ağaç Aranabilir, Sonuç Beraberlik

- XOX'un ağacı küçük: tüm oyunları sayan program **255.168** farklı oyun ve **5.478** erişilebilir konum bulur.
- Boş tahtada minimaks değeri **0**: iki taraf da hatasız oynarsa oyun **berabere**.
- Sağdaki konumda X yalnız sol alt kareyle kaybetmez: O'nun çaprazını keser, aynı anda iki tehdit kurar (+1).
- Satrançta tam arama imkânsız ($\approx 10^{120}$ oyun): derinlik sınırı, değerlendirme fonksiyonu ve alfa-beta budaması.

X	-1	O
-1	O	-1
+1	-1	X

sıra X'te

+1: X kazanır

-1: O kazanır

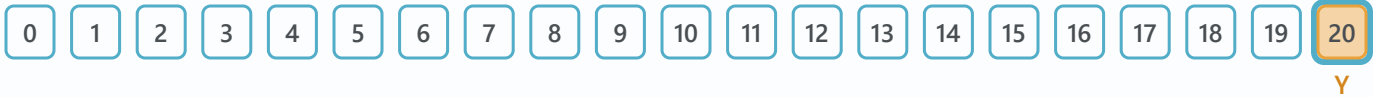
sol alt: O'yu
keser ve çift
tehdit kurar

Boş karelerde: o hamleden sonraki minimaks değeri

Deneyin: bölüm sonundaki [Minimaks XOX](#) rakibi bu değerleri hesaplayarak oynar; onu yenemezsiniz, en iyi ihtimalle berabere kalırsınız.

Kod İzleme: Geriye Doğru Çıkarım — 21'i Söyleme

son söylenen sayı c →



K: sırası gelen kazanır · Y (turuncu): sırası gelen kaybeder, rakibe bırakılacak kontrol sayısı

Taban durum: son söylenen 20 ise sıradaki oyuncu 21'i söylemek zorunda → **kaybeder** (Y).

Adım 1 / 16

```
► HEDEF = 21
► kazanir = {HEDEF - 1: False} # c=20: kayıp
3 for c in range(HEDEF - 2, -1, -1):
4     kazanir[c] = any(not kazanir[c + k]
5         for k in (1, 2, 3) if c + k < HEDEF)
6 kontrol = [c for c in kazanir if not kazanir[c]]
7 print(sorted(kontrol))
```

DEĞİŞKENLER

HEDEF = 21

kazanir[20] = False

ÇIKTI



Kod İzleme: Geriye Doğru Çıkarım — 21'i Söyleme

son söylenen sayı c →



K: sırası gelen kazanır · Y (turuncu): sırası gelen kaybeder, rakibe bırakılacak kontrol sayısı

c = 19: 20 söylenerek rakip Y konumuna itilir → c = 19 **K**.

Adım 2 / 16

```
1 HEDEF = 21
2 kazanir = {HEDEF - 1: False} # c=20: kayıp
▶ for c in range(HEDEF - 2, -1, -1):
▶     kazanir[c] = any(not kazanir[c + k]
▶         for k in (1, 2, 3) if c + k < HEDEF)
6 kontrol = [c for c in kazanir if not kazanir[c]]
7 print(sorted(kontrol))
```

DEĞİŞKENLER

c = 19

kazanir[c] = True

ÇIKTI



Kod İzleme: Geriye Doğru Çıkarım — 21'i Söyleme

son söylenen sayı c →



K: sırası gelen kazanır · Y (turuncu): sırası gelen kaybeder, rakibe bırakılacak kontrol sayısı

c = 18: 20 söylenerek rakip Y konumuna itilir → c = 18 **K**.

Adım 3 / 16

```
1 HEDEF = 21
2 kazanir = {HEDEF - 1: False} # c=20: kayıp
▶ for c in range(HEDEF - 2, -1, -1):
▶     kazanir[c] = any(not kazanir[c + k]
▶         for k in (1, 2, 3) if c + k < HEDEF)
6 kontrol = [c for c in kazanir if not kazanir[c]]
7 print(sorted(kontrol))
```

DEĞİŞKENLER

c = 18

kazanir[c] = True

ÇIKTI



Kod İzleme: Geriye Doğru Çıkarım — 21'i Söyleme

son söylenen sayı c →



K: sırası gelen kazanır · Y (turuncu): sırası gelen kaybeder, rakibe bırakılacak kontrol sayısı

c = 17: 20 söylenerek rakip Y konumuna itilir → c = 17 **K**.

Adım 4 / 16

```
1 HEDEF = 21
2 kazanir = {HEDEF - 1: False} # c=20: kayıp
▶ for c in range(HEDEF - 2, -1, -1):
▶     kazanir[c] = any(not kazanir[c + k]
▶         for k in (1, 2, 3) if c + k < HEDEF)
6 kontrol = [c for c in kazanir if not kazanir[c]]
7 print(sorted(kontrol))
```

DEĞİŞKENLER

c = 17

kazanir[c] = True

ÇIKTI



Kod İzleme: Geriye Doğru Çıkarım — 21'i Söyleme

son söylenen sayı c →



K: sırası gelen kazanır · Y (turuncu): sırası gelen kaybeder, rakibe bırakılacak kontrol sayısı

c = 16: 17, 18, 19 hepsi K (rakip kazanır) → c = 16 **Y**. Yeni kontrol sayısı!

Adım 5 / 16

```

1 HEDEF = 21
2 kazanir = {HEDEF - 1: False} # c=20: kayıp
▶ for c in range(HEDEF - 2, -1, -1):
▶     kazanir[c] = any(not kazanir[c + k]
▶         for k in (1, 2, 3) if c + k < HEDEF)
6 kontrol = [c for c in kazanir if not kazanir[c]]
7 print(sorted(kontrol))

```

DEĞİŞKENLER

c = 16

kazanir[c] = False

ÇIKTI



Kod İzleme: Geriye Doğru Çıkarım — 21'i Söyleme

son söylenen sayı c →



K: sırası gelen kazanır · Y (turuncu): sırası gelen kaybeder, rakibe bırakılacak kontrol sayısı

c = 15: 16 söylenerek rakip Y konumuna itilir → c = 15 **K**.

Adım 6 / 16

```

1 HEDEF = 21
2 kazanir = {HEDEF - 1: False} # c=20: kayıp
▶ for c in range(HEDEF - 2, -1, -1):
▶     kazanir[c] = any(not kazanir[c + k]
▶         for k in (1, 2, 3) if c + k < HEDEF)
6 kontrol = [c for c in kazanir if not kazanir[c]]
7 print(sorted(kontrol))

```

DEĞİŞKENLER

c = 15

kazanir[c] = True

ÇIKTI



Kod İzleme: Geriye Doğru Çıkarım — 21'i Söyleme

son söylenen sayı c →



K: sırası gelen kazanır · Y (turuncu): sırası gelen kaybeder, rakibe bırakılacak kontrol sayısı

c = 14: 16 söylenerek rakip Y konumuna itilir → c = 14 **K**.

Adım 7 / 16

```

1 HEDEF = 21
2 kazanir = {HEDEF - 1: False} # c=20: kayıp
▶ for c in range(HEDEF - 2, -1, -1):
▶     kazanir[c] = any(not kazanir[c + k]
▶         for k in (1, 2, 3) if c + k < HEDEF)
6 kontrol = [c for c in kazanir if not kazanir[c]]
7 print(sorted(kontrol))

```

DEĞİŞKENLER

c = 14

kazanir[c] = True

ÇIKTI



Kod İzleme: Geriye Doğru Çıkarım — 21'i Söyleme

son söylenen sayı c →



K: sırası gelen kazanır · Y (turuncu): sırası gelen kaybeder, rakibe bırakılacak kontrol sayısı

c = 13: 16 söylenerek rakip Y konumuna itilir → c = 13 **K**.

Adım 8 / 16

```
1 HEDEF = 21
2 kazanir = {HEDEF - 1: False} # c=20: kayıp
▶ for c in range(HEDEF - 2, -1, -1):
▶     kazanir[c] = any(not kazanir[c + k]
▶         for k in (1, 2, 3) if c + k < HEDEF)
6 kontrol = [c for c in kazanir if not kazanir[c]]
7 print(sorted(kontrol))
```

DEĞİŞKENLER

c = 13

kazanir[c] = True

ÇIKTI

Kod İzleme: Geriye Doğru Çıkarım — 21'i Söyleme

son söylenen sayı c →



K: sırası gelen kazanır · Y (turuncu): sırası gelen kaybeder, rakibe bırakılacak kontrol sayısı

c = 12: 13, 14, 15 hepsi K (rakip kazanır) → c = 12 **Y**. Yeni kontrol sayısı!

Adım 9 / 16

```

1 HEDEF = 21
2 kazanir = {HEDEF - 1: False} # c=20: kayıp
▶ for c in range(HEDEF - 2, -1, -1):
▶     kazanir[c] = any(not kazanir[c + k]
▶         for k in (1, 2, 3) if c + k < HEDEF)
6 kontrol = [c for c in kazanir if not kazanir[c]]
7 print(sorted(kontrol))

```

DEĞİŞKENLER

c = 12

kazanir[c] = False

ÇIKTI



Kod İzleme: Geriye Doğru Çıkarım — 21'i Söyleme

son söylenen sayı c →



K: sırası gelen kazanır · Y (turuncu): sırası gelen kaybeder, rakibe bırakılacak kontrol sayısı

c = 11...9: aynı örüntü, hepsi bir sonraki Y'ye (12) ulaşabilir → K.

Adım 10 / 16

```

1 HEDEF = 21
2 kazanir = {HEDEF - 1: False} # c=20: kayıp
▶ for c in range(HEDEF - 2, -1, -1):
▶     kazanir[c] = any(not kazanir[c + k]
▶         for k in (1, 2, 3) if c + k < HEDEF)
6 kontrol = [c for c in kazanir if not kazanir[c]]
7 print(sorted(kontrol))

```

DEĞİŞKENLER

c = 9

kazanir[c] = True

ÇIKTI



Kod İzleme: Geriye Doğru Çıkarım — 21'i Söyleme

son söylenen sayı c →



K: sırası gelen kazanır · Y (turuncu): sırası gelen kaybeder, rakibe bırakılacak kontrol sayısı

c = 8: 9, 10, 11 hepsi K (rakip kazanır) → c = 8 **Y**. Yeni kontrol sayısı!

Adım 11 / 16

```

1 HEDEF = 21
2 kazanir = {HEDEF - 1: False} # c=20: kayıp
▶ for c in range(HEDEF - 2, -1, -1):
▶     kazanir[c] = any(not kazanir[c + k]
▶         for k in (1, 2, 3) if c + k < HEDEF)
6 kontrol = [c for c in kazanir if not kazanir[c]]
7 print(sorted(kontrol))

```

DEĞİŞKENLER

c = 8

kazanir[c] = False

ÇIKTI



Kod İzleme: Geriye Doğru Çıkarım — 21'i Söyleme

son söylenen sayı c →



K: sırası gelen kazanır · Y (turuncu): sırası gelen kaybeder, rakibe bırakılacak kontrol sayısı

c = 7...5: aynı örüntü, hepsi bir sonraki Y'ye (8) ulaşabilir → K.

Adım 12 / 16

```

1 HEDEF = 21
2 kazanir = {HEDEF - 1: False} # c=20: kayıp
▶ for c in range(HEDEF - 2, -1, -1):
▶     kazanir[c] = any(not kazanir[c + k]
▶         for k in (1, 2, 3) if c + k < HEDEF)
6 kontrol = [c for c in kazanir if not kazanir[c]]
7 print(sorted(kontrol))

```

DEĞİŞKENLER

c = 5

kazanir[c] = True

ÇIKTI



Kod İzleme: Geriye Doğru Çıkarım — 21'i Söyleme

son söylenen sayı c →



K: sırası gelen kazanır · Y (turuncu): sırası gelen kaybeder, rakibe bırakılacak kontrol sayısı

c = 4: 5, 6, 7 hepsi K (rakip kazanır) → c = 4 **Y**. Yeni kontrol sayısı!

Adım 13 / 16

```

1 HEDEF = 21
2 kazanir = {HEDEF - 1: False} # c=20: kayıp
▶ for c in range(HEDEF - 2, -1, -1):
▶     kazanir[c] = any(not kazanir[c + k]
▶         for k in (1, 2, 3) if c + k < HEDEF)
6 kontrol = [c for c in kazanir if not kazanir[c]]
7 print(sorted(kontrol))

```

DEĞİŞKENLER

c = 4

kazanir[c] = False

ÇIKTI



Kod İzleme: Geriye Doğru Çıkarım — 21'i Söyleme

son söylenen sayı $c \rightarrow$



K: sırası gelen kazanır · Y (turuncu): sırası gelen kaybeder, rakibe bırakılacak kontrol sayısı

$c = 3 \dots 1$: aynı örüntü, hepsi bir sonraki Y'ye (4) ulaşabilir $\rightarrow$ K.

Adım 14 / 16

```

1 HEDEF = 21
2 kazanir = {HEDEF - 1: False} # c=20: kayıp
▶ for c in range(HEDEF - 2, -1, -1):
▶     kazanir[c] = any(not kazanir[c + k]
▶         for k in (1, 2, 3) if c + k < HEDEF)
6 kontrol = [c for c in kazanir if not kazanir[c]]
7 print(sorted(kontrol))

```

DEĞİŞKENLER

$c = 1$

$kazanir[c] = \text{True}$

ÇIKTI



Kod İzleme: Geriye Doğru Çıkarım — 21'i Söyleme

son söylenen sayı $c \rightarrow$

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
Y	K	K	K	Y	K	K	K	Y	K	K	K	Y	K	K	K	Y	K	K	K	Y

K: sırası gelen kazanır · Y (turuncu): sırası gelen kaybeder, rakibe bırakılacak kontrol sayısı

$c = 0$ (oyun başı) Y: ilk oyuncu kaybeder. İkinci oyuncu hep 4, 8, 12, 16, 20'ye tamamlar.

Adım 15 / 16

```

1 HEDEF = 21
2 kazanir = {HEDEF - 1: False} # c=20: kayıp
▶ for c in range(HEDEF - 2, -1, -1):
▶     kazanir[c] = any(not kazanir[c + k]
▶         for k in (1, 2, 3) if c + k < HEDEF)
6 kontrol = [c for c in kazanir if not kazanir[c]]
7 print(sorted(kontrol))

```

DEĞİŞKENLER

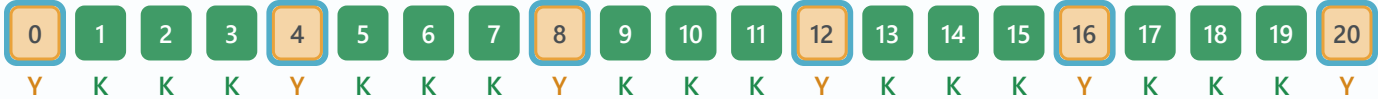
$c = 0$

$kazanir[c] = False$

ÇIKTI

Kod İzleme: Geriye Doğru Çıkarım — 21'i Söyleme

son söylenen sayı c →



K: sırası gelen kazanır · Y (turuncu): sırası gelen kaybeder, rakibe bırakılacak kontrol sayısı

Y konumları 4'ün katları: rakibe hep 4k bırak. Rakip x sayı söylerse sen $4 - x$ söyle.

Adım 16 / 16

```

1 HEDEF = 21
2 kazanir = {HEDEF - 1: False} # c=20: kayıp
3 for c in range(HEDEF - 2, -1, -1):
4     kazanir[c] = any(not kazanir[c + k]
5         for k in (1, 2, 3) if c + k < HEDEF)
▶ kontrol = [c for c in kazanir if not kazanir[c]]
▶ print(sorted(kontrol))

```

DEĞİŞKENLER

`kontrol = [0, 4, 8, 12, 16, 20]`

ÇIKTI

`[0, 4, 8, 12, 16, 20]`

Nim: XOR ile Tam Çözülen Oyun

- **Kurallar:** birkaç jeton yığını; sırası gelen *tek* bir yığından en az bir jeton alır. Son jetonu alan kazanır.
- **Nim toplamı:** yığın boylarının bit bit XOR'u (eldesiz ikili toplama): $3 \oplus 4 \oplus 5 = 2$.
- **Bouton (1901):** Konum kaybedendir (P) $\Leftrightarrow$ nim toplamı 0.
- Neden? XOR 0 iken her hamle bir yığının en az bir bitini değiştirir $\rightarrow$ XOR $\neq$ 0. XOR = $x \neq 0$ iken, x 'in en soldaki 1 bitini taşıyan yığını $h \oplus x$ 'e indirmek XOR'u 0 yapar.

yığın	4	2	1
0: 3 ○○○	0	1	1
1: 4 ○○○○	1	0	0
2: 5 ○○○○○	1	0	1
$\oplus$ (XOR)	0	1	0
	2× 1	1× 1	2× 1

= 2

tek sayıda 1 olan sütun $\rightarrow$ XOR'da 1

Sprague–Grundy teoremi: normal oyun kuralındaki her tarafsız oyun (iki oyuncunun hamleleri aynı) tek bir Nim yığına denktir.

Kod İzleme: Nim'de Kazandıran Hamle (XOR)

yığınlar

0: ○ ○ ○
 1: ○ ○ ○ ○
 2: ○ ○ ○ ○ ○

ikili

4 2 1

3	0	1	1
4	1	0	0
5	1	0	1

her sütunda 1'lerin
sayısı çiftse XOR = 0

XOR

0	0	0	0
---	---	---	---

Konum [3, 4, 5]. Sıra bizde: nim toplamını (bit bit XOR) hesaplayalım.

Adım 1 / 7

```

▶ yiginlar = [3, 4, 5]
▶ x = 0
3 for h in yiginlar:
4     x ^= h           # nim toplamı (XOR)
5 for i, h in enumerate(yiginlar):
6     if h ^ x < h:    # küçültülebilir mi?
7         yiginlar[i] = h ^ x
8         print("yığın", i, ":", h, "->", h ^ x)
9         break
10 print(yiginlar)

```

DEĞİŞKENLER

yiginlar = [3, 4, 5]

x = 0

ÇIKTI

Kod İzleme: Nim'de Kazandıran Hamle (XOR)

yığınlar

0: ○ ○ ○
 1: ○ ○ ○ ○
 2: ○ ○ ○ ○ ○

ikili

4 2 1

3	0	1	1
4	1	0	0
5	1	0	1

her sütunda 1'lerin
sayısı çiftse XOR = 0

XOR

3	0	1	1
----------	----------	----------	----------

$x = 0 \text{ XOR } 3 = \mathbf{3}$ (ikili $000 \oplus 011 = 011$).

Adım 2 / 7

```

1 yiginlar = [3, 4, 5]
2 x = 0
▶ for h in yiginlar:
▶     x ^= h           # nim toplamı (XOR)
5 for i, h in enumerate(yiginlar):
6     if h ^ x < h:     # küçültülebilir mi?
7         yiginlar[i] = h ^ x
8         print("yığın", i, ":", h, "->", h ^ x)
9         break
10 print(yiginlar)
```

DEĞİŞKENLER

yiginlar = [3, 4, 5] h = 3

x = 3

ÇIKTI

Kod İzleme: Nim'de Kazandıran Hamle (XOR)

yığınlar

0: ○ ○ ○
 1: ○ ○ ○ ○
 2: ○ ○ ○ ○ ○

ikili

4 2 1

3	0	1	1
4	1	0	0
5	1	0	1

her sütunda 1'lerin
sayısı çiftse XOR = 0

XOR

7	1	1	1
----------	----------	----------	----------

$x = 3 \text{ XOR } 4 = \mathbf{7}$ (ikili $011 \oplus 100 = 111$).

Adım 3 / 7

```

1 yiginlar = [3, 4, 5]
2 x = 0
▶ for h in yiginlar:
▶     x ^= h           # nim toplamı (XOR)
5 for i, h in enumerate(yiginlar):
6     if h ^ x < h:    # küçültülebilir mi?
7         yiginlar[i] = h ^ x
8         print("yığın", i, ":", h, "->", h ^ x)
9         break
10 print(yiginlar)

```

DEĞİŞKENLER

yiginlar = [3, 4, 5]

h = 4

x = 7

ÇIKTI

Kod İzleme: Nim'de Kazandıran Hamle (XOR)

yığınlar

0: ○ ○ ○
 1: ○ ○ ○ ○
 2: ○ ○ ○ ○ ○

ikili

4 2 1

3	0	1	1
4	1	0	0
5	1	0	1

her sütunda 1'lerin
sayısı çiftse XOR = 0

XOR

2	0	1	0
----------	----------	----------	----------

$x = 7 \text{ XOR } 5 = \mathbf{2}$ (ikili $111 \oplus 101 = 010$).

Adım 4 / 7

```

1 yiginlar = [3, 4, 5]
2 x = 0
▶ for h in yiginlar:
▶     x ^= h           # nim toplamı (XOR)
5 for i, h in enumerate(yiginlar):
6     if h ^ x < h:     # küçültülebilir mi?
7         yiginlar[i] = h ^ x
8         print("yığın", i, ":", h, "->", h ^ x)
9         break
10 print(yiginlar)

```

DEĞİŞKENLER

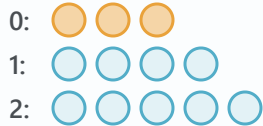
yiginlar = [3, 4, 5] h = 5

x = 2

ÇIKTI

Kod İzleme: Nim'de Kazandıran Hamle (XOR)

yığınlar



ikili

4 2 1

3	0	1	1
4	1	0	0
5	1	0	1

her sütunda 1'lerin
sayısı çiftse XOR = 0

XOR

2	0	1	0
---	---	---	---

Yığın 0: $3 \text{ XOR } 2 = 1 < 3 \rightarrow$ bu yığından 2 jeton alınabilir. (XOR'un en soldaki 1 biti, 3'ün de 1 biti.)

Adım 5 / 7

```

1 yiginlar = [3, 4, 5]
2 x = 0
3 for h in yiginlar:
4     x ^= h           # nim toplamı (XOR)
5     for i, h in enumerate(yiginlar):
6         if h ^ x < h: # küçültülebilir mi?
7             yiginlar[i] = h ^ x
8             print("yığın", i, ":", h, "->", h ^ x)
9             break
10 print(yiginlar)

```

DEĞİŞKENLER

yiginlar = [3, 4, 5] x = 2

i = 0

h = 3

h ^ x = 1

ÇIKTI



Kod İzleme: Nim'de Kazandıran Hamle (XOR)

yığınlar

0: ○ ○ ○
 1: ○ ○ ○ ○
 2: ○ ○ ○ ○ ○

ikili

4 2 1

1 0 0 1

4 1 0 0

5 1 0 1

XOR

0 0 0 0

her sütunda 1'lerin
sayısı çiftse XOR = 0

Yığın 0: 3 → 1. Yeni konum [1, 4, 5]: nim toplamı artık 0.

Adım 6 / 7

```
1 yiginlar = [3, 4, 5]
2 x = 0
3 for h in yiginlar:
4     x ^= h           # nim toplamı (XOR)
5 for i, h in enumerate(yiginlar):
6     if h ^ x < h:    # küçültülebilir mi?
7         yiginlar[i] = h ^ x
8         print("yığın", i, ":", h, "->", h ^ x)
9         break
10 print(yiginlar)
```

DEĞİŞKENLER

yiginlar = [1, 4, 5] x = 2

i = 0 h = 3

ÇIKTI

yığın 0 : 3 -> 1

Kod İzleme: Nim'de Kazandıran Hamle (XOR)

yığınlar

0: ○

1: ○ ○ ○ ○

2: ○ ○ ○ ○ ○

ikili

4

2

1

1

0

0

1

4

1

0

0

5

1

0

1

her sütunda 1'lerin
sayısı çiftse XOR = 0

XOR

0

0

0

0

Rakip XOR = 0 konumunda: her hamlesi toplamı bozar, biz hep yeniden 0 yaparız → son jetonu biz alırız.

Adım 7 / 7

```

1 yiginlar = [3, 4, 5]
2 x = 0
3 for h in yiginlar:
4     x ^= h           # nim toplamı (XOR)
5 for i, h in enumerate(yiginlar):
6     if h ^ x < h:    # küçültülebilir mi?
7         yiginlar[i] = h ^ x
8         print("yığın", i, ":", h, "->", h ^ x)
9         break
▶ print(yiginlar)

```

DEĞİŞKENLER

yiginlar = [1, 4, 5]

x = 2

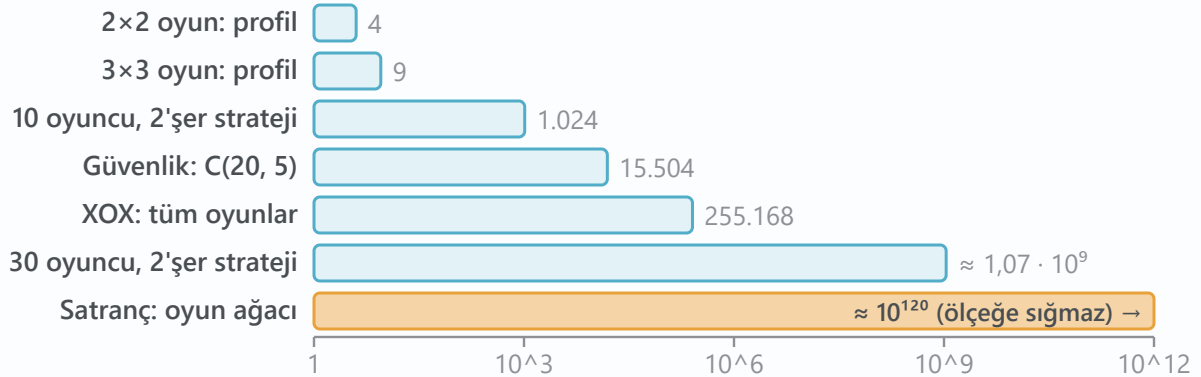
ÇIKTI

```

yığın 0 : 3 -> 1
[1, 4, 5]

```

Algoritmik Oyun Kuramı Neden “Algoritmik”?



Büyüklük

Oyunun tanımı bile üstel büyür: n oyuncu $\times$ 2 strateji $\rightarrow 2^n$ profil. Kaba kuvvet tarama ölçeklenmez.

Karmaşıklık

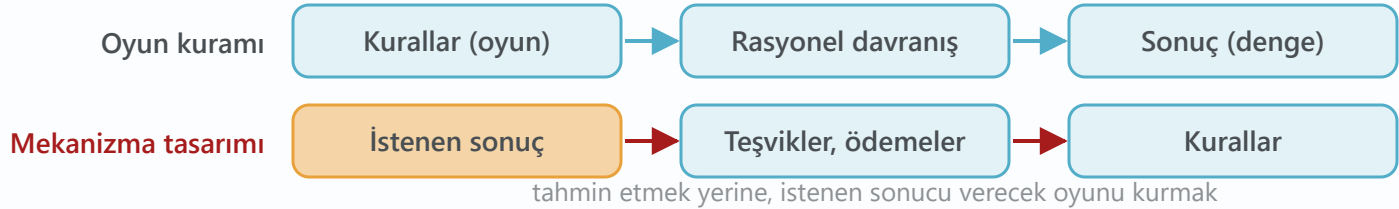
Nash dengesi hep var, ama bulmak iki oyunculu oyunlarda bile PPAD-tam (Daskalakis–Goldberg–Papadimitriou; iki oyuncu: Chen–Deng, 2006). Bölüm 3.

Yapı ve tasarım

Özel yapı verimlilik getirir: sıfır toplamı da doğrusal programlama, Nim'de XOR. Kurallar da algoritmadır (Bölüm 4–5, 11).

Mekanizma Tasarımı: Tersine Oyun Kuramı

- Oyun kuramı verilen bir oyunun sonucunu tahmin eder; **mekanizma tasarımı** istenen sonucu verecek kuralları ve teşvikleri tasarlar.
- Zorluk: Ajanların özel bilgisi (değerleme, konum, tercih) tasarımcıya kapalıdır; ajanlar yalan söyleyebilir.



Para ile

Tasarımcı ödeme alabilir; ödemeler doğru söylemeyi teşvik eder.

açık artırmalar, reklam yerleşimi

Para olmadan

Teşvikler yalnızca kuralla sağlanır; genellikle hedeften biraz ödün verilir.

seçimler, böbrek takası, tesis yeri

Örnek: Bir Nesneyi Kime Verelim?

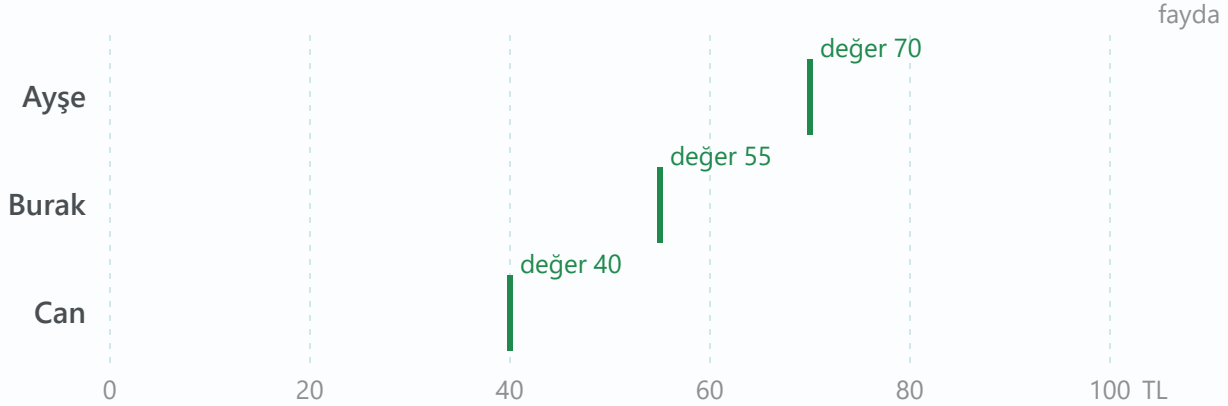
- **Amaç:** Nesne ona en çok değer verene gitsin (verimli tahsis).
- Sorun: Değerlemeler özeldir; katılımcılar nesneyi farklı değerlendirir ve söylediklerinin doğru olduğu garanti değildir.
- **Kural 1:** Herkes değerini söylesin, en yüksek söyleyen alsın (ödeme yok).
- **Kural 2:** Herkes değerini söylesin, en yüksek söyleyen alsın ve **söylediğini ödesin** (birinci fiyat).

Soru

Herkesin *gerçek* değerini söylemesini nasıl sağlarız?

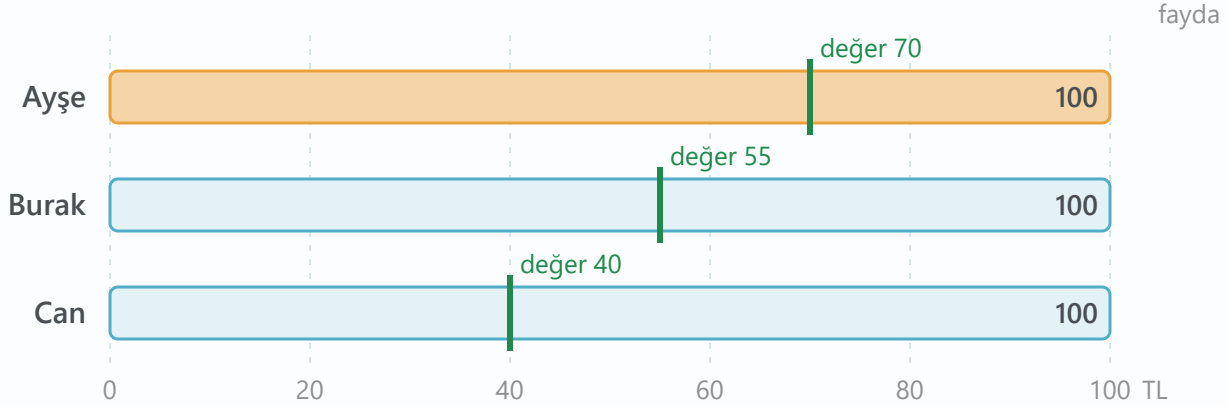
Cevap sonraki slaytta, üç teklifçi üzerinde adım adım.

Cevap: İkinci Fiyat (Vickrey) Açık Artırması



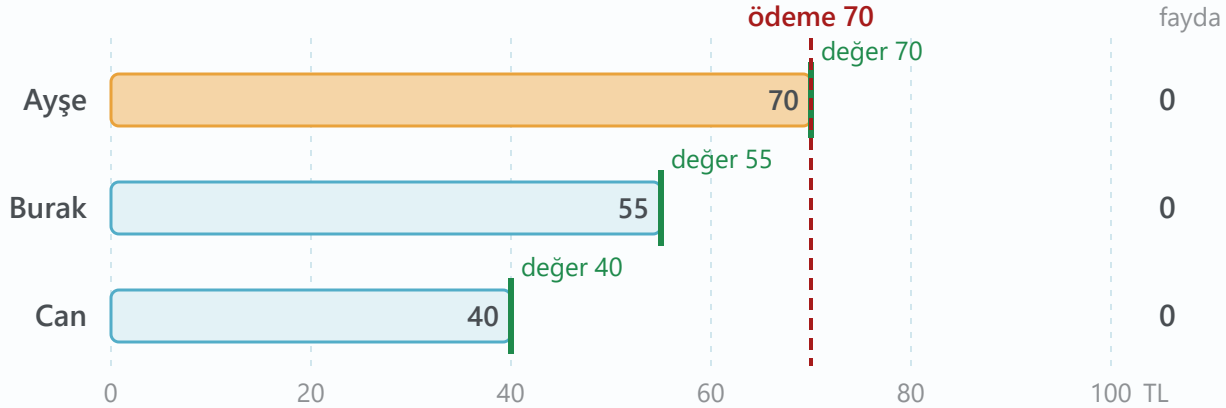
Her teklifçi nesneye verdiği değeri (yeşil çizgi) yalnızca kendisi bilir. Amaç: nesne en çok değer verene, yani Ayşe'ye gitsin.

Cevap: İkinci Fiyat (Vickrey) Açık Artırması



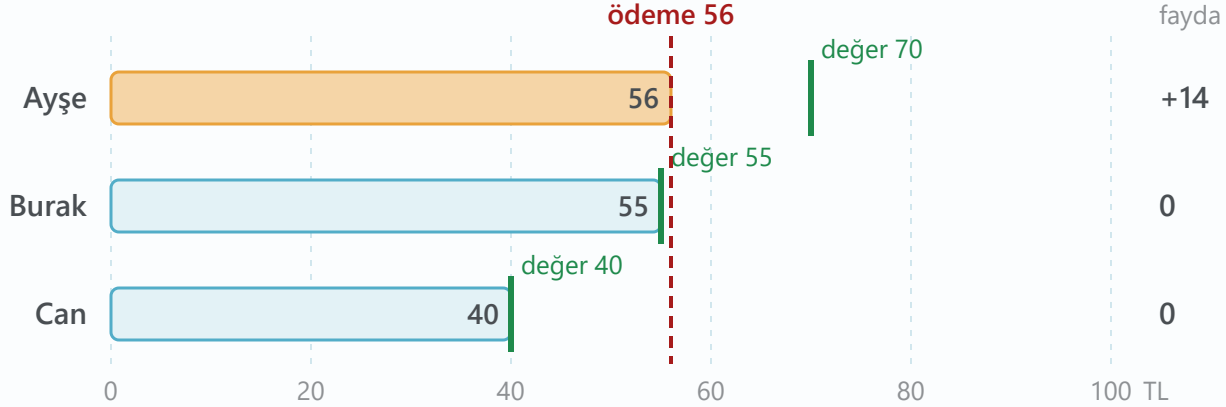
Kural 1 (en yüksek teklif kazanır, ödeme yok): herkes olabildiğince abartır. Teklifler hiçbir bilgi taşımaz.

Cevap: İkinci Fiyat (Vickrey) Açık Artırması



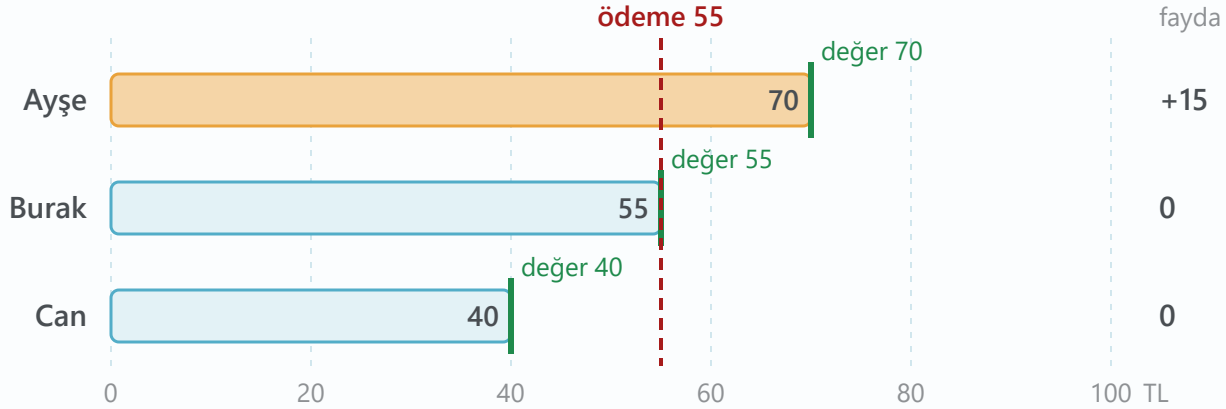
Kural 2 (birinci fiyat: kazanan kendi teklifini öder): Ayşe dürüst 70 derse 70 öder, faydası $70 - 70 = 0$.

Cevap: İkinci Fiyat (Vickrey) Açık Artırması



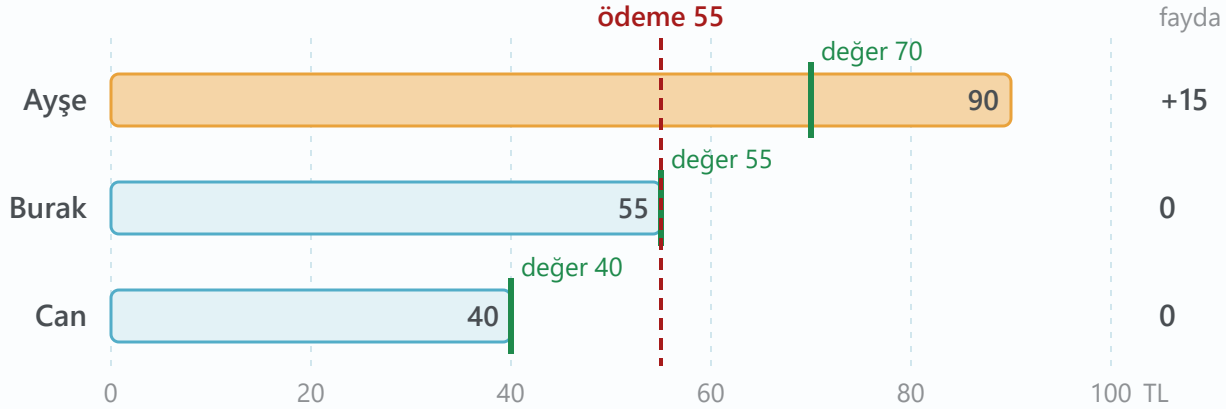
Bu yüzden Ayşe teklifini **kırpar**: 56 der, yine kazanır, fayda 14. Ama ne kadar kırpacağı başkalarının teklifine bağlı; dürüstlük kaybolur.

Cevap: İkinci Fiyat (Vickrey) Açık Artırması



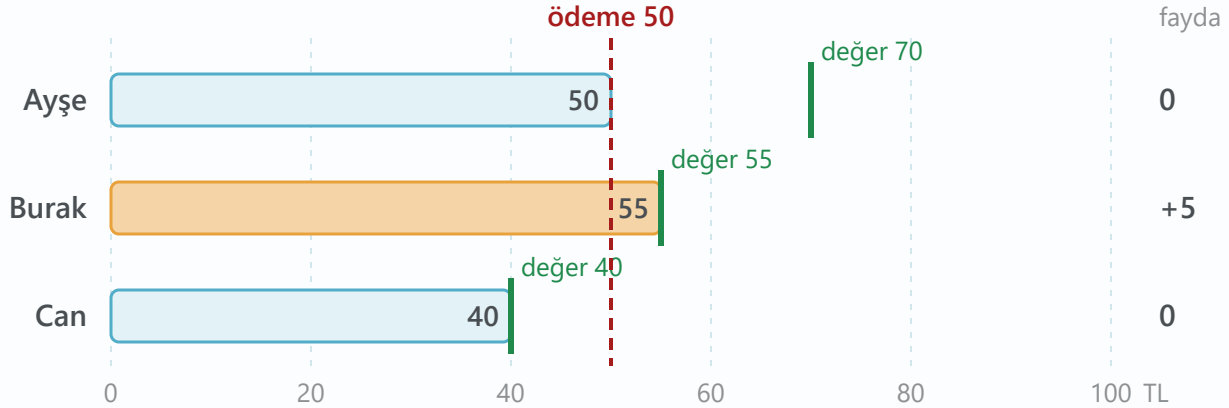
İkinci fiyat (Vickrey): en yüksek teklif kazanır ama **ikinci en yüksek teklifi** öder. Herkes dürüst: Ayşe kazanır, 55 öder, fayda 15.

Cevap: İkinci Fiyat (Vickrey) Açık Artırması



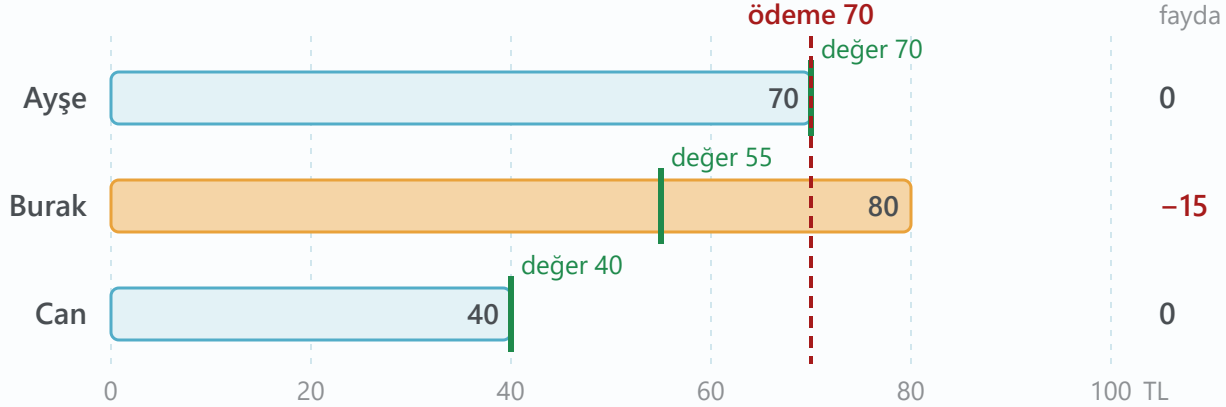
Ayşe abartıp 90 derse: yine kazanır, yine 55 öder. Ödeme kendi teklifine bağlı değil; fayda değişmez (15).

Cevap: İkinci Fiyat (Vickrey) Açık Artırması



Ayşe kırpıp 50 derse: Burak kazanır (50 öder). Ayşe'nin faydası 15'ten 0'a düşer. Kırpmak zarar verebilir.

Cevap: İkinci Fiyat (Vickrey) Açık Artırması



Burak abartıp 80 derse: kazanır ama 70 öder, fayda $55 - 70 = -15$. Kimse yalan söyleyerek faydasını artıramaz.

Örnek: Hastane Nereye Kurulmalı?

- Bir yol boyunca yaşayan ajanlar var; tek bir hastane kurulacak.
- **Maliyet:** Her ajan için hastaneye uzaklık. Ajanlar maliyetlerini en aza indirmek ister.
- **Amaç:** Toplam maliyeti (uzaklıkların toplamını) en aza indirmek.
- **Kısıt:** Parasal işlem yok; ajanlara ödeme yapılamaz, onlardan para alınamaz.
- Ajanlar konumlarını kendileri bildiriyor: yalan söyleyerek hastaneyi kendilerine çekebilirler mi?

Soru

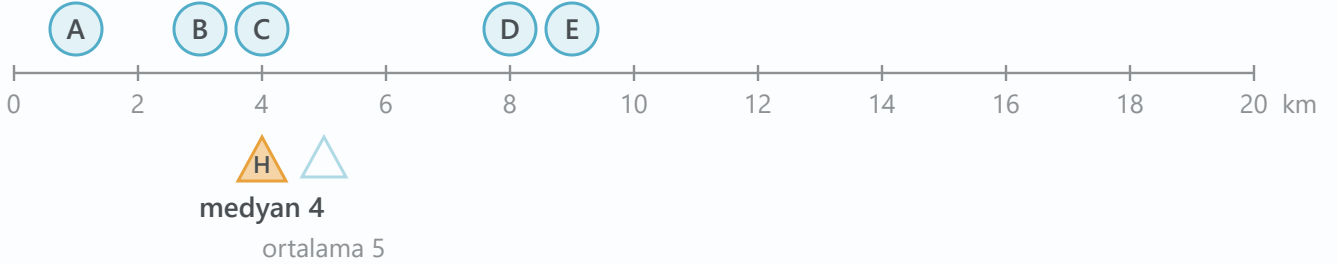
Bildirilen konumlardan hastane yerini seçen hangi kural hem toplam maliyeti en aza indirir hem de yalanı kârsız kılar?

Cevap: Medyan Mekanizması



Beş ajan (ev) bir yol üzerinde: A=1, B=3, C=4, D=8, E=9 km. Hastane nereye? Her ajanın maliyeti hastaneye uzaklığı; para yok.

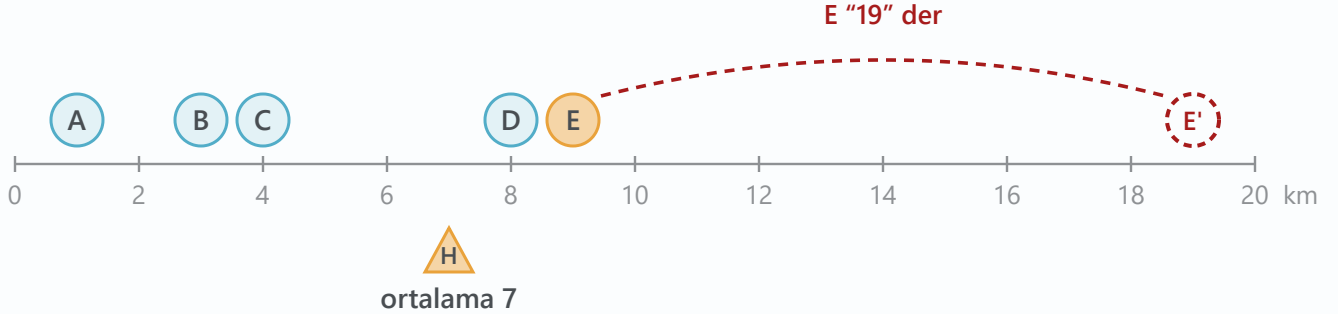
Cevap: Medyan Mekanizması



Toplam uzaklık: medyanda $3+1+0+4+5 = 13$, ortalamada 14

Toplam uzaklığı en aza indiren nokta **medyan**dır (4 km, toplam 13). Ortalama (5 km) toplamı 14'e çıkarır.

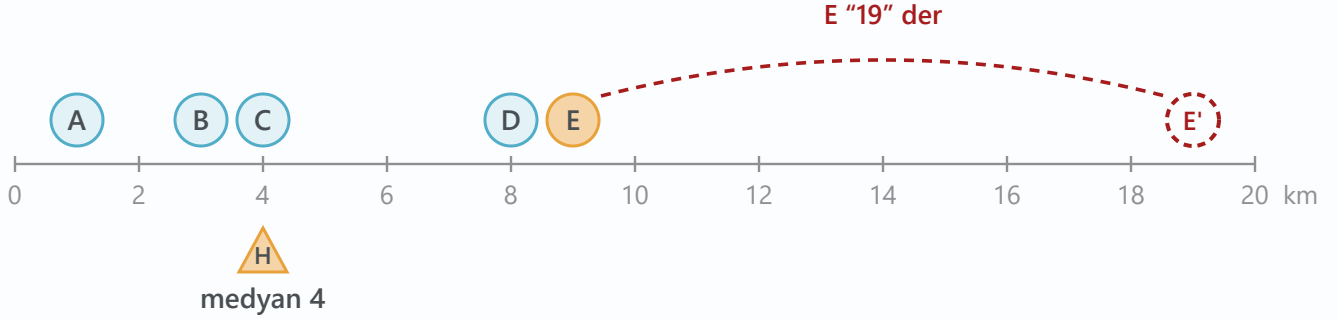
Cevap: Medyan Mekanizması



E'nin maliyeti: dürüstken $|9 - 5| = 4$, yalanla $|9 - 7| = 2$

"Bildirilen konumların ortalaması" kuralında E, 19 km'de oturduğunu söylerse hastane 7'ye kayar: E'nin maliyeti 4'ten 2'ye düşer. **Yalan kârlı.**

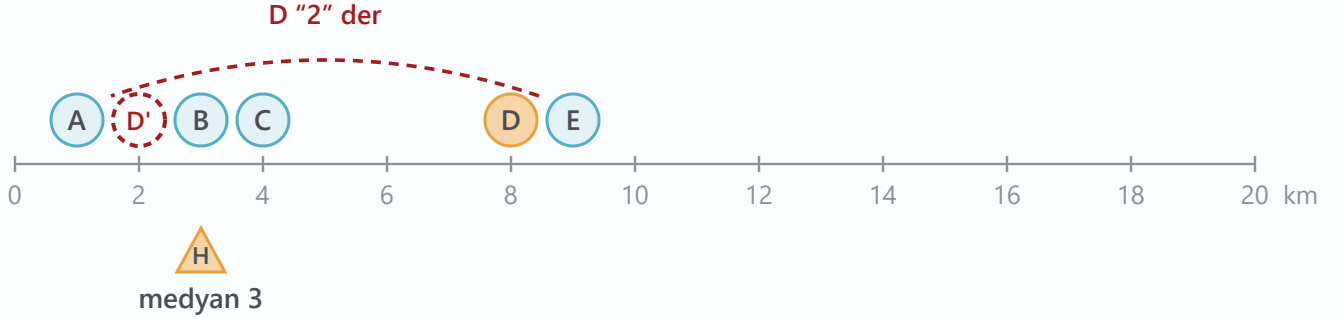
Cevap: Medyan Mekanizması



E'nin maliyeti: $|9 - 4| = 5$, yalan söylese de söylemese de

Medyan kuralında aynı yalan hiçbir şeyi değiştirmez: sıralamada ortadaki değer hâlâ 4. E'nin kazancı yok.

Cevap: Medyan Mekanizması



D'nin maliyeti: dürüstken $|8 - 4| = 4$, yalanla $|8 - 3| = 5$

D medyanı kendine çekemez: sağa kaysa medyan değişmez, sola kaysa (2 km) medyan 3'e iner ve D'den uzaklaşır. Doğru söylemek **baskın strateji**dir.

Parasız Mekanizma Tasarımı Neyi Hedefler?

- Doğru söyletmenin (teşvik uyumluluğu) ötesinde **adalet**, **istikrar** ve **verimlilik** hedeflenir.
- Genel bir imkânsızlık sonucu (Gibbard–Satterthwaite, Bölüm 6) nedeniyle çoğu zaman hedeften biraz ödün vermek gerekir; tek boyutlu tesis yerleşiminde medyan bu engeli aşar.

Kararlı eşleştirme

Öğrenci-okul, doktor-hastane yerleştirmesi (Gale–Shapley).

Bölüm 4

Oylama

Tercihleri bir toplumsal karara dönüştürmek.

Bölüm 6

Adil bölüşüm

Pasta kesme, kaynak paylaşımı, böbrek takası zincirleri.

Bölüm 10

Gerçek Dünya Uygulamaları

Arama reklamları

Reklam alanları her aramada **genelleştirilmiş ikinci fiyat** (GSP) açık artırmasıyla dağıtılır. Çok alanlı GSP'de dürüst teklif baskın değildir.

Bölüm 5, 9

Spektrum açık artırmaları

Devletler frekans lisanslarını tasarlanmış açık artırmalarla satar (ABD'de 1994'ten beri).

Bölüm 5

Okul yerleştirme

Gale–Shapley algoritmasında öğrencinin gerçek tercihlerini yazması en iyisidir; sonuç kararlı bir eşleştirmedir.

Bölüm 4

Böbrek takası

Uyumsuz donör–hasta çiftleri takas döngüleri ve zincirleriyle eşleştirilir; para kullanılmaz.

Bölüm 4

Trafik ve yönlendirme

Her sürücü kendi en hızlı yolunu seçer; bencil seçimlerin toplam gecikmeye maliyeti Anarşinin Bedeli ile ölçülür.

Bölüm 8, 11

Platformlar ve teşvikler

Yoğunluk fiyatlandırması sürücülerini yoğun saatlere çeker; Bitcoin madencileri işlemleri ödül için doğrular.

Bölüm 9

Ortak fikir: kurallar öyle tasarlanır ki herkes **kendi çıkarına** davranırsa bile sistem iyi çalışır.

Özet: Bölümün Kavram Haritası

Kavram	Kısa tanım	Bu bölümdeki örnek
Normal form	Oyuncular, stratejiler, fayda fonksiyonları; matris	TKM, fiyat rekabeti
Genişletilmiş form	Sıra ve bilgiyi gösteren oyun ağacı	Satranç, fiyat ağacı
Baskın strateji	Karşı taraf ne yaparsa yapsın daha iyi	İhanet; 100 TL
Nash dengesi	Herkes en iyi tepkide; tek taraflı sapma kârsız	(İhanet, İhanet); herkes 0
Yinelemeli eleme	Baskılanan stratejileri tekrar tekrar sil	3×3 oyun $\rightarrow (1, 2)$; $2/3$ oyunu $\rightarrow 0$
Karışık strateji	Olasılıkla seçim; dengede kayıtsızlık	TKM $1/3-1/3-1/3$; devriye $p = 2/3$
Pareto optimallik	Kimseyi kötüleştirmeden iyileştirilemez	Denge $(-2, -2)$ değil
Minimaks, geriye doğru çıkarım	Yapraktan köke en iyi seçim; P/N konumları	Ağaç değeri 3; XOX berabere; $21 \rightarrow 4$ 'ün katları; Nim XOR
Mekanizma tasarımı	Doğru söyletecek kuralı tasarlamak	İkinci fiyat; medyan

Örnek Problemler (1/2)

1. Yandaki oyunda baskın strateji var mı? Saf Nash dengelerini en iyi tepki işaretlemesiyle bulun.
2. Fiyat rekabeti oyununda baskın stratejiler ve denge nedir? Denge Pareto optimal mi?
3. Oyun 0–100 aralığında ve hedef "ortalamanın yarısı" olsaydı yinelemeli eleme nasıl ilerlerdi, denge ne olurdu?

	Sol	Sağ
Üst	(4, 3)	(1, 2)
Alt	(3, 1)	(2, 4)

► Cevapları göster

Örnek Problemler (2/2)

4. TKM'de rakip yarı yarıya Taş ve Kağıt oynuyor (hiç Makas yok). En iyi tepkiniz ve beklenen kazancınız nedir?
5. İkinci fiyat açık artırmasında değerler 30, 50 ve 45; herkes dürüst teklif veriyor. Kim kazanır, ne öder, faydası nedir?
6. Ajanlar 2, 5, 6, 12, 20 km'de. Medyan ve ortalama kurallarında toplam uzaklık nedir? 20'deki ajan 100 derse ne olur?

► Cevapları göster

Alıştırmalar (1/2)

1. Yandaki 3×3 oyunda kesin baskılanan stratejileri yinelemeli olarak silin. Ne kalır? Saf Nash dengesiyle karşılaştırın.
2. Nim konumu $[2, 5, 6]$. Sıra sizde: kazanan konum mu? Kazandıran hamle hangisi?
3. MAX kökünün üç MIN çocuğu var: $B = [7, 2, 9]$, $C = [4, 6]$, $D = [1, 8]$. Oyunun değeri ve MAX'ın seçimi nedir?

	0	1	2
0	(2, 5)	(1, 3)	(3, 1)
1	(5, 2)	(3, 3)	(1, 2)
2	(0, 0)	(5, 5)	(0, 0)

Hücre (u_1, u_2) ; satır i , sütun j .

► Cevapları göster

Alıştırmalar (2/2)

4. "16'yı Söyleme": sırayla 1–3 ardışık sayı söylenir, 16'yı söyleyen kaybeder. Kontrol sayıları nedir? Kim kazanır?
5. Sıfır toplamı $A = \begin{bmatrix} 3 & 1 & 4 \\ 2 & 0 & 1 \\ 5 & 2 & 3 \end{bmatrix}$ ve $B = \begin{bmatrix} 0 & 2 \\ 3 & 1 \end{bmatrix}$ matrislerinde maksimin ve minimaksı bulun. Hangisinde eyer noktası var?
6. 2/3 oyununda (0–90) sınıfın %20'si seviye 0, %80'i seviye 1 düşünüyorsa hangi seviye kazanır?

► Cevapları göster

Etkileşimli Oyunlar

2/3 Ortalaması Oyunu

Tahmininizi yapın;
yinelemeli elemeyi
kendiniz yaşayın.

Minimaks XOX

Küçük bir mükemmel
bilgili oyunun tüm
ağacını arayan rakip.

Nim

XOR ile tamamen
çözülen sıralı oyun.

21'i Söyleme

Kaybettiren sayıdan
geriye doğru akıl
yürütme.

Mahkûmun İkilemi

Sessiz mi, ihanet mi?
Simülatörde deneyin.

Taş-Kağıt-Makas

Örüntülü oynarsanız
bilgisayar sizi sömürür
mü?

Kaynaklar (1/2)

- Nisan, Roughgarden, Tardos ve Vazirani (ed.), *Algorithmic Game Theory*, Cambridge University Press, 2007.
- Roughgarden, *Twenty Lectures on Algorithmic Game Theory*, Cambridge University Press, 2016.
- Osborne, *An Introduction to Game Theory*, Oxford University Press, 2004.
- Shoham ve Leyton-Brown, *Multiagent Systems*, Cambridge University Press, 2009.
- Nash, "Equilibrium points in n-person games", *PNAS* 36(1), 1950.
- von Neumann, "Zur Theorie der Gesellschaftsspiele", *Mathematische Annalen* 100, 1928.
- Bouton, "Nim, a game with a complete mathematical theory", *Annals of Mathematics* 3, 1901–1902.
- Nagel, "Unraveling in guessing games: An experimental study", *American Economic Review* 85(5), 1995.
- Tambe, *Security and Game Theory*, Cambridge University Press, 2011.

Kaynaklar (2/2): Ek Kitaplar, Ders Sayfaları, Araçlar

Ek kitaplar

- Karlin ve Peres, *Game Theory, Alive*, American Mathematical Society, 2017. Kombinatoryal oyunlar, Nim, sıfır toplamli oyunlar.
- Easley ve Kleinberg, *Networks, Crowds, and Markets*, Cambridge University Press, 2010. Oyunlar, ağlar ve piyasalar.
- Hearn ve Demaine, *Games, Puzzles, and Computation*, A K Peters, 2009. Oyunların hesaplama karmaşıklığı.

Ders sayfaları

- [Roughgarden, CS364A](#) (Stanford): Twenty Lectures kitabının dersi.
- [Shah, CSC304](#) (Toronto) · [Procaccia, 15-896](#) (CMU)
- [Dinitz, AGT](#) (Johns Hopkins) · [Hoefer, AGT](#) (Frankfurt)

Araçlar ve video

- [Gambit](#): oyun ağaçları ve matris oyunlarında denge hesaplayan açık kaynak araçlar.
- [Nashpy](#): iki oyunculu oyunlar için Python kütüphanesi.
- [Game Theory 101](#) (YouTube oynatma listesi).

Son

Bölüm 1: Oyun Kuramı Temelleri

Sonraki bölüm: [Bölüm 2: Oyun Kuramına Giriş](#)

Çalışma soruları: [Bölüm 1 testi](#) · [Terim sözlüğü](#)